THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)
(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC,
Approved by AICTE, New Delhi)
Villupuram - 605 403

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms.**AKSHAYA M** (**20524UCSC074)** a student of III B.Sc. Computer Science , has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course Incharge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** - CREATE - ALTER - DROP - TRUNCATE		
2	**DML Commands** - INSERT - UPDATE - DELETE		
3	**DQL Commands** - SELECT - WHERE - ORDER BY - GROUP BY - HAVING		
4	**Transactions** - COMMIT - ROLLBACK - SAVEPOINT		
5	**Joins** - Inner Join - Left Join - Right Join - Full Join - Self Join		
6	**SQL Functions** - Aggregate Functions - String Functions - Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**PL/SQL Exception Handling**		
12	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
13	**Mini Project**		
	Annexures: **Khan Academy** **Hacker Rank** **Code Chef**		

EX. NO:01 Implementation of DDL Commands – Students Marks

DATE:4.8.2026

Aim

To implement and understand the DDL commands CREATE, ALTER, DROP, and TRUNCATE using a Students Marks database table.

Software Used

Oracle /MySQL/sqlite3

Description

DDL (Data Definition Language), it is used to create and modify the structure of database objects in database

DDL Commands

- CREATE
- ALTER
- DROP
- TRUNCATE

CREATE

The CREATE command is used to create a new table with constraints.

Syntax

```
CREATE TABLE table_name (
    column1 datatype constraints,
    column2 datatype constraints,
    column3 datatype constraints
);
```

Table Structure

```
CREATE TABLE STUDENT_MARKS (
STUDENT_ID NUMBER PRIMARY KEY,
STUDENT_NAME VARCHAR2(50) NOT NULL,
TAMIL NUMBER(3)
```

ENGLISH NUMBBER(3)

MATHS NUMBER(3)

SCIENCE NUMBER(3)

SOCIAL NUMBER(3)

);

Table Created

ALTER

The ALTER command is used to modify the structure of a table.

ADD

Syntax

ALTER TABLE table_name ADD column_name datatype;

Query:

SQL> ALTER TABLE STUDENT_MARK ADD TOTAL NUMBER(4);

Table Altered

Syntax:

ALTER TABLE table_name MODIFY column_name datatype;

Query:

SQL>ALTER TABLE STUDENT_MARK MODIFY STUDENT_NAME VARCHAR2(100);

TABLE customer MODIFY customer_name VARCHAR(150);

(MySQL / Oracle)

Or (SQL Server)

ALTER TABLE

ALTER COLUMN STUDENTS_MARKSVARCHAR(1Syntax:

ALTER TABLE table_name MODIFY column_name datatype;

Query:

SQL>ALTER TABLE STUDENT_MARK MODIFY STUDENT_NAME VARCHAR2(100)50);

DROP:Drop command is used to delete the attribute/column in a table.

Syntax:

ALTER TABLE table_name DROP COLUMN column_name;

Query:

SQL> ALTER TABLE STUDENT_MARK DROP COLUMN SOCIAL;

RENAME:Rename command is used to re-change the table name.

SQL> RENAME old_table_name TO new_table_name;

Table renamed.

ALTER TABLE STUDENT_MARKS

RENAME COLUMN STUDENT_NAME TO NAME;

TRUNCATE: Truncate command is used to delete a value in the table.

Syntax: TRUNCATE TABLE table_name;

Query:

TRUNCATE TABLE STUDENT_MARKS;

DROP: Deletes the table and all its data permanently.

Syntax:DROP TABLE table_name;

Query:

DROP TABLE STUDENT_MARKS;

Explanation

- Create a table named STUDENT_MARK.
- Insert student details and marks into the table.

;

Create Table

```
CREATE TABLE STUDENT_MARK (
 STUDENT_ID NUMBER PRIMARY KEY,
 STUDENT_NAME VARCHAR2(50) NOT NULL
TAMIL NUMBER(3),
ENGLISH NUMBER(3),
 MATHS NUMBER(3),
SCIENCE NUMBER(3),
SOCIAL NUMBER(3)
);
NSERT INTO STUDENT_MARK VALUES (101, 'Akshaya', 85, 90, 88, 92, 87);
INSERT INTO STUDENT_MARK VALUES (102, 'Keerthi', 78, 82, 80, 85, 79);
INSERT INTO STUDENT_MARK VALUES (103, 'Priya', 92, 95, 90, 94, 91);
INSERT INTO STUDENT_MARK VALUES (104, , 'Riya', 75, 80, 78, 82, 76);
INSERT INTO STUDENT_MARK VALUES (105, 'Abi', 88, 86, 91, 89, 90);
```

SQL Query

Query 1: Display all movie records

```
SQL> SELECT * FROM STUDENT_MARK;;
```

Resultant Query

STUENTS-ID	STUDENTS-NAME	TAMIL	ENGLISH	MATHS	SCIENCE	SOCIAL
101	AKSHAYA	85	90	88	92	87
102	KEERTHI	78	82	80	85	79
103	PRIYA	92	95	90	94	91
104	RIYA	75	80	78	82	76
105	ABI	88	86	91	89	90

Query 2: Display movies released from the year 2000 onwards

SQL> SELECT *FROM STUDENT_MARKWHERE MATHS >= 80ORDER BY MATHS;

Resultant Query:

STUENTS-ID	STUDENTS-NAME	TAMIL	ENGLISH	MATHS	SCIENCE	SOCIAL
101	AKSHAYA	85	90	88	92	87
102	KEERTHI	78	82	80	85	79
103	PRIYA	92	95	90	94	91
105	ABI	88	86	91	89	90

RESULT:

Thus, the STUDENT_MARK table was created successfully,tudent details were inserted, all records were displayed, and students with Maths marks 80 and above were retrieved successfully.

EX. NO: 02 Creation of DML Operations using Product Table

DATE: 6.8.2026

Aim

To perform DML (Data Manipulation Language) operations such as INSERT, UPDATE, DELETE, and SELECT on the Product table.

Software Used

- Oracle / MySQL / SQLite3

Description

DML (Data Manipulation Language) is used to manipulate the data stored in database tables.

DML Commands

- INSERT
- UPDATE
- DELETE

INSERT

Syntax

INSERT INTO product VALUES (product_id, product_name, price);

Create Table

```
CREATE TABLE product (
   product_id INTEGER PRIMARY KEY,
   product_name VARCHAR(100),
   price INTEGER
);
```

Sample Data

INSERT INTO product VALUES (101,'Laptop',55000);

INSERT INTO product VALUES (102,'Mouse',800);

INSERT INTO product VALUES (103,'Keyboard',1500);

INSERT INTO product VALUES (104,'Monitor',12000);

INSERT INTO product VALUES (105,'Printer',9000);

INSERT INTO product VALUES (106,'Speaker',2500);

6 Rows Inserted

UPDATE

Syntax

UPDATE product

SET price = value

WHERE product_id = value;

Query

UPDATE product

SET price = 9500

WHERE product_id = 105;

1 Row Updated

DELETE

Syntax

DELETE FROM product

WHERE product_id = value;

Query

DELETE FROM product

WHERE product_id = 102;

1 Row Deleted

Query 1: Update Printer price to 9500

UPDATE product

SET price = 9500

WHERE product_id = 105;

Query 2: Delete Mouse record

DELETE FROM product

WHERE product_id = 102;

Query 3: Display all records after UPDATE and DELETE

SELECT * FROM product;

Resultant Query

product_id	product_name	price
101	Laptop	55000
103	Keyboard	1500
104	Monitor	12000
105	Printer	9500
106	Speaker	2500

RESULT

Thus, the DML operations (INSERT, UPDATE, DELETE) were performed successfully on the
Product table, and the required results were obtained.

EX. NO: 03 Creation of DQL Operations using Book Table

DATE: 11.8.2026

Aim

To create a Book table, insert book records, and perform DQL (Data Query Language) operations using SELECT, WHERE, ORDER BY, GROUP BY, and HAVING clauses.

Software Used

- Oracle / MySQL / SQLite3

Description

DQL (Data Query Language) is used to retrieve data from database tables.

DQL Commands

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING

Create Table

```
CREATE TABLE book (
   book_id INTEGER PRIMARY KEY,
   book_name VARCHAR(100),
   category VARCHAR(50),
   price INTEGER
);
```

Sample Data

INSERT INTO book VALUES (101,'Java Programming','Programming',550);

INSERT INTO book VALUES (102,'Python Basics','Programming',600);

INSERT INTO book VALUES (103,'Database Systems','Database',450);

INSERT INTO book VALUES (104,'Operating System','System',500);

INSERT INTO book VALUES (105,'Computer Networks','Network',650);

INSERT INTO book VALUES (106,'SQL Essentials','Database',400);

SELECT

The SELECT command is used to retrieve all records from a table.

Syntax

SELECT * FROM book;

Query

SELECT * FROM book;

Resultant Query

book_id	book_name	category	price
101	Java Programming	Programming	550
102	Python Basics	Programming	600
103	Database Systems	Database	450
104	Operating System	System	500
105	Computer Networks	Network	650
106	SQL Essentials	Database	400

WHERE

The WHERE clause is used to filter records.

Query

SELECT * FROM book

WHERE price >= 500;

Resultant Query

book_id	book_name	category	price
101	Java Programming	Programming	550
102	Python Basics	Programming	600
104	Operating System	System	500
105	Computer Networks	Network	650

ORDER BY

The ORDER BY clause is used to sort records.

Query

SELECT * FROM book

ORDER BY price;

Resultant Query

book_id	book_name	category	price
106	SQL Essentials	Database	400
103	Database Systems	Database	450
104	Operating System	System	500
101	Java Programming	Programming	550
102	Python Basics	Programming	600
105	Computer Networks	Network	650

GROUP BY

The GROUP BY clause is used to group records having the same category.

Query

```
SELECT category, COUNT(*) AS total_books
FROM book
GROUP BY category;
```

Resultant Query

category	total_books
Database	2
Network	1
Programming	2
System	1

HAVING

The HAVING clause is used to filter grouped records.

Query

```
SELECT category, COUNT(*) AS total_books
FROM book
GROUP BY category
HAVING COUNT(*) > 1;
```

Resultant Query

category	total_books
Database	2
Programming	2

RESULT

Thus, the DQL operations (SELECT, WHERE, ORDER BY, GROUP BY, and HAVING)
were performed successfully on the Book table, and the required results were obtained.

EX. NO: 04 Creation of Transaction Control Operations using Library Table

DATE:15.8.2026

Aim

To create a **Library** table and perform **Transaction Control Language (TCL)** operations using **COMMIT, ROLLBACK, and SAVEPOINT**.

Software Used

Oracle / MySQL / SQLite3

Description

Transaction Control Language (TCL) is used to manage transactions in a database.

TCL Commands

COMMIT

ROLLBACK

SAVEPOINT

Create Table

```
CREATE TABLE library (
   book_id INTEGER PRIMARY KEY,
   book_name VARCHAR(100),
   author VARCHAR(100)
);
```

Sample Data

```
INSERT INTO library VALUES (101,'Java Programming','James Gosling');
INSERT INTO library VALUES (102,'Python Basics','Guido van Rossum');
INSERT INTO library VALUES (103,'Database Systems','C. J. Date');
INSERT INTO library VALUES (104,'Operating System','Abraham Silberschatz');
```

COMMIT

The **COMMIT** command permanently saves all changes made during the current transaction.

Syntax

COMMIT;

Query

INSERT INTO library VALUES (105,'Computer Networks','Andrew S. Tanenbaum');

COMMIT;

Transaction Committed Successfully

SAVEPOINT

The **SAVEPOINT** command creates a point within a transaction to which you can later roll back.

Syntax

SAVEPOINT savepoint_name;

Query

SAVEPOINT sp1;

INSERT INTO library VALUES (106,'SQL Essentials','Allen G. Taylor');

Savepoint Created Successfully

ROLLBACK

The **ROLLBACK** command is used to undo changes made after the last **SAVEPOINT**.

Syntax

ROLLBACK TO savepoint_name;

Query

ROLLBACK TO sp1;

Transaction Rolled Back Successfully

Resultant Query

SELECT * FROM library;

book_id	book_name	author
101	Java Programming	James Gosling
102	Python Basics	Guido van Rossum
103	Database Systems	C. J. Date
104	Operating System	Abraham Silberschatz
105	Computer Networks	Andrew S. Tanenbaum

Note: Record **106 (SQL Essentials)** is removed because of **ROLLBACK TO sp1**.

RESULT

Thus, the **Transaction Control Language (TCL)** operations using **COMMIT, SAVEPOINT, and ROLLBACK** were performed successfully on the **Library** table.

EX. NO: 05 Creation of JOIN Operations using Hospital Tables

DATE: 18.8.2026

Aim

To create **Doctor** and **Patient** tables and perform SQL JOIN operations such as **INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, and SELF JOIN**.

Software Used

Oracle / MySQL / SQLite3

Description

Joins are used to combine rows from two or more tables based on a related column.

JOIN Types

INNER JOIN

LEFT JOIN

RIGHT JOIN

FULL JOIN

SELF JOIN

Create Table

```
CREATE TABLE doctor (
   doctor_id INTEGER PRIMARY KEY,
   doctor_name VARCHAR(100),
   department VARCHAR(50)
);
CREATE TABLE patient (
   patient_id INTEGER PRIMARY KEY,
```

```
    patient_name VARCHAR(100),
    doctor_id INTEGER
);
```

Sample Data

```
INSERT INTO doctor VALUES (1,'Dr. Arun','Cardiology');
INSERT INTO doctor VALUES (2,'Dr. Priya','Neurology');
INSERT INTO doctor VALUES (3,'Dr. Karthik','Orthopedics');
INSERT INTO doctor VALUES (4,'Dr. Meena','Pediatrics');
INSERT INTO patient VALUES (101,'Ravi',1);
INSERT INTO patient VALUES (102,'Suresh',2);
INSERT INTO patient VALUES (103,'Divya',2);
INSERT INTO patient VALUES (104,'Anitha',5);
```

INNER JOIN

Syntax

```
SELECT *
FROM doctor
INNER JOIN patient
ON doctor.doctor_id = patient.doctor_id;
```

Result

doctor_id	doctor_name	department	patient_id	patient_name
1	Dr. Arun	Cardiology	101	Ravi
2	Dr. Priya	Neurology	102	Suresh
2	Dr. Priya	Neurology	103	Divya

LEFT JOIN

```
SELECT *
FROM doctor
LEFT JOIN patient
ON doctor.doctor_id = patient.doctor_id;
```

Result

doctor_id	doctor_name	department	patient_id	patient_name
1	Dr. Arun	Cardiology	101	Ravi
2	Dr. Priya	Neurology	102	Suresh
2	Dr. Priya	Neurology	103	Divya
3	Dr. Karthik	Orthopedics	NULL	NULL
4	Dr. Meena	Pediatrics	NULL	NULL

RIGHT JOIN

```
SELECT *
FROM doctor
RIGHT JOIN patient
ON doctor.doctor_id = patient.doctor_id;
```

Result

doctor_id	doctor_name	department	patient_id	patient_name
1	Dr. Arun	Cardiology	101	Ravi
2	Dr. Priya	Neurology	102	Suresh
2	Dr. Priya	Neurology	103	Divya
NULL	NULL	NULL	104	Anitha

FULL JOIN

```
SELECT *
FROM doctor
FULL OUTER JOIN patient
ON doctor.doctor_id = patient.doctor_id;
```

Result

doctor_id	doctor_name	department	patient_id	patient_name
1	Dr. Arun	Cardiology	101	Ravi
2	Dr. Priya	Neurology	102	Suresh
2	Dr. Priya	Neurology	103	Divya
3	Dr. Karthik	Orthopedics	NULL	NULL
4	Dr. Meena	Pediatrics	NULL	NULL
NULL	NULL	NULL	104	Anitha

SELF JOIN

```
CREATE TABLE staff (
   staff_id INTEGER PRIMARY KEY,
   staff_name VARCHAR(100),
   manager_id INTEGER
);
INSERT INTO staff VALUES (1,'Arun',NULL);
INSERT INTO staff VALUES (2,'Priya',1);
INSERT INTO staff VALUES (3,'Rahul',1);
INSERT INTO staff VALUES (4,'Meena',2);
SELECT A.staff_name AS Employee,
      B.staff_name AS Manager
```

FROM staff A

LEFT JOIN staff B

ON A.manager_id = B.staff_id;

Result

Employee	**Manager**
Arun	NULL
Priya	Arun
Rahul	Arun
Meena	Priya



RESULT

Thus, the **INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, and SELF JOIN** operations were performed successfully using the **Doctor** and **Patient** tables.

EX. NO: 06 Creation of SQL Functions using Vehicle Table

DATE:2 1.8.2026

Aim

To create a Vehicle table and perform SQL Functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / SQLite3

Description

SQL Functions are used to perform calculations, manipulate strings, and work with date values in a database.

SQL Functions

Aggregate Functions

String Functions

Date Functions

Create Table

```
CREATE TABLE vehicle (
   vehicle_id INTEGER PRIMARY KEY,
   vehicle_name VARCHAR(100),
   brand VARCHAR(50),
   price INTEGER,
   purchase_date DATE
);
```

Sample Data

INSERT INTO vehicle VALUES (101,'Activa','Honda',85000,'2024-01-10');

INSERT INTO vehicle VALUES (102,'Pulsar','Bajaj',150000,'2024-02-15');

INSERT INTO vehicle VALUES (103,'Splendor','Hero',95000,'2024-03-20');

INSERT INTO vehicle VALUES (104,'Apache','TVS',140000,'2024-04-12');

INSERT INTO vehicle VALUES (105,'Access','Suzuki',98000,'2024-05-25');

Aggregate Functions

Aggregate functions perform calculations on multiple rows.

COUNT()

SELECT COUNT(*) AS Total_Vehicles

FROM vehicle;

Result

Total_Vehicles
5

SUM()

SELECT SUM(price) AS Total_Price

FROM vehicle;

Total_Price
568000

AVG()

SELECT AVG(price) AS Average_Price

FROM vehicle;

Average_Price
113600

MAX()

SELECT MAX(price) AS Highest_Price

FROM vehicle;

Highest_Price
150000

MIN()

SELECT MIN(price) AS Lowest_Price

FROM vehicle;

Lowest_Price
85000

String Functions

UPPER()

SELECT UPPER(vehicle_name)

FROM vehicle;

UPPER(vehicle_name)
ACTIVA
PULSAR
SPLENDOR
APACHE
ACCESS

LOWER()

SELECT LOWER(brand)

FROM vehicle;

LOWER(brand)
honda
bajaj
hero
tvs
suzuki

LENGTH()

SELECT vehicle_name,

LENGTH(vehicle_name)

FROM vehicle;

vehicle_name	LENGTH(vehicle_name)
Activa	6
Pulsar	6
Splendor	8
Apache	6
Access	6

Date Functions

Current Date

Oracle

SELECT SYSDATE

FROM DUAL;

MySQL

SELECT CURDATE();

SYSDATE / CURDATE
Current System Date

Extract Year

Oracle

SELECT EXTRACT(YEAR FROM purchase_date)

FROM vehicle;

MySQL

SELECT YEAR(purchase_date)

FROM vehicle;

YEAR(purchase_date)
2024
2024
2024
2024
2024

RESULT

Thus, the Aggregate Functions, String Functions, and Date Functions were performed successfully using the Vehicle table.

EX. NO: 07 Subqueries and Nested Queries

DATE:22.8.2026

Aim

To implement subqueries and nested queries using an Employee Salary Management System.

Software Used

Oracle / MySQL / SQLite3

Description

A **Subquery** is a query written inside another SQL query. The inner query is executed first and its result is used by the outer query.

TYPES

- Single-row subquery
- Multiple-row subquery
- Nested subquery

Create Table

```
CREATE TABLE EMP_SALARY
(
   EMP_ID NUMBER PRIMARY KEY,
   EMP_NAME VARCHAR2(50),
   DEPARTMENT VARCHAR2(30),
   SALARY NUMBER(10,2)
);
```

price NUMBER

);

Table Created

Sample Data

```
INSERT INTO EMP_SALARY VALUES(101, 'Arun', 'IT', 45000);
    INSERT INTO EMP_SALARY VALUES(102, 'Bala', 'HR', 35000);
    INSERT INTO EMP_SALARY VALUES(103, 'Divya', 'IT', 60000);
    INSERT INTO EMP_SALARY VALUES(104, 'Kavi', 'Sales', 40000);
     INSERT INTO EMP_SALARY VALUES(105, 'Meena', 'IT', 75000);


COMMIT;
```

SQL Query

Query 1: Employees Having Salary Greater Than Average Salary

```
SELECT EMP_ID, EMP_NAME, SALARY
FROM EMP_SALARY
WHERE SALARY >
    (SELECT AVG(SALARY)
     FROM EMP_SALARY);
```

Resultant Query

EMP_id	**EMP_name**	SALARY
103	DIVYA	60000
105	MEENA	75000

Query 2: Query 2: Employee With Maximum Salary

```
SELECT EMP_ID, EMP_NAME, DEPARTMENT, SALARY
FROM EMP_SALARYWHERE SALARY =
    (SELECT MAX(SALARY)
     FROM EMP_SALARY);
```

OUTPUT:

Resultant Query

EMP_id	**EMP_name**	DEPARTMENT	SALARY
105	MEENA	IT	75000

RESULT

Thus, Subqueries and Nested Queries were successfully implemented using the Employee Salary Management System.

EX. NO: 08 Views, Indexes and Sequences

DATE:25.8.2026

Aim

To implement and understand Views, Indexes, and Sequences using a Hospital Management System.

Software Used

Oracle / MySQL / SQLite3

Description

View: A virtual table created from a SQL query.

Index: Used to improve the speed of data retrieval.

Sequence: Used to generate sequential numeric values.

Create Table

```
CREATE TABLE HOSPITAL (
 PATIENT_ID NUMBER PRIMARY KEY,
 PATIENT_NAME VARCHAR2(50),
 AGE NUMBER(3),
 DISEASE VARCHAR2(50),
 FEES NUMBER(10,2));
```

Table Created

Sample Data

INSERT INTO HOSPITAL VALUES **(101, 'ARUN', 21, 'Fever', 2000);**

INSERT INTO HOSPITALVALUES (102, 'BALA', 25, 'Cold', 1500);

INSERT INTO HOSPITAL VALUES(103, 'CATHY', 30, 'Diabetes', 5000);

INSERT INTO HOSPITAL VALUES (104, 'DAVID', 35, 'Heart Problem', 10000);

COMMIT;

SQL Query

Query 1: Display View

SELECT * FROM PATIENT_VIEW;

Resultant Query

PATIENT_	PATIENT_NA	DISEASE	**AEG**	**FEES**
101	Arun	FEVER	21	2000
102	Bala	COLD	25	1500
103	Cathy	DIABETES	30	5000
104	David	HEART PROBLEM	35	10000

Query 3: Create Index

CREATE INDEX IDX_PATIENT_NAMON HOSPITAL(PATIENT_NAME);

Output: Index created.

Query 3: Display Indexed Data

SELECT PATIENT_ID, PATIENT_NAMEFROM HOSPITALWHERE PATIENT_NAME = 'cathy';

Resultant Query

Patient_id	**patient_name**
103	cathy

Query 4: Insert Using Sequence

```
INSERT INTO HOSPITALVALUES
(PATIENT_SEQ.NEXTVAL, 'Kavya', 24, 'Asthma', 3500);

COMMIT;
```

Query 5: Display New Record

```
SELECT *FROM HOSPITAL
WHERE PATIENT_NAME = 'Kavya';
```

Output:

PATIENT_ID PATIENT_NAME AGE DISEASE FEES

Resultant Query

PATIENT_ID	PATIENT_NAME	AGE	DISEASE	FEES
105	KAVYA	24	ASTHMA	3500

RESULT:Thus, the View, Index and Sequence were created and executed successfully.

EX. NO: 09 Stored Procedures and Functions

DATE:29.8.2026

Aim

To create and execute stored procedures and functions using the **BANK_ACCOUNT** table.

Software Used

Oracle / MySQL / SQLite3

Description

A **Stored Procedure** is a named PL/SQL block stored in the database.

A **Function** is a stored PL/SQL program that returns a value.

Create Table

```
CREATE TABLE bank_account (
   account_no INTEGER PRIMARY KEY,
   holder_name VARCHAR2(50),
   balance NUMBER
);
```

Table Created

Sample Data

```
INSERT INTO bank_account VALUES (501, 'Arun', 25000);
INSERT INTO bank_account VALUES (502, 'Bala', 35000);
INSERT INTO bank_account VALUES (503, 'Cathy', 45000);

COMMIT;
```

SQL Query

Query 1: Display all account records

SELECT * FROM bank_account;

Resultant Query

account_no	holder_name	balance
501	Arun	25000
502	Bala	35000
503	Cathy	45000

STORED PROCEDURE

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name
IS
BEGIN
   statements;
END;
/
```

Query

```
CREATE OR REPLACE PROCEDURE display_accounts
IS
BEGIN
   FOR rec IN (
      SELECT account_no, holder_name, balance
      FROM bank_account
   )
   LOOP
      DBMS_OUTPUT.PUT_LINE(
         'Account No: ' || rec.account_no ||
         'Name: ' || rec.holder_name ||
```

```
        'Balance: ' || rec.balance
      );
   END LOOP;
END;
/
```

Procedure Created

Execute Procedure

```
SET SERVEROUTPUT ON;

EXEC display_accounts;
```

Resultant Query

Account No: 501 Name: Arun Balance: 25000

Account No: 502 Name: Bala Balance: 35000

Account No: 503 Name: Cathy Balance: 45000

FUNCTION

Query

```
CREATE OR REPLACE FUNCTION get_balance
(
   p_account_no NUMBER
)
RETURN NUMBER
IS
   v_balance NUMBER;
BEGIN
```

```
    SELECT balance
    INTO v_balance
    FROM bank_account
    WHERE account_no = p_account_no;

    RETURN v_balance;
END;
/
```

Function Created

Execute Function

```
SELECT get_balance(501) AS balance
FROM dual;
```

Resultant Query

BALANCE
25000

Explanation

The procedure displays all bank account records. The function accepts an account number and returns the corresponding account balance.

RESULT

Thus, the stored procedure and function were created and executed successfully using the Bank Account table.

EX. NO: 10 Triggers

DATE:5.9.2026

Aim

To implement and understand Triggers using an Employee Audit Management System.

Software Used

Oracle / MySQL / SQLite3

Description

A **Trigger** is a stored PL/SQL program that automatically executes when an event such as INSERT, UPDATE, or DELETE occurs.

TYPES OF TRIGGERS

- BEFORE Trigger
- AFTER Trigger
- INSERT Trigger
- UPDATE Trigger
- DELETE Trigger

Syntax

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE INSERT OR UPDATE
ON table_name
FOR EACH ROW
BEGIN
   statements;
END;
/
```

Table Created

Create Audit Table

```
CREATE TABLE EMP_AUDIT_LOG
(
   EMP_ID NUMBER,
   EMP_NAME VARCHAR2(50),
   ACTION_TYPE VARCHAR2(20),
   ACTION_DATE DATE
);
```

Sample Data

```
INSERT INTO EMPLOYEE_AUDITVALUES (101, 'Arun', 'IT', 45000);
INSERT INTO EMPLOYEE_AUDITVALUES (102, 'Bala', 'HR', 35000);

COMMIT;
```

SQL Query

Query 1: Query 1: Insert Employee

INSERT INTO EMPLOYEE_AUDITVALUES (103, 'Divya', 'Sales', 40000);

Resultant Query

emp_id	**emp_name**	Action_type	**Action_date**
103	Divya	INSERT	06-OCT-26
103	Divya	update	06-OCT-26

CREATE TRIGGER

```
CREATE OR REPLACE TRIGGER TRG_EMP_AUDIT
AFTER INSERT OR UPDATE OR DELETE
ON EMPLOYEE_AUDIT
FOR EACH ROW
BEGIN
   IF INSERTING THEN
      INSERT INTO EMP_AUDIT_LOG
```

```
    VALUES
        (:NEW.EMP_ID,
         :NEW.EMP_NAME,
         'INSERT',
         SYSDATE);
    ELSIF UPDATING THEN
         INSERT INTO EMP_AUDIT_LOG
         VALUES
         (:NEW.EMP_ID,
          :NEW.EMP_NAME,
          'UPDATE',
          SYSDATE);
    ELSIF DELETING THEN
         INSERT INTO EMP_AUDIT_LOG
         VALUES
         (:OLD.EMP_ID,
          :OLD.EMP_NAME,
          'DELETE',
          END/
```

.RESULT:Thus, the Trigger was successfully created and automatically recorded employee operations in the audit table.

EX. NO: 11 PL/SQL Exception Handling

DATE:12.9.2026

Aim

To implement exception handling in PL/SQL using the **PRODUCT** table.

Software Used

Oracle / MySQL / SQLite3

Description

Exception handling is used to handle runtime errors in a PL/SQL program.

TYPES OF EXCEPTIONS

- NO_DATA_FOUND
- TOO_MANY_ROWS
- ZERO_DIVIDE
- OTHERS

Syntax

```
BEGIN
   statements;

EXCEPTION
   WHEN exception_name THEN
      statements;

   WHEN OTHERS THEN
      statements;
END;
/
```

Create Table

```
CREATE TABLE product (
   product_id INTEGER PRIMARY KEY,
   product_name VARCHAR2(50),
   price NUMBER
);
```

Table Created

Sample Data

```
INSERT INTO product VALUES (201, 'Laptop', 55000);
INSERT INTO product VALUES (202, 'Keyboard', 1500);
INSERT INTO product VALUES (203, 'Mouse', 800);
INSERT INTO product VALUES (204, 'Monitor', 12000);

COMMIT;
```

SQL Query

Query 1: Display all product records

```
SELECT * FROM product;
```

Resultant Query

product_id	product_name	price
201	Laptop	55000
202	Keyboard	1500
203	Mouse	800
204	Monitor	12000

Query 2: Exception Handling

```
SET SERVEROUTPUT ON;

DECLARE
   v_price product.price%TYPE;

BEGIN
   SELECT price
   INTO v_price
   FROM product
   WHERE product_id = 999;

   DBMS_OUTPUT.PUT_LINE('Price = ' || v_price);

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Product not found');

   WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE('More than one product found');

   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Some other error occurred');
END;
/
```

Resultant Query

Product not found

Explanation

The program searches for product ID 999. Since the product does not exist, the NO_DATA_FOUND exception occurs and the error message is displayed.

RESULT

Thus, PL/SQL exception handling was implemented successfully using the Product table.

EX. NO: 12 Database Normalization (1NF)

DATE:16.9.2026

Aim

To normalize a database table into **First Normal Form (1NF)** by ensuring that each attribute contains atomic values.

Software Used

Oracle / MySQL / SQLite3

Description

Normalization is the process of organizing data in a database to reduce data redundancy and avoid insertion, deletion, and update anomalies.

First Normal Form (1NF) requires that each column contains only atomic values and there are no repeating groups.

1NF RULES

- Each column must contain atomic values.
- Each row must be unique.
- There should be no repeating groups.
- Each field should contain only one value.

Syntax

```
CREATE TABLE table_name (
   column1 datatype,
   column2 datatype,
   column3 datatype
);
```

Create Table

```
CREATE TABLE order_1nf (
   order_id INTEGER,
```

```
    customer_name VARCHAR2(50),
    item_name VARCHAR2(50),
    quantity NUMBER
);
```

Table Created

Sample Data

```
INSERT INTO order_1nf VALUES (1001, 'Arun', 'Laptop', 1);
INSERT INTO order_1nf VALUES (1002, 'Bala', 'Keyboard', 2);
INSERT INTO order_1nf VALUES (1003, 'Cathy', 'Mouse', 3);
INSERT INTO order_1nf VALUES (1004, 'David', 'Monitor', 1);

COMMIT;
```

SQL Query

Query 1: Display all order records

```
SELECT * FROM order_1nf;
```

Resultant Query

ORDER_ID	CUSTOMER_NAM	ITEM_NAME	QUANTITY
1001	Arun	Laptop	1
1002	Bala	Keyboard	2
1003	Cathy	Mouse	3
1004	David	Monitor	1

Query 2: Display items with quantity greater than 1

```
SELECT *
FROM order_1nf
WHERE quantity > 1;
```

Resultant Query

ORDER_ID	CUSTOMER_NAM	ITEM_NAM	QUANTITY
1002	Bala	Keyboard	2
1003	Cathy	Mouse	3

Explanation

- Each column contains a single value.
- There are no multiple values in a single field.
- Each order is represented as a separate row.
- Therefore, the table satisfies the conditions of 1NF.

RESULT

Thus, the Order table was successfully normalized into **First Normal Form (1NF)**

Database Normalization (2NF)

Aim

To normalize a database into **Second Normal Form (2NF)** by removing partial dependency.

Software Used

Oracle / MySQL / SQLite3

Description

A table is in **Second Normal Form (2NF)** when:

- It is already in 1NF.
- There is no partial dependency.
- Every non-key attribute depends on the complete primary key.

2NF RULE

1NF + No Partial Dependency = 2NF

Create Table

```
CREATE TABLE sales_2nf (
   sale_id INTEGER,
   product_code INTEGER,
   sale_date DATE,
   product_name VARCHAR2(50),
   quantity NUMBER,
   PRIMARY KEY (sale_id, product_code)
);
```

Table Created

Sample Data

```
INSERT INTO sales_2nf
```

```
VALUES (2001, 301, DATE '2026-09-01', 'Laptop', 2);

INSERT INTO sales_2nf
VALUES (2001, 302, DATE '2026-09-01', 'Mouse', 3);

INSERT INTO sales_2nf
VALUES (2002, 303, DATE '2026-09-02', 'Keyboard', 4);

COMMIT;
```

SQL Query

Query 1: Display all sales records

```
SELECT * FROM sales_2nf;
```

Resultant Query

SALE_ID	PRODUCT_COD	SALE_DATE	PRODUCT_NAM	QUANTITY
2001	301	01-SEP-26	Laptop	2
2001	302	01-SEP-26	Mouse	3
2002	303	02-SEP-26	Keyboard	4

Query 2: Display products with quantity greater than 2

```
SELECT *
FROM sales_2nf
WHERE quantity > 2;
```

Resultant Query

SALE_ID	PRODUCT_COD	SALE_DATE	PRODUCT_NAM	QUANTITY
2001	302	01-SEP-26	Mouse	3
2002	303	02-SEP-26	Keyboard	4

Explanation

- The table is in 1NF.
- The combination of sale_id and product_code forms the primary key.
- Attributes are separated based on their dependency.
- Partial dependency is removed during normalization.
- This avoids unnecessary data repetition.

RESULT

Thus, the Sales table was successfully normalized into **Second Normal Form (2NF)**.

Database Normalization (3NF)

Aim

To normalize a database into **Third Normal Form (3NF)** by removing transitive dependency.

Software Used

Oracle / MySQL / SQLite3

Description

A table is in **Third Normal Form (3NF)** when:

- It is already in 2NF.
- There is no transitive dependency.
- Non-key attributes depend only on the primary key.

3NF RULE

2NF + No Transitive Dependency = 3NF

Create Table

```
CREATE TABLE hospital_patient (
   patient_id INTEGER PRIMARY KEY,
   patient_name VARCHAR2(50),
   ward_id INTEGER
);
```

Table Created

```
CREATE TABLE hospital_ward (
   ward_id INTEGER PRIMARY KEY,
   ward_name VARCHAR2(50)
);
```

Table Created

Sample Data

INSERT INTO hospital_ward VALUES (10, 'General Ward');

INSERT INTO hospital_ward VALUES (20, 'ICU');

INSERT INTO hospital_ward VALUES (30, 'Children Ward');

INSERT INTO hospital_patient VALUES (3001, 'Arun', 10);

INSERT INTO hospital_patient VALUES (3002, 'Bala', 20);

INSERT INTO hospital_patient VALUES (3003, 'Cathy', 30);

INSERT INTO hospital_patient VALUES (3004, 'David', 10);

COMMIT;

SQL Query

Query 1: Display patient records

SELECT * FROM hospital_patient;

Resultant Query

PATIENT_ID	PATIENT_NAM	WARD_ID
3001	Arun	10
3002	Bala	20
3003	Cathy	30
3004	David	10

Query 2: Display ward records

SELECT * FROM hospital_ward;

Resultant Query

WARD_ID	WARD_NAM
10	General Ward
20	ICU
30	Children Ward

Query 3: Display patient and corresponding ward

```
SELECT P.patient_id,
    P.patient_name,
    W.ward_name
FROM hospital_patient P
JOIN hospital_ward W
ON P.ward_id = W.ward_id;
```

Resultant Query

PATIENT_ID	PATIENT_NAME	WARD_NAME
3001	Arun	General Ward
3002	Bala	ICU
3003	Cathy	Children Ward
3004	David	General Ward

Explanation

- Patient details and ward details are stored separately.
- patient_id is the primary key of the patient table.
- ward_id is used to connect the two tables.
- Ward name depends on ward_id, not directly on patient_id.
- Therefore, transitive dependency is removed.

RESULT

Thus, the Hospital Patient database was successfully normalized into **Third Normal Form (3NF)**.

Database Normalization (BCNF)

Aim

To normalize a database into **Boyce-Codd Normal Form (BCNF)** by ensuring that every determinant is a candidate key.

Software Used

Oracle / MySQL / SQLite3

Description

BCNF (Boyce-Codd Normal Form) is a stronger version of 3NF.

A relation is in BCNF if **every determinant is a candidate key**.

BCNF RULE

Every determinant must be a candidate key.

Create Table

```
CREATE TABLE flight (
   flight_id INTEGER PRIMARY KEY,
   flight_name VARCHAR2(50),
   source_city VARCHAR2(50),
   destination_city VARCHAR2(50)
);
```

Table Created

Sample Data

```
INSERT INTO flight
VALUES (4001, 'Air India Express', 'Chennai', 'Delhi');

INSERT INTO flight
VALUES (4002, 'IndiGo', 'Chennai', 'Mumbai');
```

```
INSERT INTO flight
VALUES (4003, 'Vistara', 'Delhi', 'Bangalore');

INSERT INTO flight
VALUES (4004, 'SpiceJet', 'Mumbai', 'Chennai');

COMMIT;
```

SQL Query

Query 1: Display all flight records

```
SELECT * FROM flight;
```

Resultant Query

FLIGHT_ID	FLIGHT_NAME	SOURCE_CITY	DESTINATION_CIT
4001	Air India Express	Chennai	Delhi
4002	IndiGo	Chennai	Mumbai
4003	Vistara	Delhi	Bangalore
4004	SpiceJet	Mumbai	Chennai

Query 2: Display flights starting from Chennai

```
SELECT *
FROM flight
WHERE source_city = 'Chennai';
```

Resultant Query

FLIGHT_ID	FLIGHT_NAM	SOURCE_CIT	DESTINATION_CIT
4001	Air India Express	Chennai	Delhi
4002	IndiGo	Chennai	Mumbai

Explanation

- flight_id uniquely identifies every flight.
- It is the primary key and therefore a candidate key.
- All other attributes depend on the flight identifier.
- There is no non-key determinant in the relation.
- Therefore, the table satisfies the BCNF condition.

RESULT

Thus, the Flight table was successfully normalized into **Boyce-Codd Normal Form (BCNF)**.

KHAN ACADEMY

BOOK LIST DATABASE

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating.

PROGRAM:

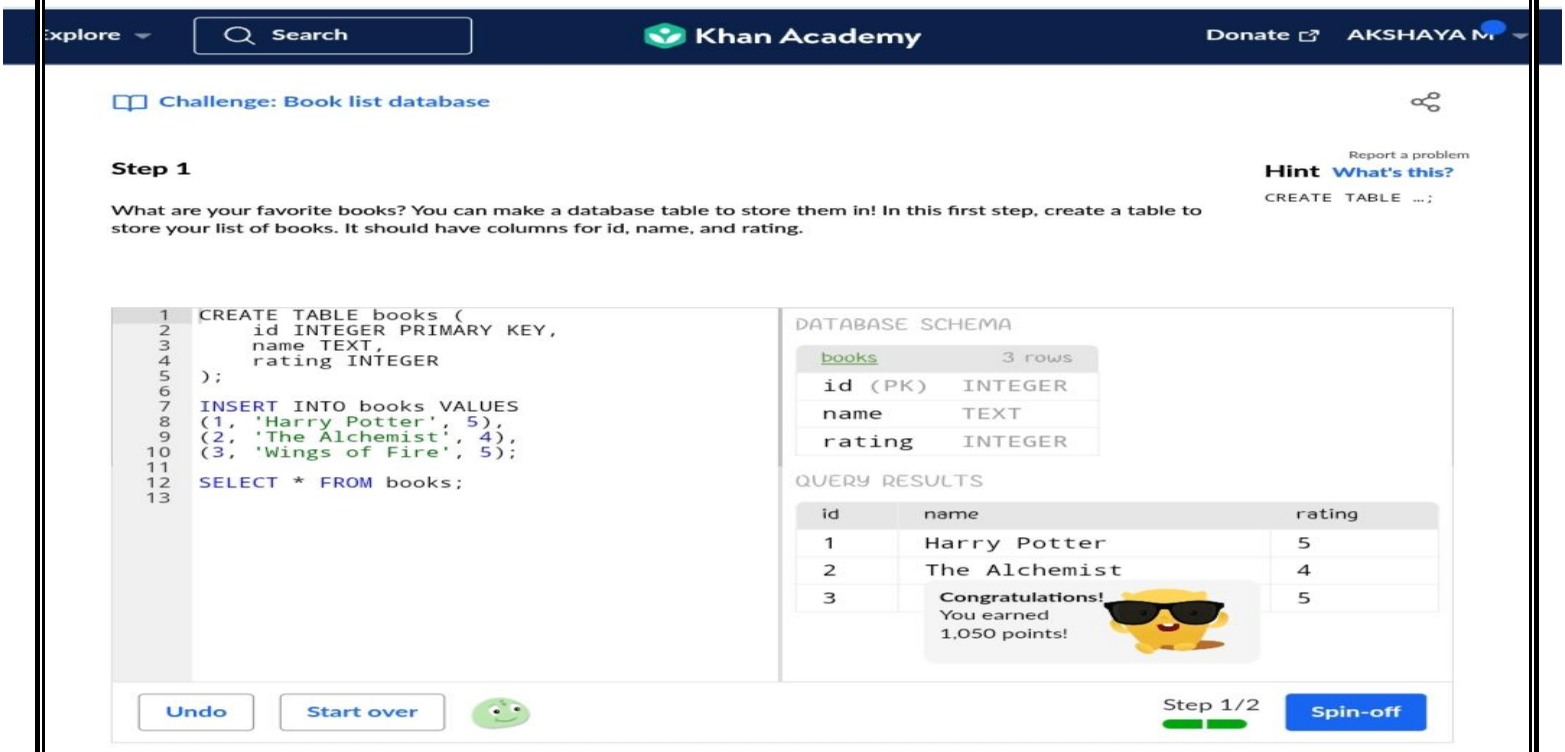
```
create table book (id integer primary key , name text ,rating integer);

insert INTO book VALUES (1, 'The Alchemist', 5);

INSERT INTO book VALUES (2, 'Harry Potter and the Sorcerer"s Stone', 5);

INSERT INTO book VALUES (3, 'To Kill a Mockingbird', 4);
```

OUTPUT:



BOX OFFICE HITS DATABASE

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies.

PROGRAM:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, 'Avatar', 2009);

INSERT INTO movies VALUES (2, 'Titanic', 1997);

INSERT INTO movies VALUES (3, 'Star Wars: Episode IV - A New Hope', 1977);

INSERT INTO movies VALUES (4, 'Shrek 2', 2004);

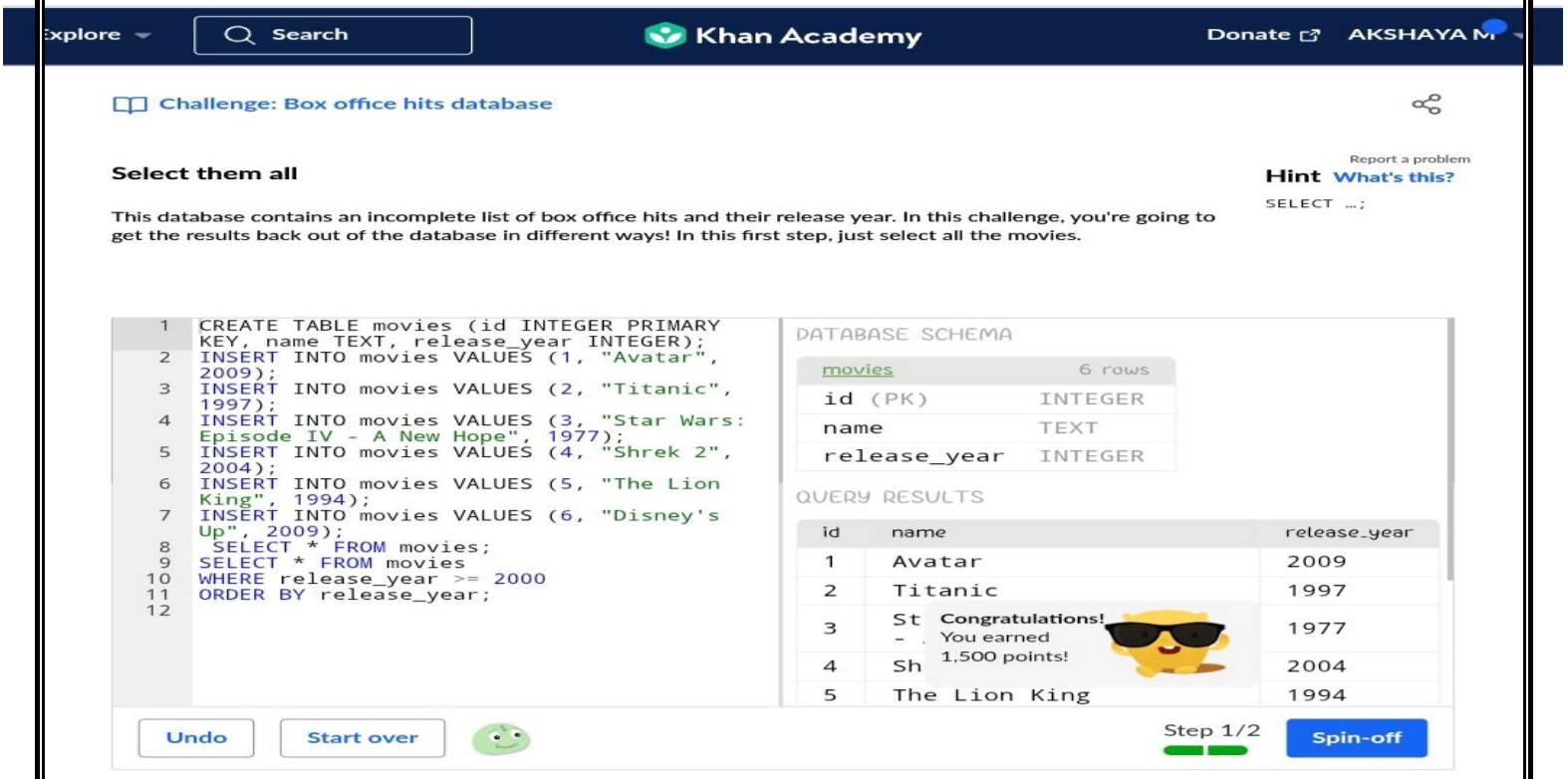
INSERT INTO movies VALUES (5, 'The Lion King', 1994);

INSERT INTO movies VALUES (6, 'Disney"s Up', 2009);

SELECT * FROM movies;

SELECT * FROM movies
WHERE release_year >= 2000
ORDER BY release_year;
```

OUTPUT:



TODO LIST DATABASE STATS

Here's a table containing a TODO list with the number of minutes it will take to complete each item.
Insert another item to your todo list with the estimated minutes it will take.

PROGRAM:

```
CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

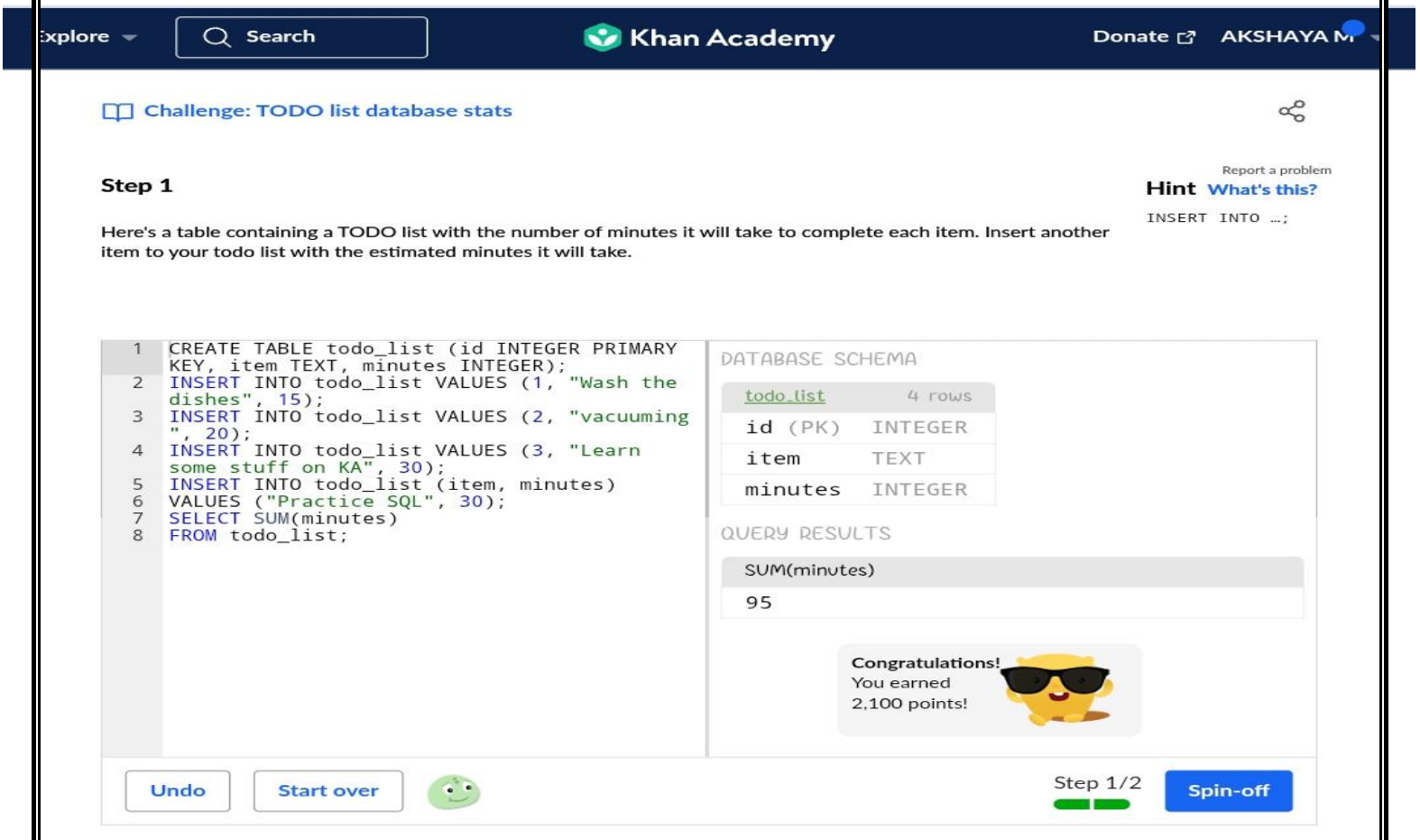
INSERT INTO todo_list VALUES (2, "vacuuming", 20);

INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4, 'Dry your clothes', 40);


SELECT SUM(minutes) FROM todo_list;
```

OUTPUT:



PROJECT: DESIGN A STORE DATABASE

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

PROGRAM:

```
CREATE TABLE sneakers (
   id INTEGER PRIMARY KEY,
   brand TEXT,
   model TEXT,
   size INTEGER,
   color TEXT,
   price REAL
);
INSERT INTO sneakers VALUES (1, "Nike", "Air Max", 9, "Black", 120);
INSERT INTO sneakers VALUES (2, "Adidas", "Ultraboost", 10, "White", 180);
INSERT INTO sneakers VALUES (3, "Puma", "RS-X", 8, "Blue", 110);
INSERT INTO sneakers VALUES (4, "New Balance", "574", 9, "Gray", 95);
INSERT INTO sneakers VALUES (5, "Converse", "Chuck Taylor", 10, "Red", 60);
INSERT INTO sneakers VALUES (6, "Vans", "Old Skool", 8, "Black", 70);
INSERT INTO sneakers VALUES (7, "Nike", "Jordan 1", 11, "Red", 200);
INSERT INTO sneakers VALUES (8, "Adidas", "NMD", 9, "Black", 140);
INSERT INTO sneakers VALUES (9, "Puma", "Suede Classic", 10, "Green", 75);
INSERT INTO sneakers VALUES (10, "New Balance", "990", 11, "Gray", 185);
INSERT INTO sneakers VALUES (11, "Converse", "Run Star Hike", 8, "White", 115);
INSERT INTO sneakers VALUES (12, "Vans", "Sk8-Hi", 9, "Blue", 80);
INSERT INTO sneakers VALUES (13, "Nike", "Air Force 1", 10, "White", 130);
INSERT INTO sneakers VALUES (14, "Adidas", "Superstar", 8, "Black", 100);
```

```
INSERT INTO sneakers VALUES (15, "Puma", "Future Rider", 9, "Yellow", 90);

/* Show all sneakers ordered by price (highest to lowest) */

SELECT *

FROM sneakers

ORDER BY price DESC;


/* Show average, maximum, and minimum price */

SELECT

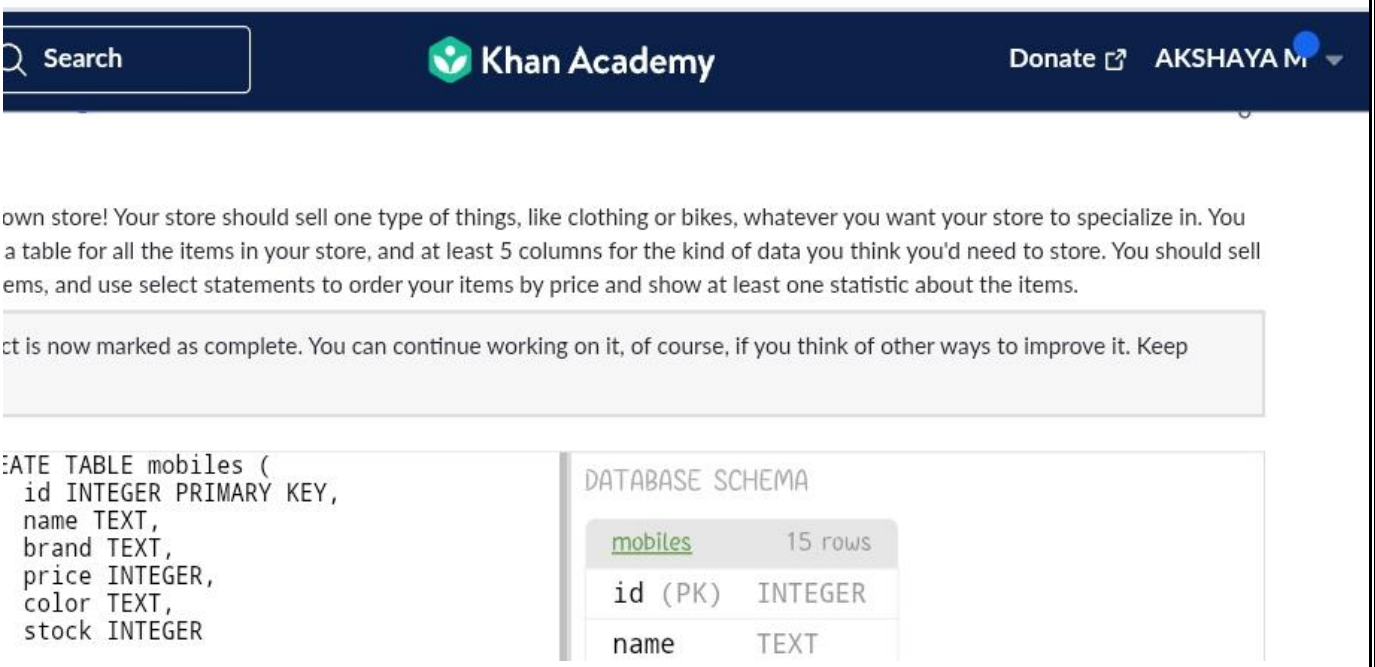
   AVG(price) AS average_price,

   MAX(price) AS highest_price,

   MIN(price) AS lowest_price

FROM sneakers;
```

OUTPUT:



KARAOKE SONG SELECTOR

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles.

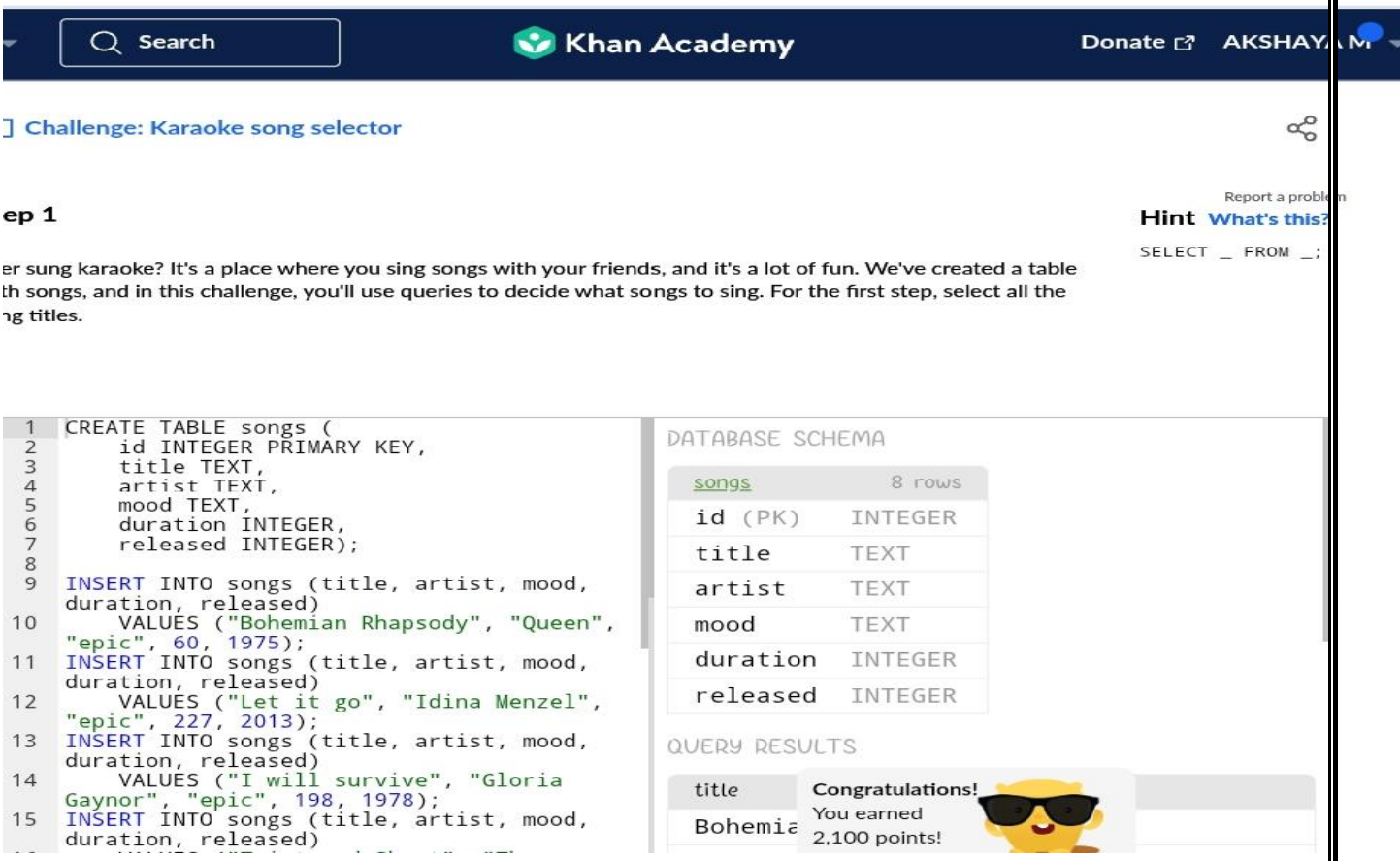
PROGRAM:

```
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
    mood TEXT,
    duration INTEGER,
    released INTEGER);


INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ('Bohemian Rhapsody', 'Queen', 'epic', 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ('Let it go', 'Idina Menzel', 'epic', 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ('I will survive', 'Gloria Gaynor', 'epic', 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ('Twist and Shout', 'The Beatles', 'happy', 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ('La Bamba', 'Ritchie Valens', 'happy', 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ('I will always love you', 'Whitney Houston', 'epic', 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ('Sweet Caroline', 'Neil Diamond', 'happy', 201, 1969);
```

```
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ('Call me maybe', 'Carly Rae Jepsen', 'happy', 193, 2011);
 SELECT title FROM Songs;
 SELECT title
FROM songs
WHERE mood = 'epic' OR released > 1990;
SELECT title
FROM songs
WHERE mood = 'epic'
  AND released > 1990
  AND duration < 240;
```

OUTPUT:



PLAYLIST MAKER

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the title of all the songs by the artist named 'Queen'.

PROGRAM:

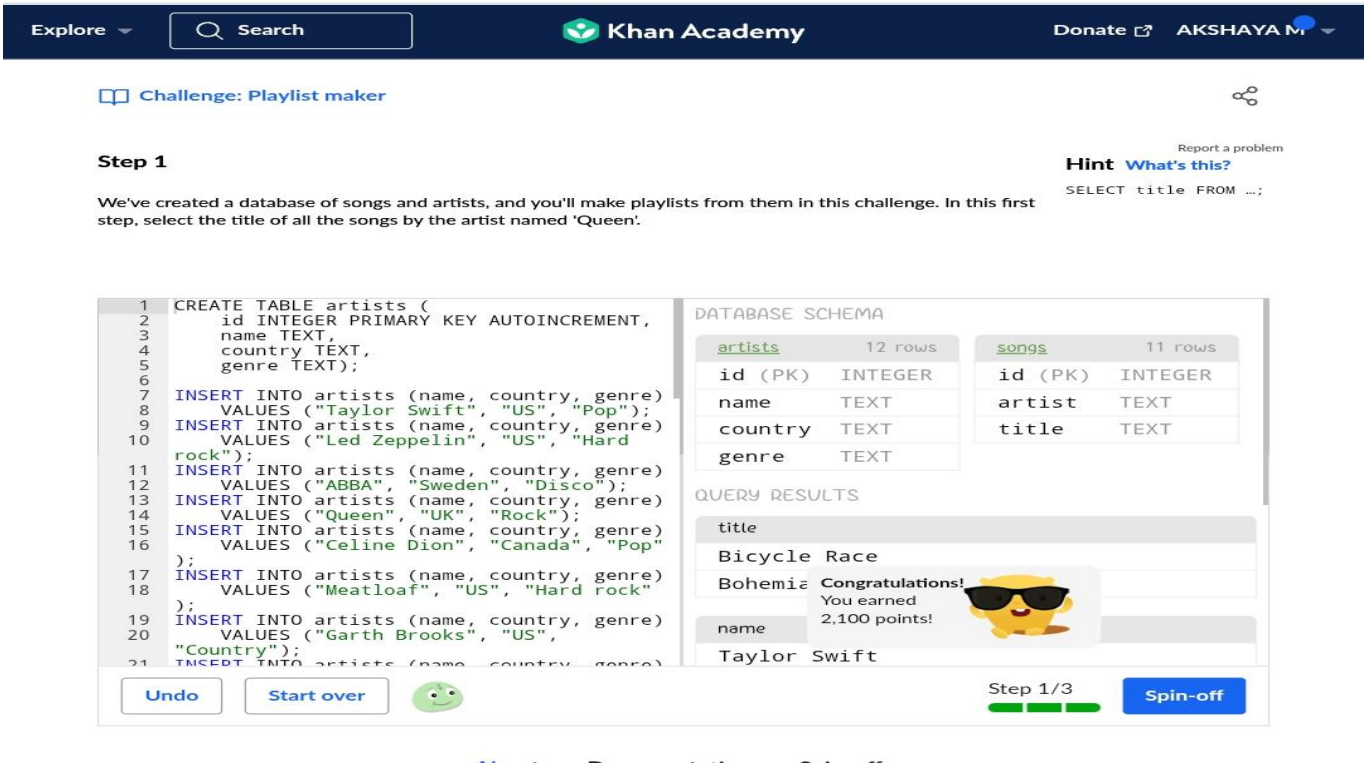
```
CREATE TABLE artists (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT,
   country TEXT,
   genre TEXT);
INSERT INTO artists (name, country, genre)
   VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
   VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
   VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
   VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
   VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
   VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
   VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
   VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
   VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
```

```
    VALUES ("Guns N' Roses", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Gloria Estefan", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Bob Marley", "Jamaica", "Reggae");
CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT,
    title TEXT);
INSERT INTO songs (artist, title)
    VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
    VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
    VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
```

```
    VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
    VALUES ("Guns N' Roses", "Don't cry");
select title from songs where artist = 'Queen';


select name from artists where genre = 'Pop';
SELECT title
FROM songs
WHERE artist IN (
    SELECT name
    FROM artists
    WHERE genre = 'Pop'
);
```

OUTPUT:



THE WORDIEST AUTHOR

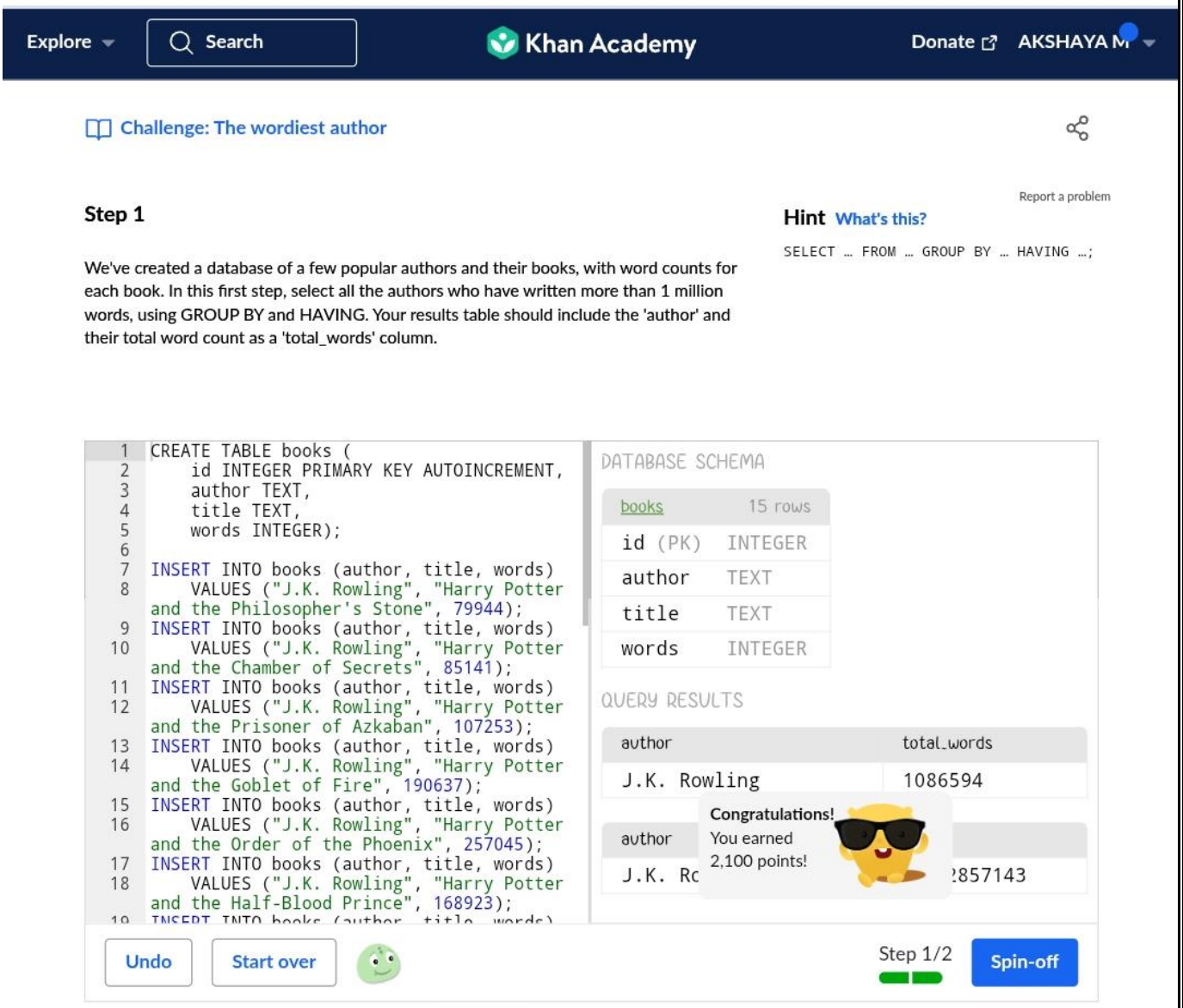
We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column.

PROGRAM:

```
CREATE TABLE books (
   id INTEGER PRIMARY KEY AUTOINCREMENT author TEXT, title TEXT,
   words INTEGER);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Eclipse", 147930);
```

```
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Breaking Dawn", 192196
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Return of the King", 134462);
SELECT author,
     SUM(words) AS total_words
FROM books
GROUP BY author
HAVING SUM(words) > 1000000;
SELECT author,
     AVG(words) AS avg_words
FROM books
GROUP BY author
HAVING AVG(words) > 150000;
```

OUTPUT:


Explore
Search
Khan Academy
Donate
AKSHAYA M
Challenge: The wordiest author
Report a problem
Step 1
We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column.
Hint What's this?
SELECT … FROM … GROUP BY … HAVING …;
CREATE TABLE books (
id INTEGER PRIMARY KEY AUTOINCREMENT,
author TEXT,
title TEXT,
words INTEGER);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
DATABASE SCHEMA
books 15 rows
id (PK) INTEGER
author TEXT
title TEXT
words INTEGER
QUERY RESULTS
author
total_words
J.K. Rowling
1086594
Congratulations!
You earned
2,100 points!
Undo
Start over
Step 1/2
Spin-off

GRADE BOOK

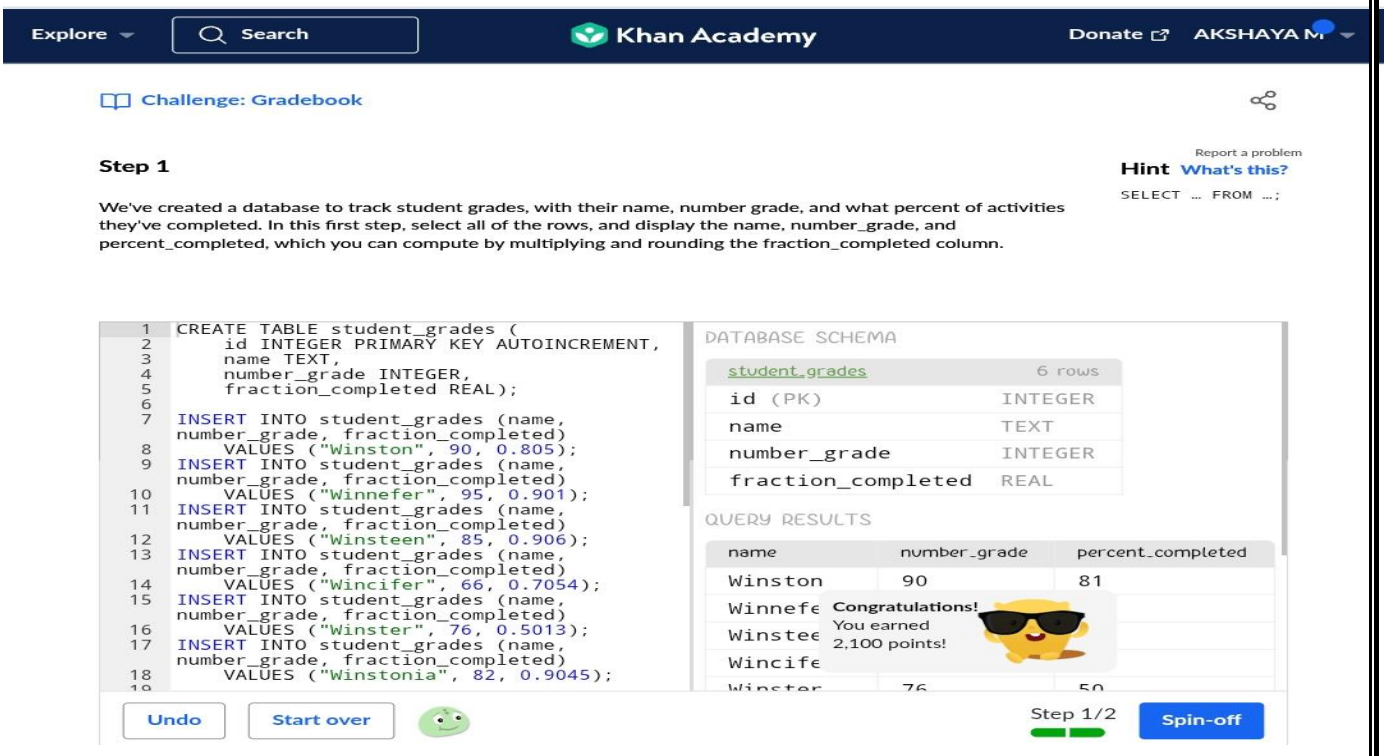
We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column.

PROGRAM:

```
CREATE TABLE student_grades (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT,
   number_grade INTEGER,
   fraction_completed REAL);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winstonia", 82, 0.9045);
SELECT name,
     number_grade,
     ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;
SELECT COUNT(*),
   CASE
```

```
    WHEN number_grade > 90 THEN 'A'
    WHEN number_grade > 80 THEN 'B'
    WHEN number_grade > 70 THEN 'C'
    ELSE 'F'
  END AS "letter_grade"
FROM student_grades
GROUP BY
  CASE
    WHEN number_grade > 90 THEN 'A'
    WHEN number_grade > 80 THEN 'B'
    WHEN number_grade > 70 THEN 'C'
    ELSE 'F'
  END;
```

OUTPUT:



PROJECT:DATA DIG

Pick one of those data sets or create a data set like that, and use advanced SELECT queries to discover things about the data. What sort of questions might one have about that data, like if they were using it for an app or a business idea? Here are some ideas:

- What are average, max, and min values in the data?
- What about those numbers per category in the data (using HAVING)?
- What ways are there to group the data values that don't exist yet (using CASE)?
- What interesting ways are there to filter the data (using AND/OR)?

```
/* Put your data in here and query it! */
CREATE TABLE pokemon (
  id INTEGER PRIMARY KEY,
  name TEXT,
  type TEXT,
  hp INTEGER,
  attack INTEGER
);

INSERT INTO pokemon VALUES (1, "Pikachu", "Electric", 35, 55);
INSERT INTO pokemon VALUES (2, "Charizard", "Fire", 78, 84);
INSERT INTO pokemon VALUES (3, "Blastoise", "Water", 79, 83);
INSERT INTO pokemon VALUES (4, "Venusaur", "Grass", 80, 82);
INSERT INTO pokemon VALUES (5, "Jolteon", "Electric", 65, 65);
INSERT INTO pokemon VALUES (6, "Arcanine", "Fire", 90, 110);
INSERT INTO pokemon VALUES (7, "Gyarados", "Water", 95, 125);
INSERT INTO pokemon VALUES (8, "Sceptile", "Grass", 70, 85);

/* Average, max, and min attack */
SELECT
```

```
    AVG(attack) AS avg_attack,
    MAX(attack) AS max_attack,
    MIN(attack) AS min_attack
FROM pokemon;

/* Average attack by type */
SELECT
    type,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type;

/* Types with average attack above 80 */
SELECT
    type,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type
HAVING AVG(attack) > 80;

/* Categorize Pokémon by strength using CASE */
SELECT
    name,
    attack,
    CASE
        WHEN attack >= 100 THEN "Very Strong"
        WHEN attack >= 80 THEN "Strong"
        ELSE "Average"
```

END AS strength

FROM pokemon;

/* Filter using AND and OR */

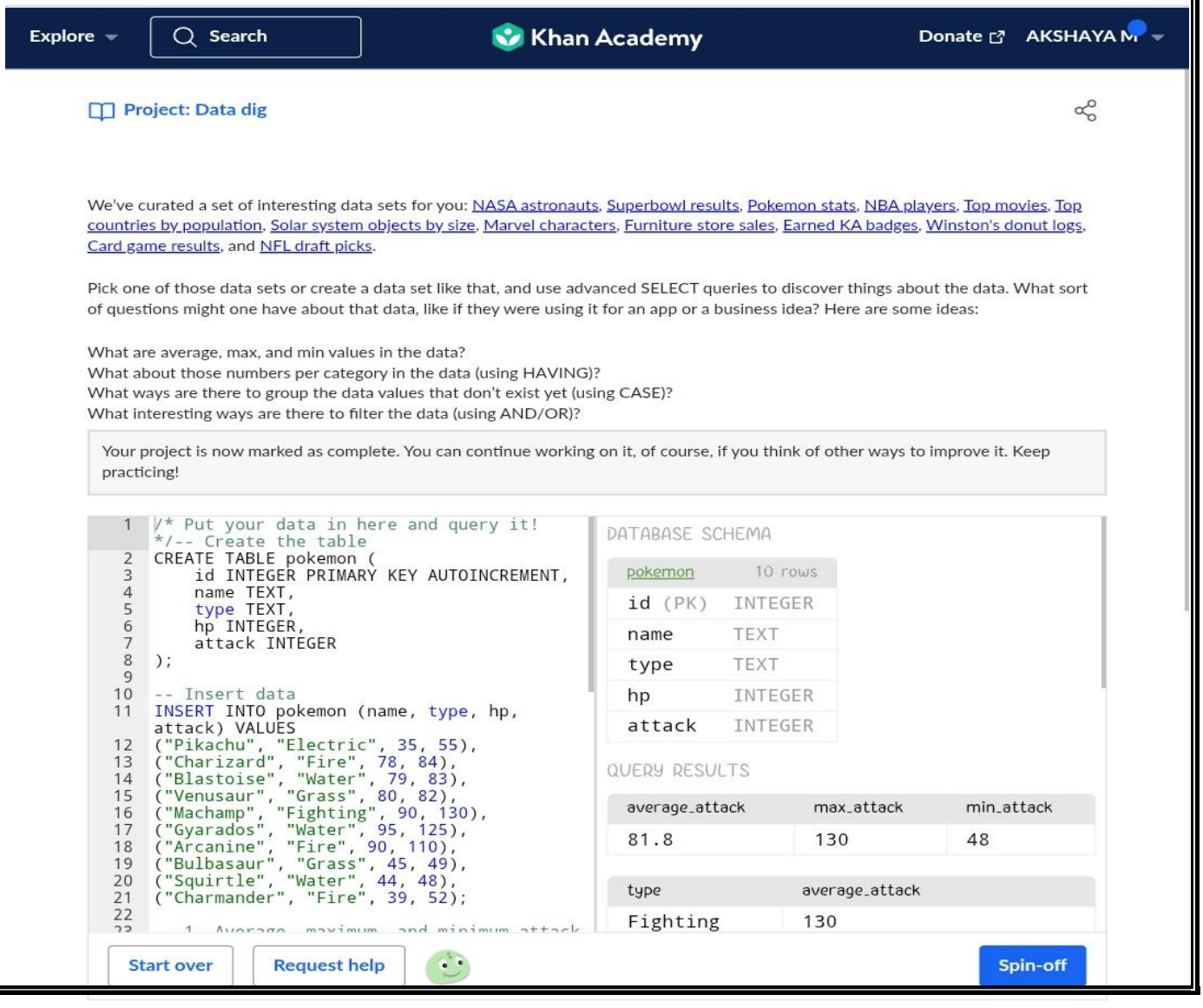
SELECT *

FROM pokemon

WHERE (type = "Fire" OR type = "Water")

AND attack > 80;/* Put your data in here and query it! */

OUTPUT:



BOBBY'S HOBBIES

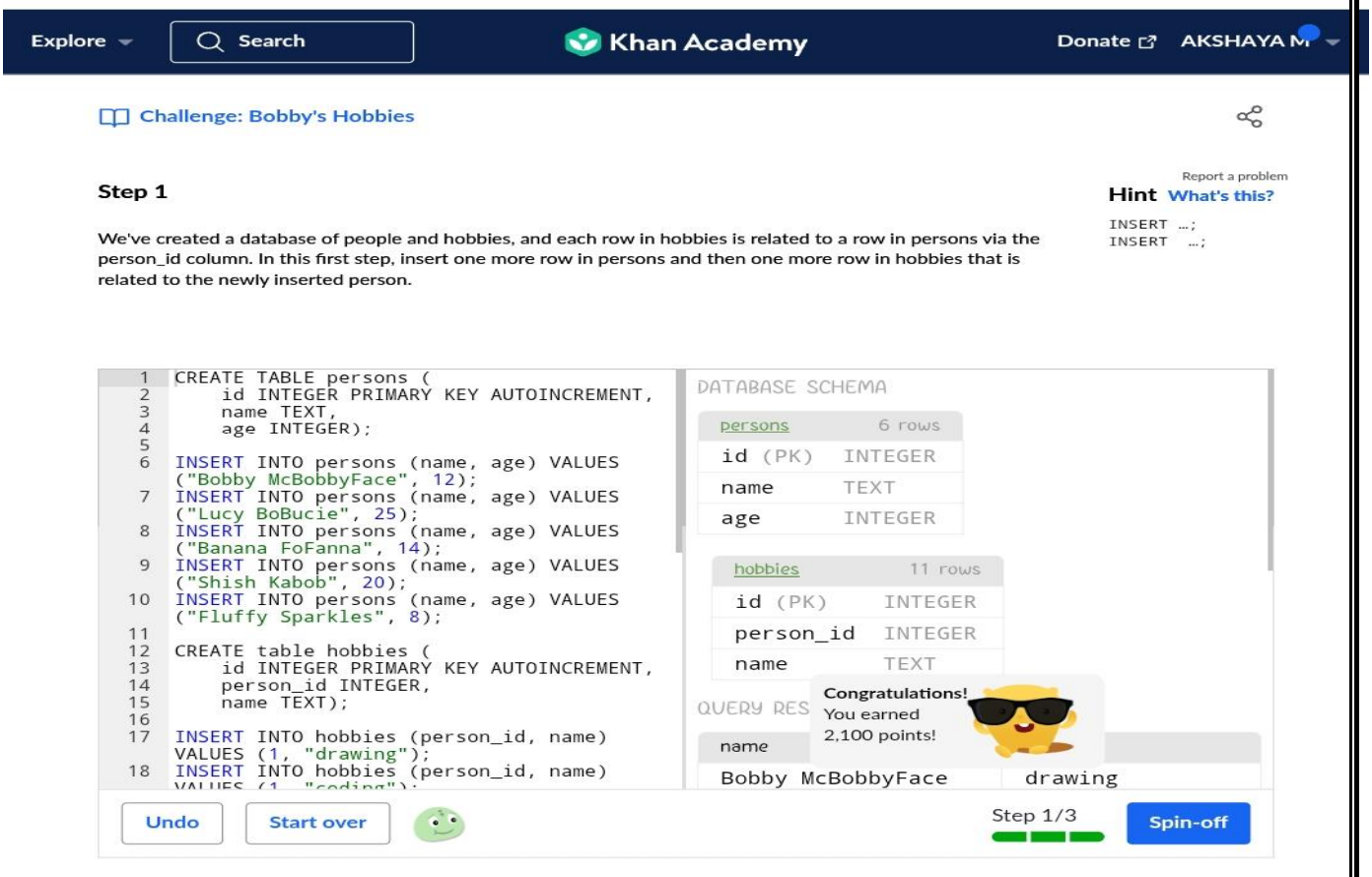
We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person.

PROGRAM:

```
CREATE TABLE persons (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT,
   age INTEGER);
INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);
CREATE table hobbies (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   person_id INTEGER,
   name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
```

```
INSERT INTO persons (name, age)
VALUES ("Sam Star", 18);
INSERT INTO hobbies (person_id, name)
VALUES (6, "gaming");
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT persons.name,
    hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace";
```

OUTPUT:



CUSTOMER'S ORDER

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY.

PROGRAM:

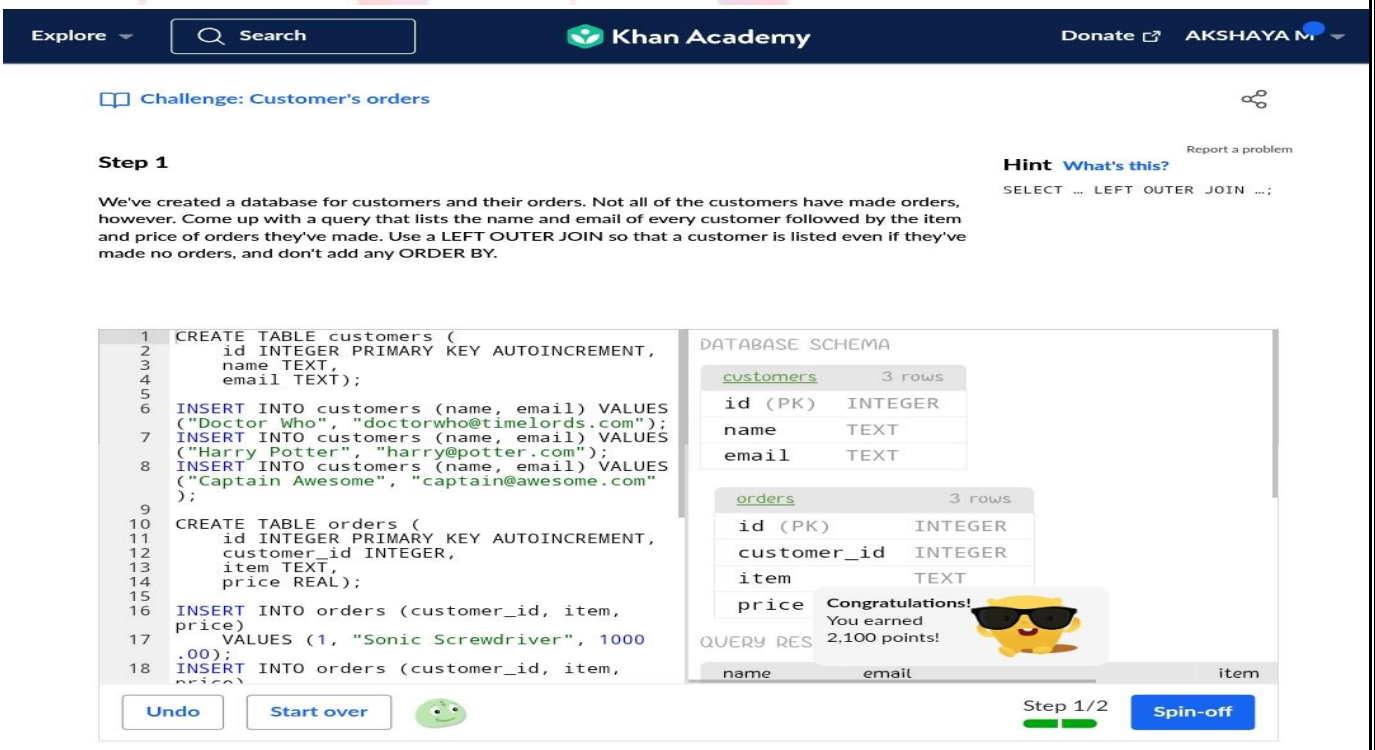
```
CREATE TABLE customers (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT,
   email TEXT);

INSERT INTO customers (name, email) VALUES ("Doctor Who", "doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome", "captain@awesome.com");

CREATE TABLE orders (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   customer_id INTEGER,
   item TEXT,
   price REAL);
INSERT INTO orders (customer_id, item, price)
   VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
   VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
   VALUES (1, "TARDIS", 1000000.00);
SELECT customers.name,
     customers.email,
```

```
    orders.item,
    orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
SELECT customers.name,
    customers.email,
    SUM(orders.price) AS total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
GROUP BY customers.id
ORDER BY total_spent DESC;
```

OUTPUT:



SEQUELS IN SQL

We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

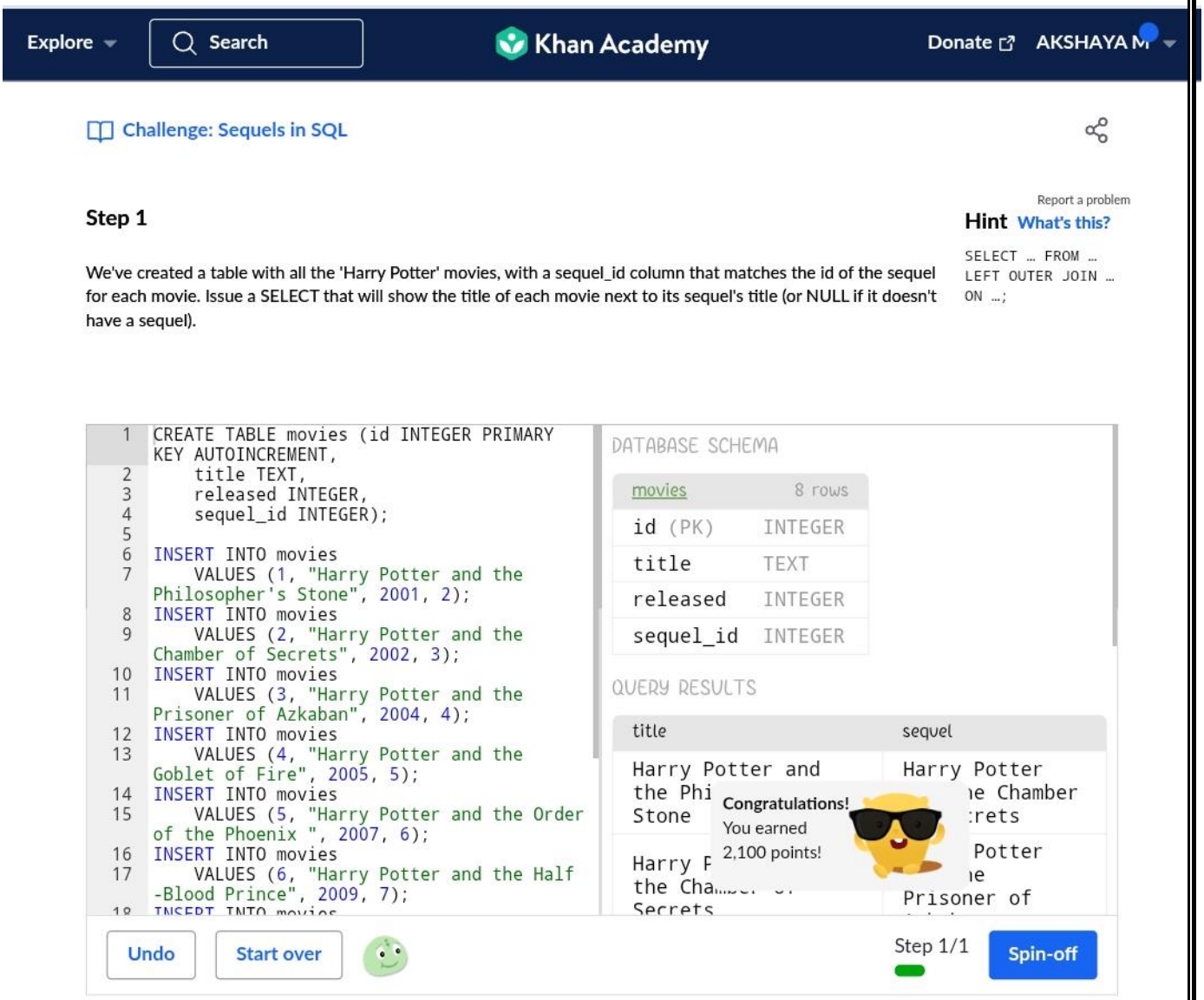
PROGRAM:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT,
   released INTEGER,
   sequel_id INTEGER);
INSERT INTO movies
   VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
   VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
   VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
   VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
   VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
INSERT INTO movies
   VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
   VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
   VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);
SELECT m1.title,
     m2.title AS sequel_title
FROM movies AS m1
```

LEFT OUTER JOIN movies AS m2

ON m1.sequel_id = m2.id;

OUTPUT:



FRIENDBOOK

We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies.

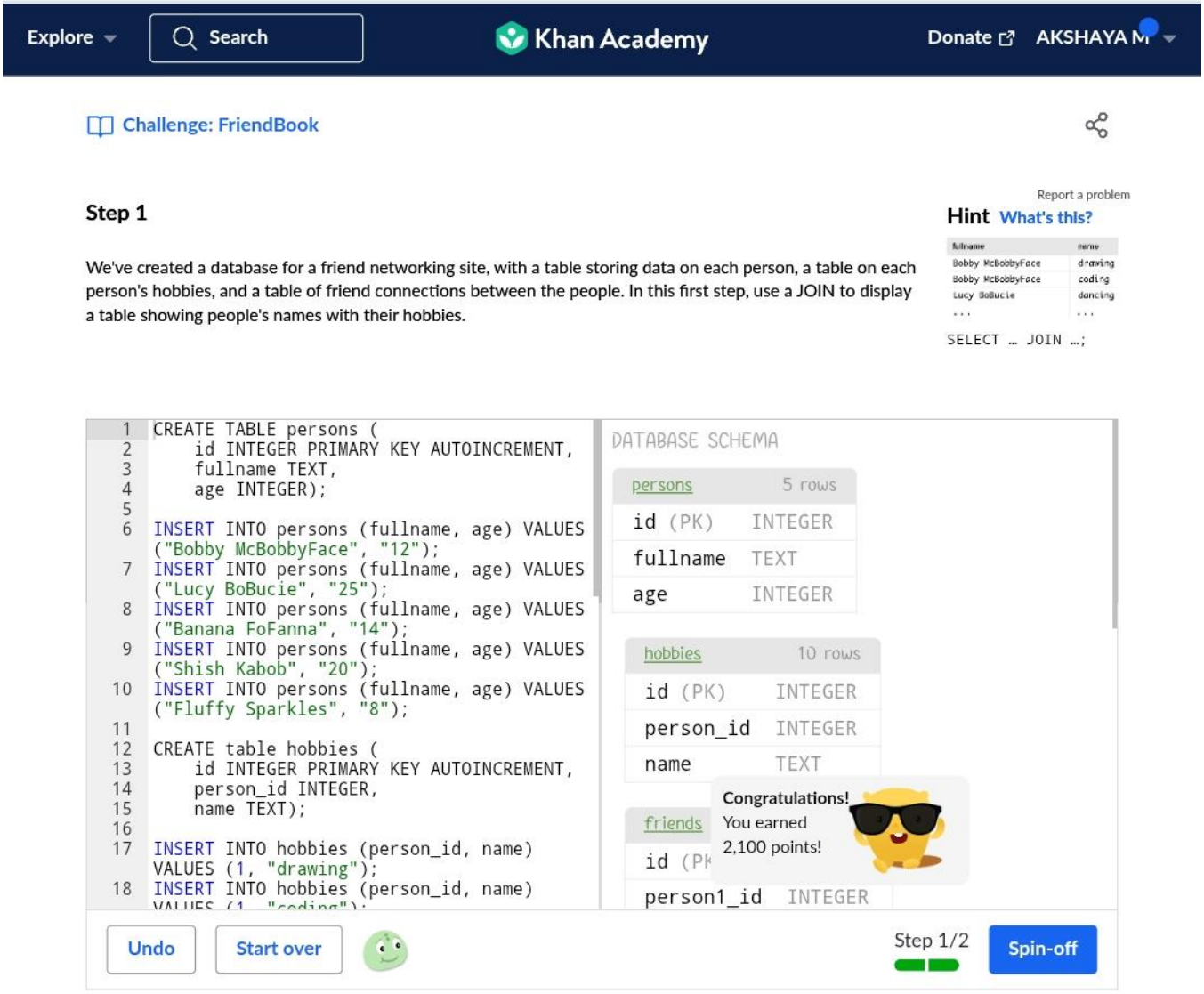
PROGRAM:

```
CREATE TABLE persons (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   fullname TEXT,
   age INTEGER);
INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");
CREATE table hobbies (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   person_id INTEGER,
   name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
```

```
CREATE table friends (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   person1_id INTEGER,
   person2_id INTEGER);
INSERT INTO friends (person1_id, person2_id)
   VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id)
   VALUES (2, 3);
SELECT persons.fullname,
     hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT p1.fullname AS person1,
     p2.fullname AS person2
FROM friends
JOIN persons AS p1
ON friends.person1_id = p1.id
JOIN persons AS p2
ON friends.person2_id = p2.id;
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

OUTPUT:


Explore
Search
Khan Academy
Donate
AKSHAYA M
Challenge: FriendBook
Step 1
We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies.
Report a problem
Hint What's this?
SELECT … JOIN …;
CREATE TABLE persons (
id INTEGER PRIMARY KEY AUTOINCREMENT,
fullname TEXT,
age INTEGER);
INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");
CREATE table hobbies (
id INTEGER PRIMARY KEY AUTOINCREMENT,
person_id INTEGER,
name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name)
DATABASE SCHEMA
persons 5 rows
id (PK) INTEGER
fullname TEXT
age INTEGER
hobbies 10 rows
id (PK) INTEGER
person_id INTEGER
name TEXT
friends
person1_id INTEGER
Congratulations!
You earned
2,100 points!
Undo
Start over
Step 1/2
Spin-off

PROJECT : FAMOUS PEOPLE

In this project, you're going to make your own table with some small set of "famous people", then make more tables about things they do and join those to create nice human readable lists.

For example, here are types of famous people and the questions your data could answer:

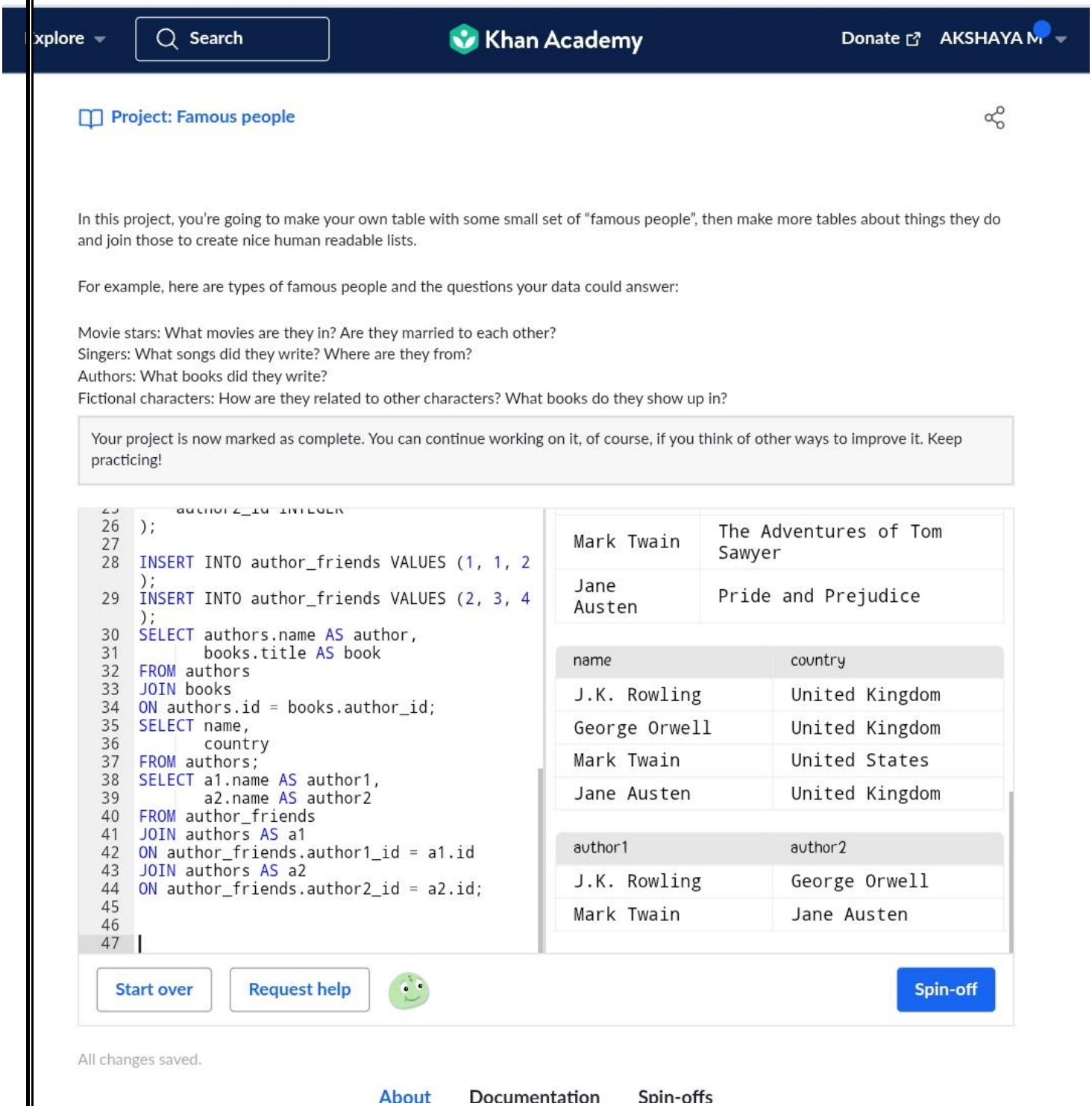
- Movie stars: What movies are they in? Are they married to each other?
- Singers: What songs did they write? Where are they from?
- Authors: What books did they write?
- Fictional characters: How are they related to other characters? What books do they show up in?

PROGRAM:

```
/* Create table about the people and what they do here */
CREATE TABLE authors (
   id INTEGER PRIMARY KEY,
   name TEXT,
   country TEXT
);
INSERT INTO authors VALUES (1, "J.K. Rowling", "United Kingdom");
INSERT INTO authors VALUES (2, "George Orwell", "United Kingdom");
INSERT INTO authors VALUES (3, "Mark Twain", "United States");
INSERT INTO authors VALUES (4, "Jane Austen", "United Kingdom");
CREATE TABLE books (
   id INTEGER PRIMARY KEY,
   title TEXT,
   author_id INTEGER
);
INSERT INTO books VALUES (1, "Harry Potter and the Sorcerer's Stone", 1);
INSERT INTO books VALUES (2, "Harry Potter and the Chamber of Secrets", 1);
INSERT INTO books VALUES (3, "1984", 2);
INSERT INTO books VALUES (4, "Animal Farm", 2);
```

```
INSERT INTO books VALUES (5, "The Adventures of Tom Sawyer", 3);
INSERT INTO books VALUES (6, "Pride and Prejudice", 4);
CREATE TABLE author_friends (
   id INTEGER PRIMARY KEY,
   author1_id INTEGER,
   author2_id INTEGER
);
INSERT INTO author_friends VALUES (1, 1, 2);
INSERT INTO author_friends VALUES (2, 3, 4);
/* Show authors and their books */
SELECT authors.name AS author,
     books.title AS book
FROM authors
JOIN books
ON authors.id = books.author_id;
/* Show authors and their countries */
SELECT name,
     country
FROM authors;
/* Show pairs of authors */
SELECT a1.name AS author1,
     a2.name AS author2
FROM author_friends
JOIN authors AS a1
ON author_friends.author1_id = a1.id
JOIN authors AS a2
ON author_friends.author2_id = a2.id;
```

OUTPUT:


xplore
Search
Khan Academy
Donate
AKSHAYA M
Project: Famous people
In this project, you're going to make your own table with some small set of "famous people", then make more tables about things they do and join those to create nice human readable lists.
For example, here are types of famous people and the questions your data could answer:
Movie stars: What movies are they in? Are they married to each other?
Singers: What songs did they write? Where are they from?
Authors: What books did they write?
Fictional characters: How are they related to other characters? What books do they show up in?
Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing!
author2_id INTEGER
);
INSERT INTO author_friends VALUES (1, 1, 2
);
INSERT INTO author_friends VALUES (2, 3, 4
);
SELECT authors.name AS author,
books.title AS book
FROM authors
JOIN books
ON authors.id = books.author_id;
SELECT name,
country
FROM authors;
SELECT a1.name AS author1,
a2.name AS author2
FROM author_friends
JOIN authors AS a1
ON author_friends.author1_id = a1.id
JOIN authors AS a2
ON author_friends.author2_id = a2.id;
Mark Twain
The Adventures of Tom Sawyer
Jane Austen
Pride and Prejudice
name
country
J.K. Rowling
United Kingdom
George Orwell
United Kingdom
Mark Twain
United States
Jane Austen
United Kingdom
author1
author2
J.K. Rowling
George Orwell
Mark Twain
Jane Austen
Start over
Request help
Spin-off
All changes saved.
About
Documentation
Spin-offs

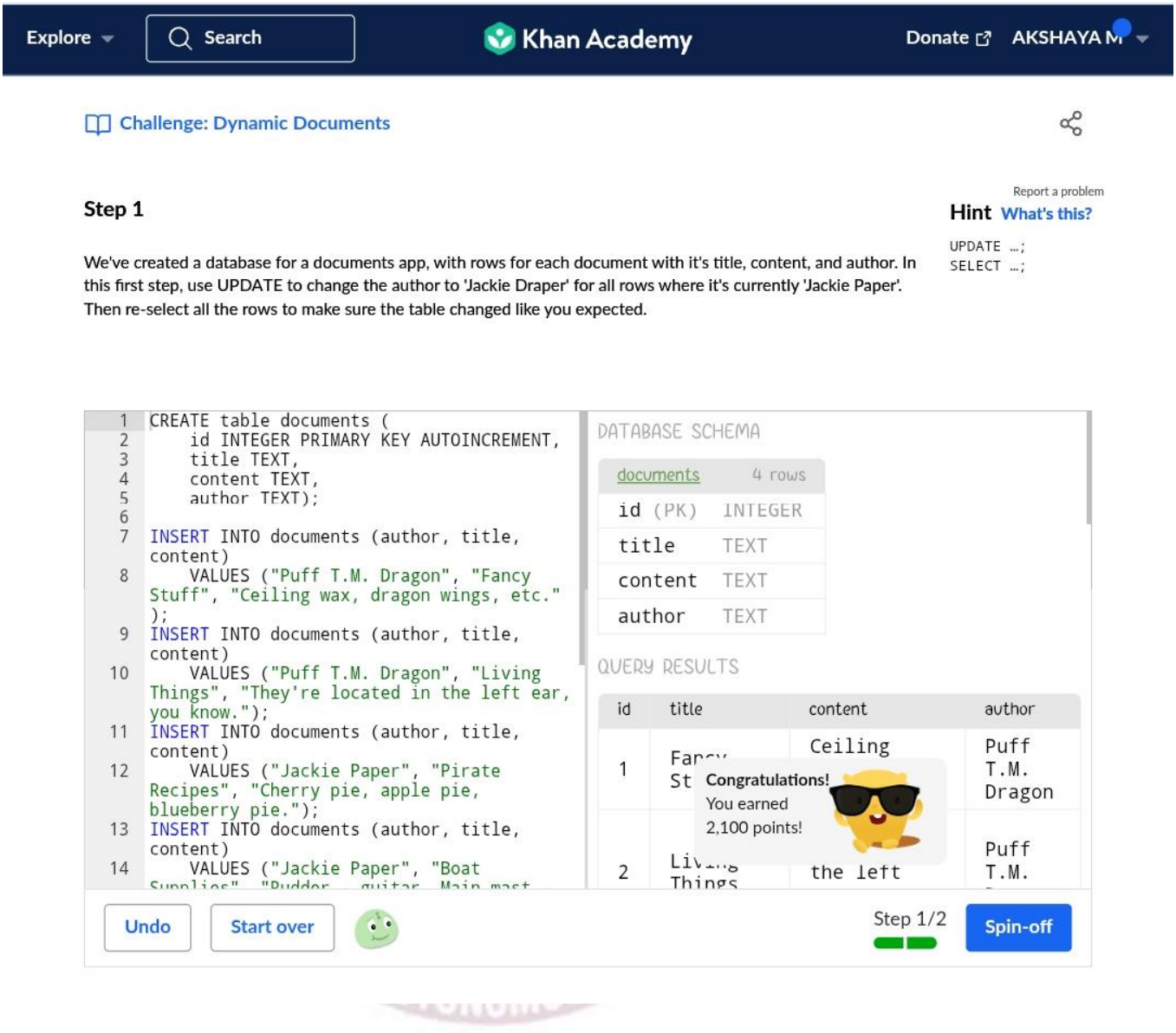
DYNAMIC DOCUMENTS

We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected.

PROGRAM:

```
CREATE table documents (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT,
   content TEXT,
   author TEXT);
INSERT INTO documents (author, title, content)
   VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
   VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
   VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
INSERT INTO documents (author, title, content)
   VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post.");
INSERT INTO documents (author, title, content)
   VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates, heights.");
SELECT * FROM documents;
UPDATE documents
SET author = "Jackie Draper"
WHERE author = "Jackie Paper";
SELECT * FROM documents;
DELETE FROM documents
WHERE title = "Things I'm Afraid Of";
SELECT * FROM documents;
```

OUTPUT:


Explore
Search
Khan Academy
Donate
AKSHAYA M
Challenge: Dynamic Documents
Step 1
We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected.
Report a problem
Hint What's this?
UPDATE ...;
SELECT ...;
CREATE table documents (
id INTEGER PRIMARY KEY AUTOINCREMENT,
title TEXT,
content TEXT,
author TEXT);
INSERT INTO documents (author, title, content)
VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
INSERT INTO documents (author, title, content)
VALUES ("Jackie Paper", "Boat
DATABASE SCHEMA
documents 4 rows
id (PK) INTEGER
title TEXT
content TEXT
author TEXT
QUERY RESULTS
id
title
content
author
1
Ceiling
Puff T.M. Dragon
Congratulations!
You earned 2,100 points!
2
the left
Puff T.M.
Undo
Start over
Step 1/2
Spin-off

CLOTHING ALTERNATION

We've created a database of clothes, and decided we need a price column. Use ALTER to add a 'price' column to the table. Then select all the columns in each row to see what your table lookslike now.

PROGRAM:

```
CREATE TABLE clothes (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT,
   design TEXT);
INSERT INTO clothes (type, design)
   VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
   VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
   VALUES ("blazer", "black sequin");
ALTER TABLE clothes
ADD COLUMN price REAL;
SELECT * FROM clothes;
UPDATE clothes
SET price = 10
WHERE id = 1;
UPDATE clothes
SET price = 20
WHERE id = 2;
UPDATE clothes
SET price = 30
WHERE id = 3;

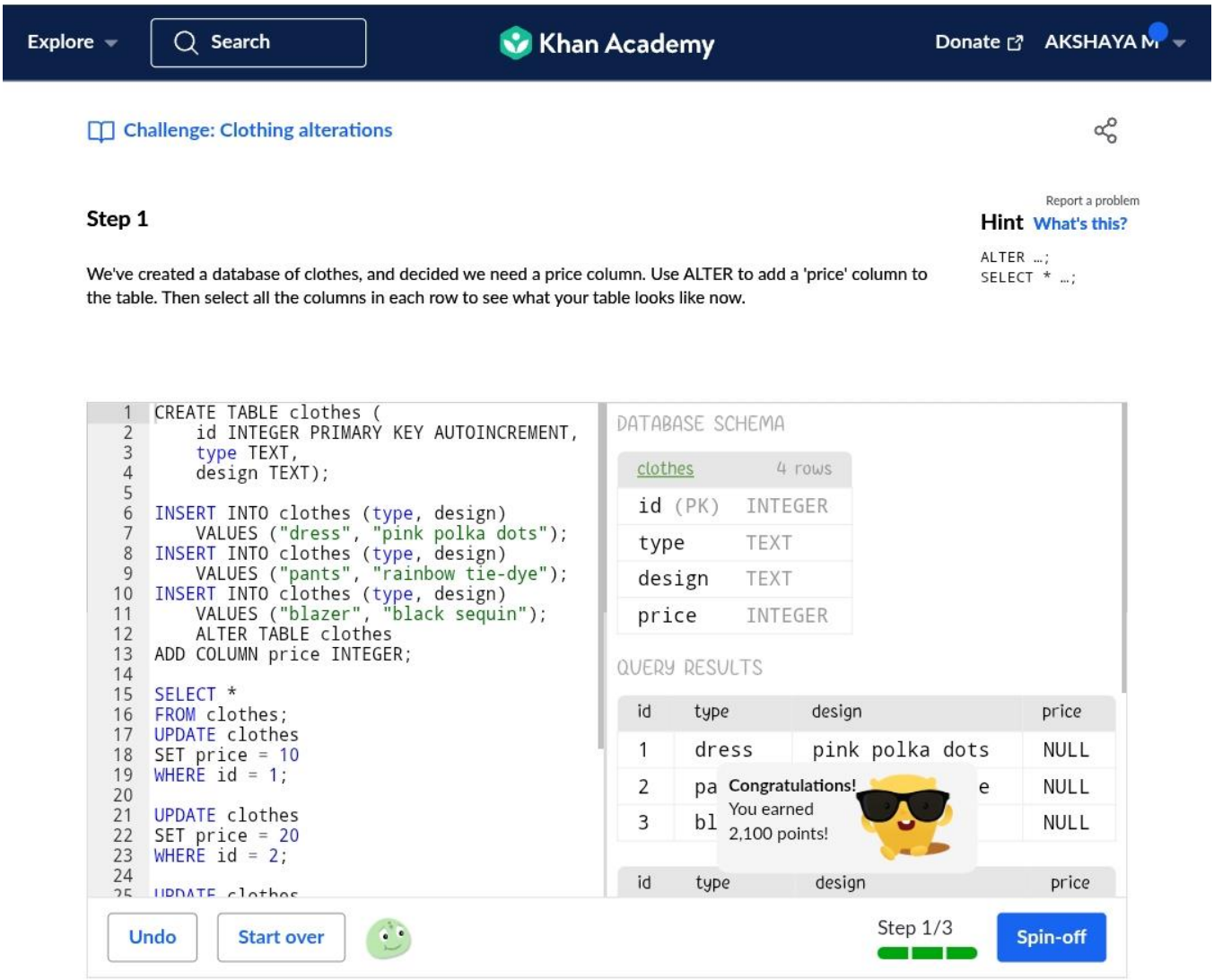

SELECT * FROM clothes;
```

INSERT INTO clothes (type, design, price)

VALUES ("shirt", "blue stripes", 15);

SELECT * FROM clothes;

OUTPUT:



PROJECT: APP IMPERSONATOR

Think about your favorite apps, and pick one that stores your data- like a game that stores scores, an app that lets you post updates, etc. Now in this project, you're going to imagine that the app stores your data in a SQL database (which is pretty likely!), and write SQL statements that might look like their own SQL.

- CREATE a table to store the data.
- INSERT a few example rows in the table.
- Use an UPDATE to emulate what happens when you edit data in the app.
- Use a DELETE to emulate what happens when you delete data in the app.

PROGRAM:

```
/* What does the app's SQL look like? */
CREATE TABLE posts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT,
  content TEXT,
  likes INTEGER
);

INSERT INTO posts (username, content, likes)
VALUES ("alex", "Just finished my homework!", 12);

INSERT INTO posts (username, content, likes)
VALUES ("sam", "Playing my favorite game tonight.", 25);

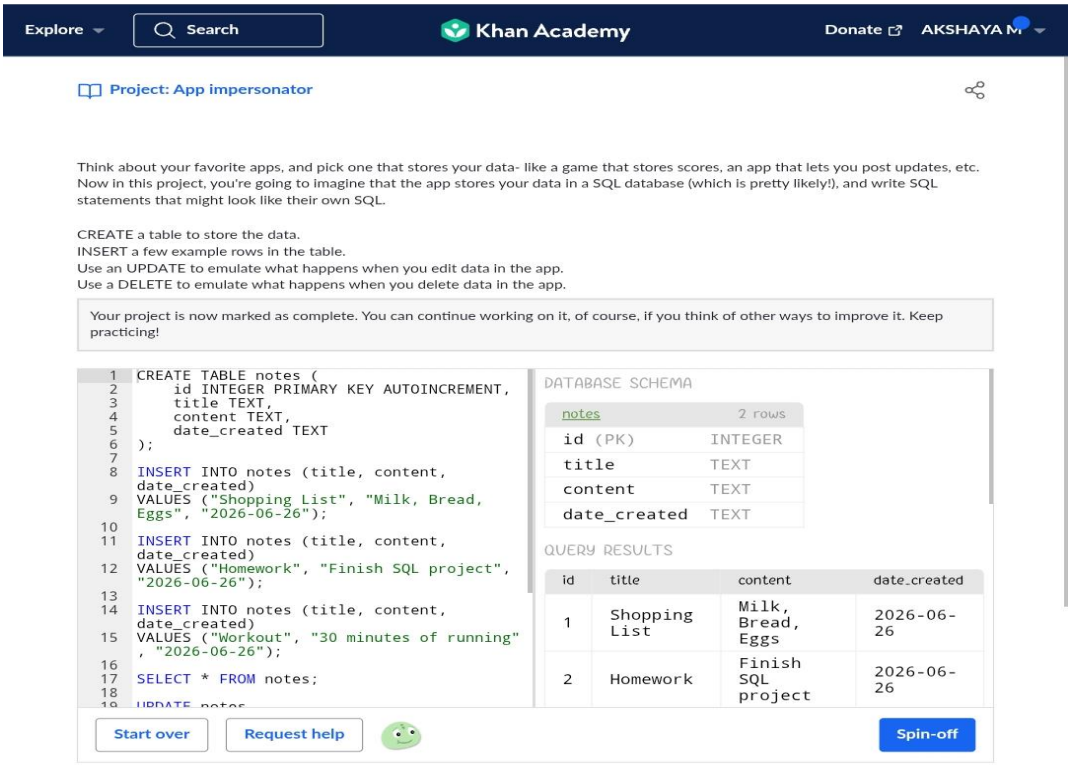
INSERT INTO posts (username, content, likes)
VALUES ("taylor", "Had a great day at the beach.", 18);

/* View all posts */
```

```
SELECT * FROM posts;


/* User edits a post */
UPDATE posts
SET content = "Just finished my homework and studied SQL!"
WHERE id = 1;
/* View updated posts */
SELECT * FROM posts;
/* User deletes a post */
DELETE FROM posts
WHERE id = 2;
/* View final table */
SELECT * FROM posts;
```

OUTPUT:



HACKER RANK

REVISING THE SELECT QUERY I

Query all columns for all American cities in the **CITY** table with populations larger than 100000.

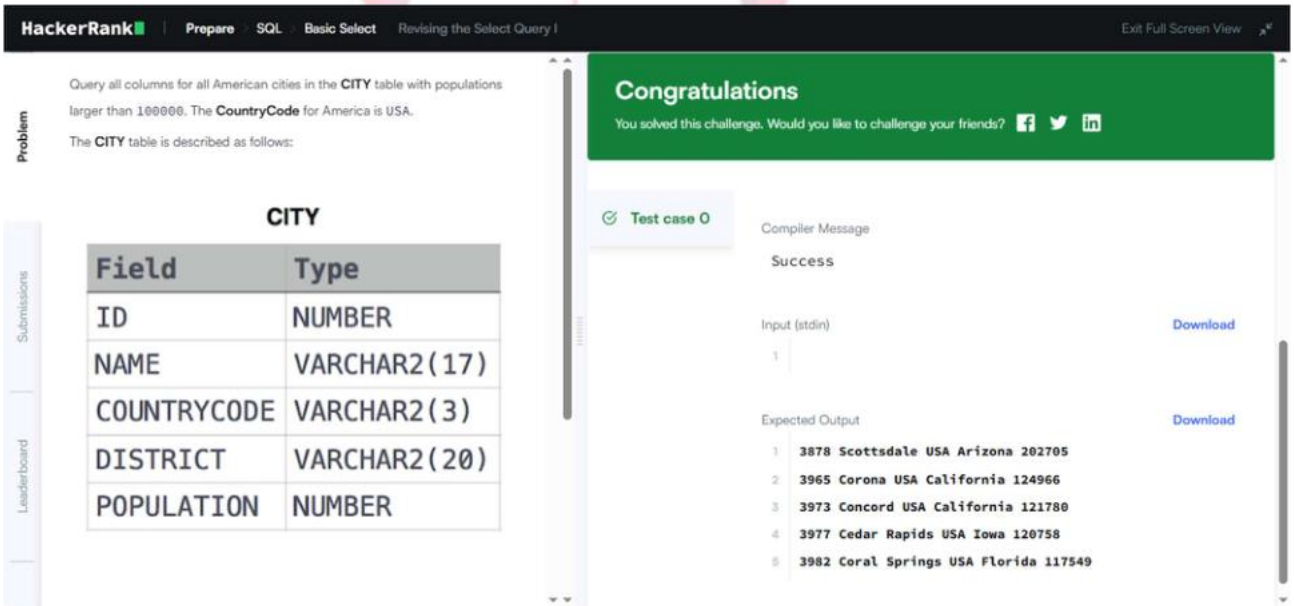
The **CountryCode** for America is USA.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
select * from city where countrycode = 'usa' and population > 100000;
```



REVISING THE SELECT QUERY II

Query the **NAME** field for all American cities in the **CITY** table with populations larger than 120000.

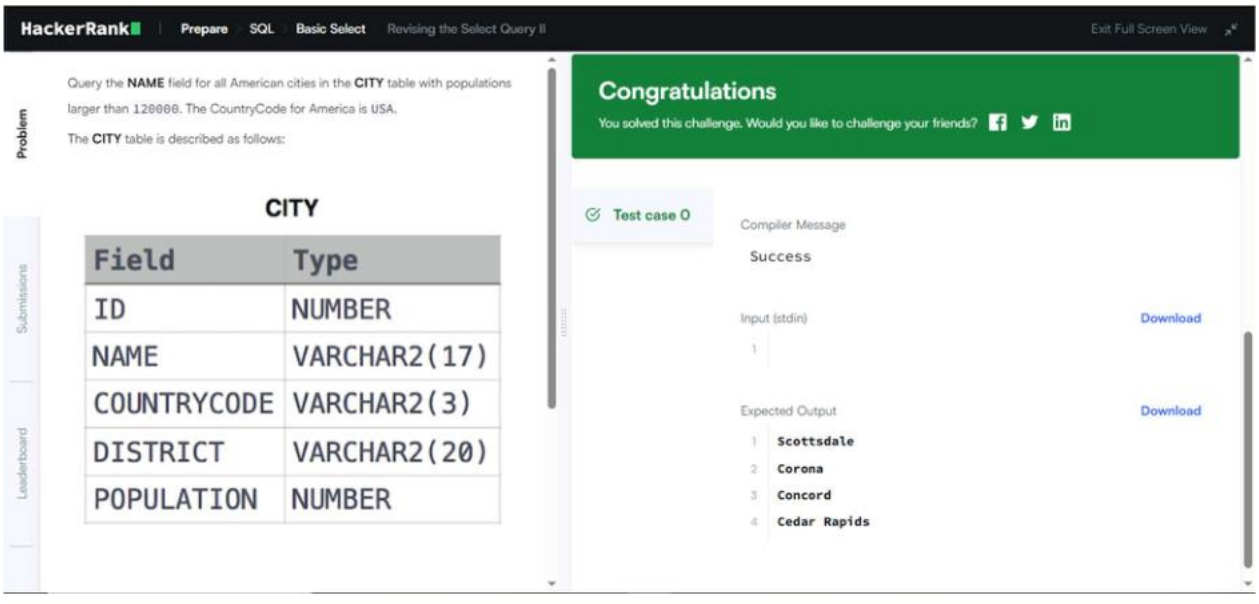
The *CountryCode* for America is USA.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
select name from city where countrycode = 'usa' and population > 120000;
```



SELECT ALL

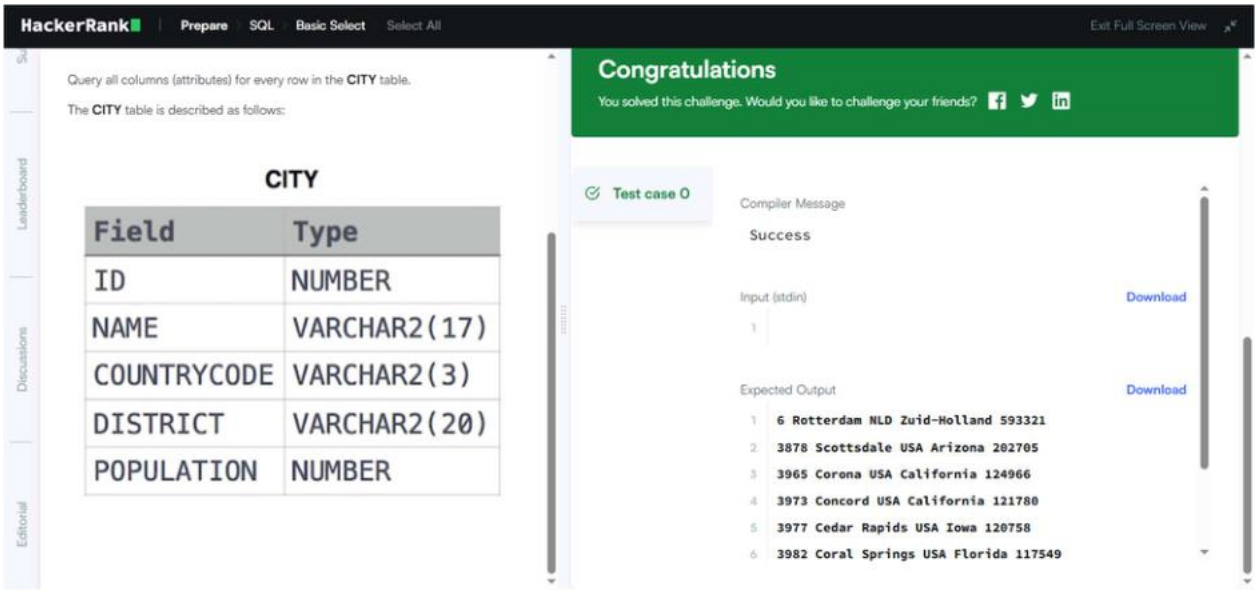
Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select * from city;



SELECT BY ID

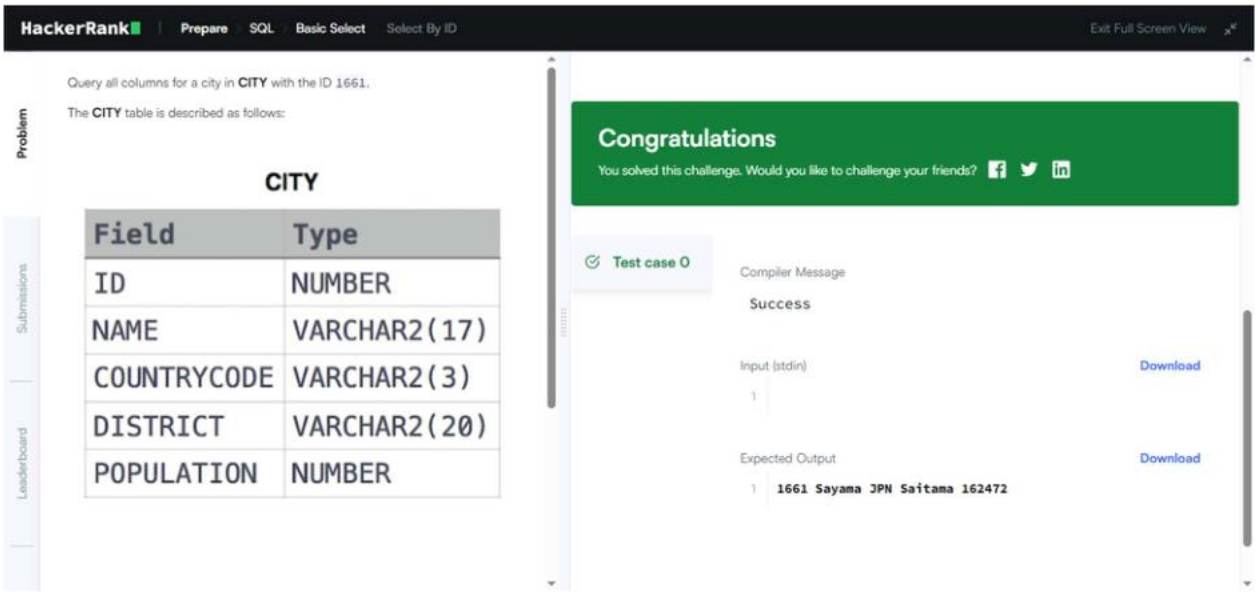
Query all columns for a city in **CITY** with the *ID* 1661.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select * from city where id = 1661;



JAPANESE CITIES' ATTRIBUTES

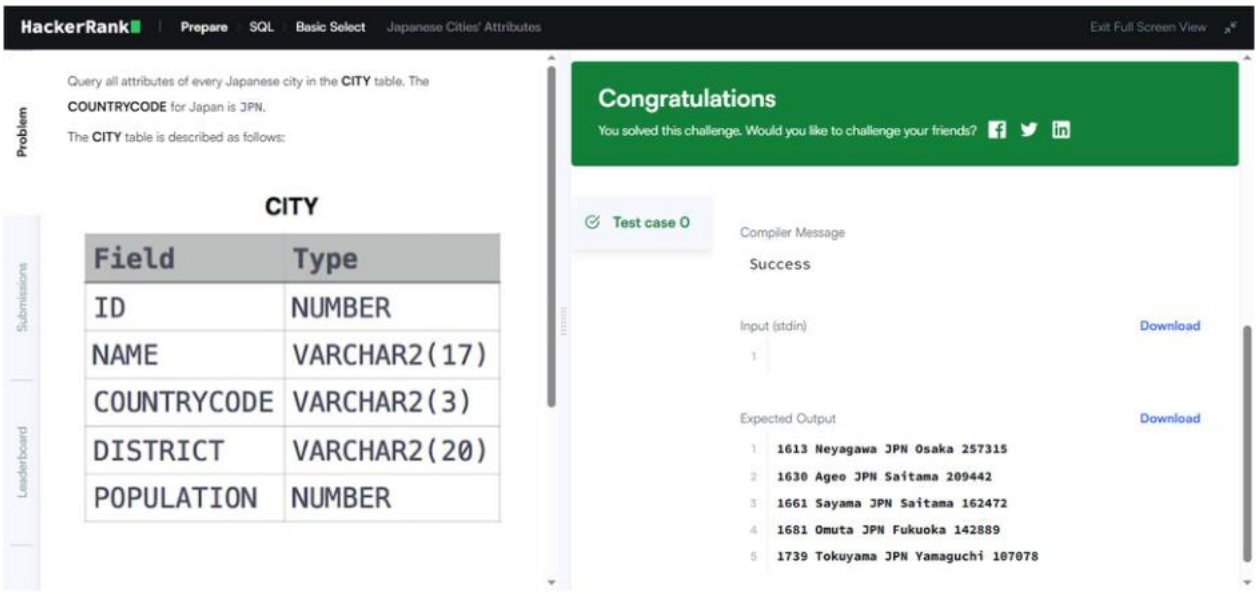
Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is JPN.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

*select * from city where countrycode = 'jpn';*



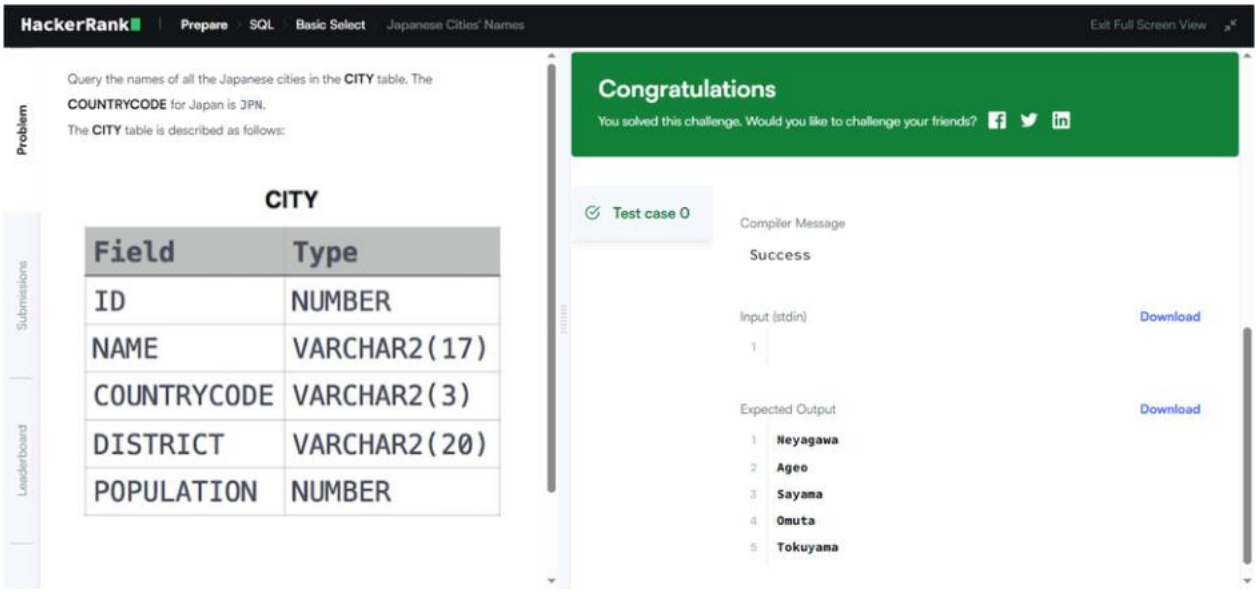
JAPANESE CITIES' NAMES

Query the names of all the Japanese cities in the **CITY** table. The **COUNTRYCODE** for Japan is JPN.

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select name from city where countrycode = 'jpn';



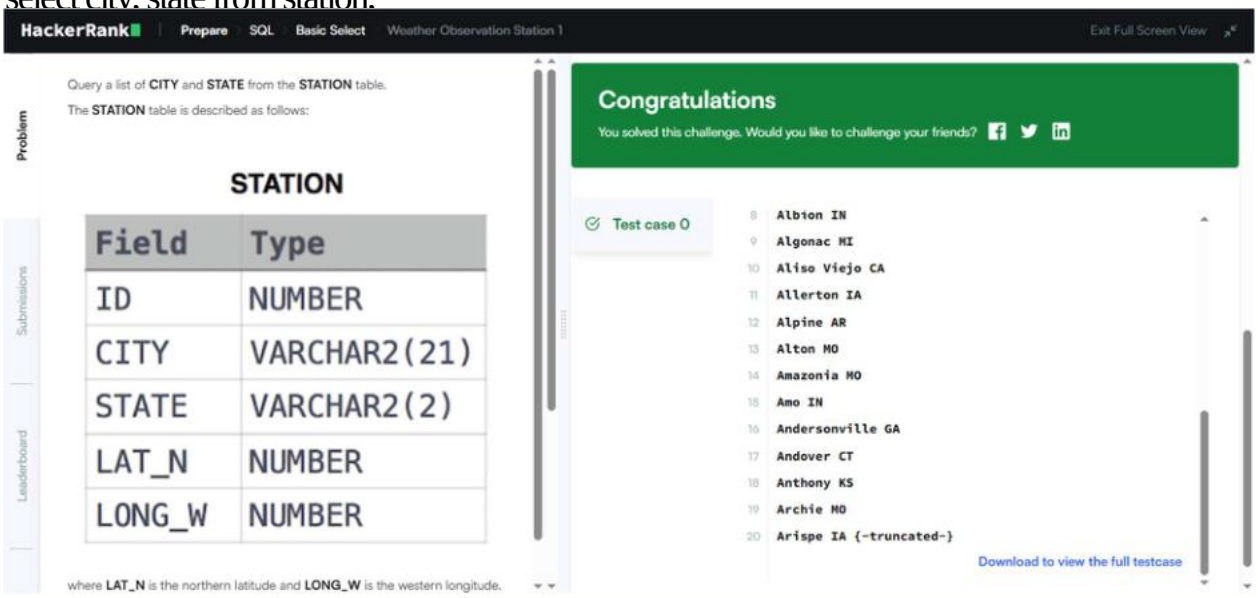
WEATHER OBSERVATION STATION 1

Query a list of **CITY** and **STATE** from the **STATION** table.

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select city, state from station;



WEATHER OBSERVATION STATION 3

Query a list of **CITY** names from **STATION** for cities that have an even **ID** number. Print the results in any

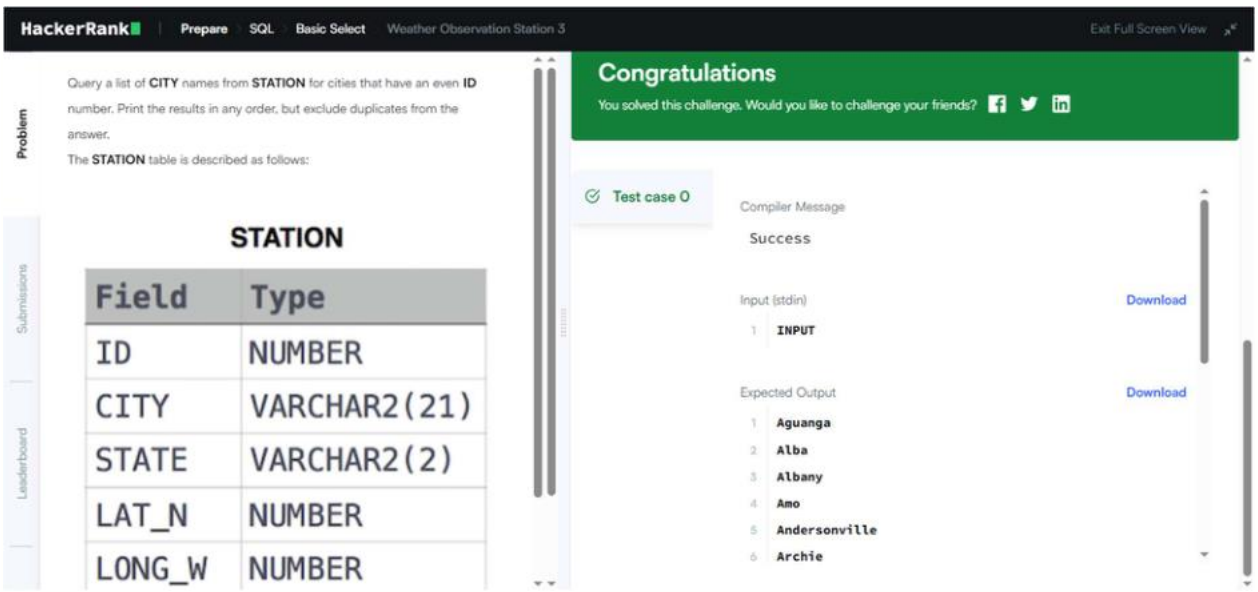
order, but exclude duplicates from the answer.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where id % 2 = 0;



WEATHER OBSERVATION STATION 4

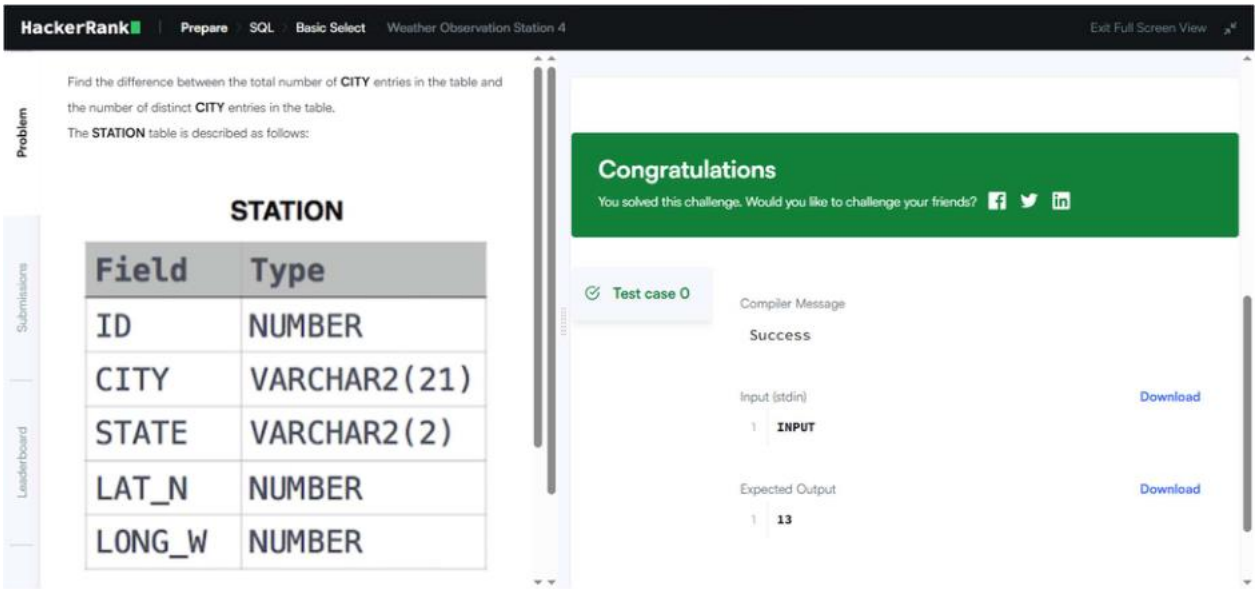
Find the difference between the total number of **CITY** entries in the table and the number of distinct **CITY** entries in the table.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select count(city) - count(distinct city)
from station;
```



WEATHER OBSERVATION STATION 5

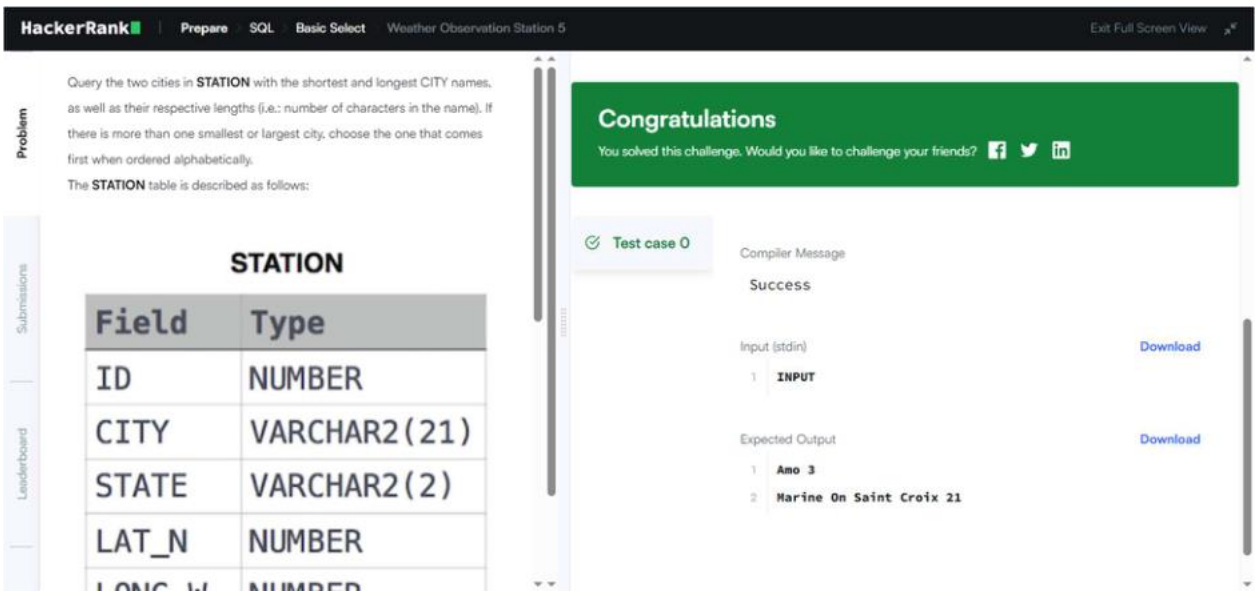
Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select city, length(city)from station order by length(city) asc ,city asc limit 1;
select city, length(city)from stationorder by length(city) desc, city asc limit 1;
```



WEATHER OBSERVATION STATION 6

Query the list of *CITY* names starting with vowels (i.e., a, e, i, o, or u) from **STATION**. Your result *cannot* contain duplicates.

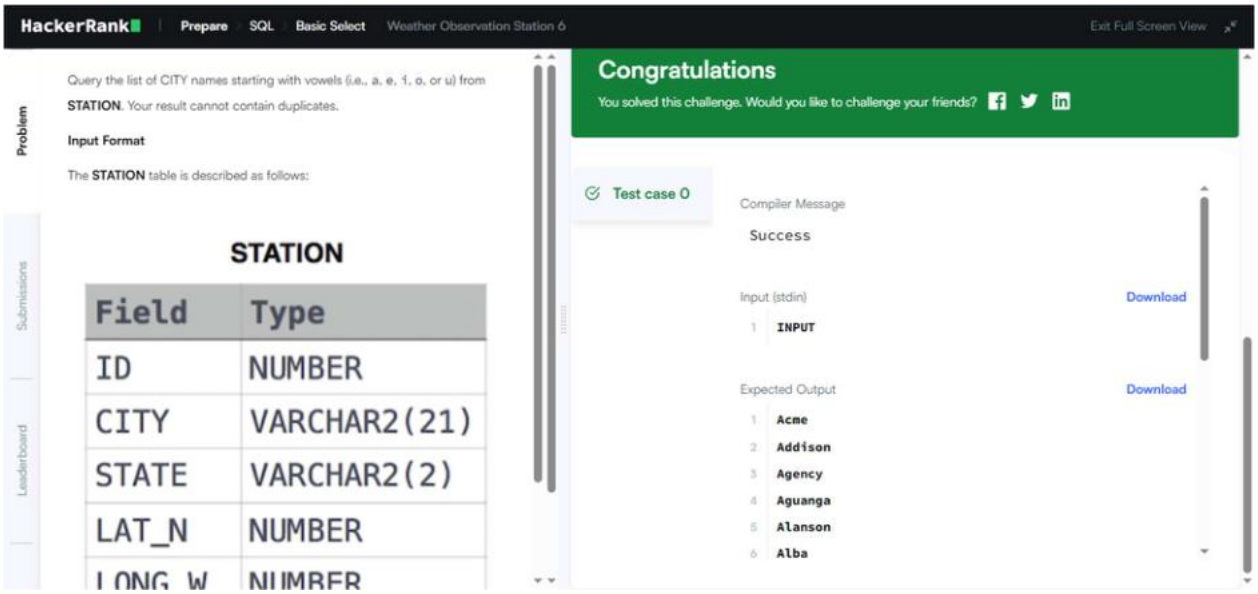
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '^[aeiouaeiou]';



WEATHER OBSERVATION STATION 7

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from **STATION**. Your result *cannot* contain

duplicates.

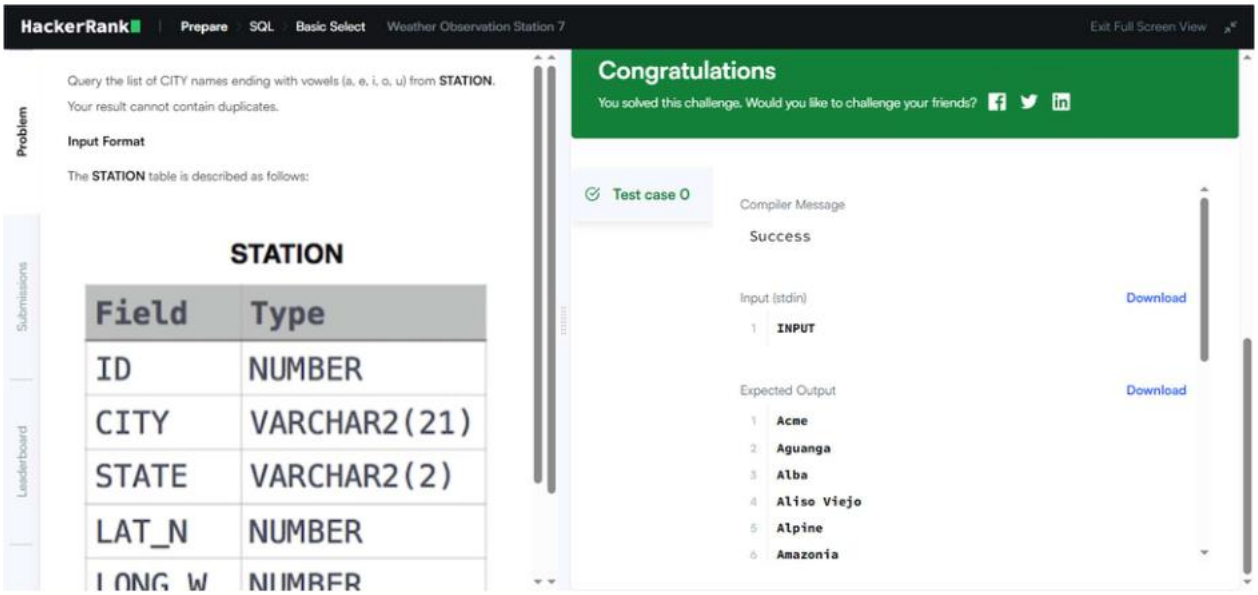
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '[aeiouaeiou]$';



WEATHER OBSERVATION STATION 8

Query the list of *CITY* names from **STATION** which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

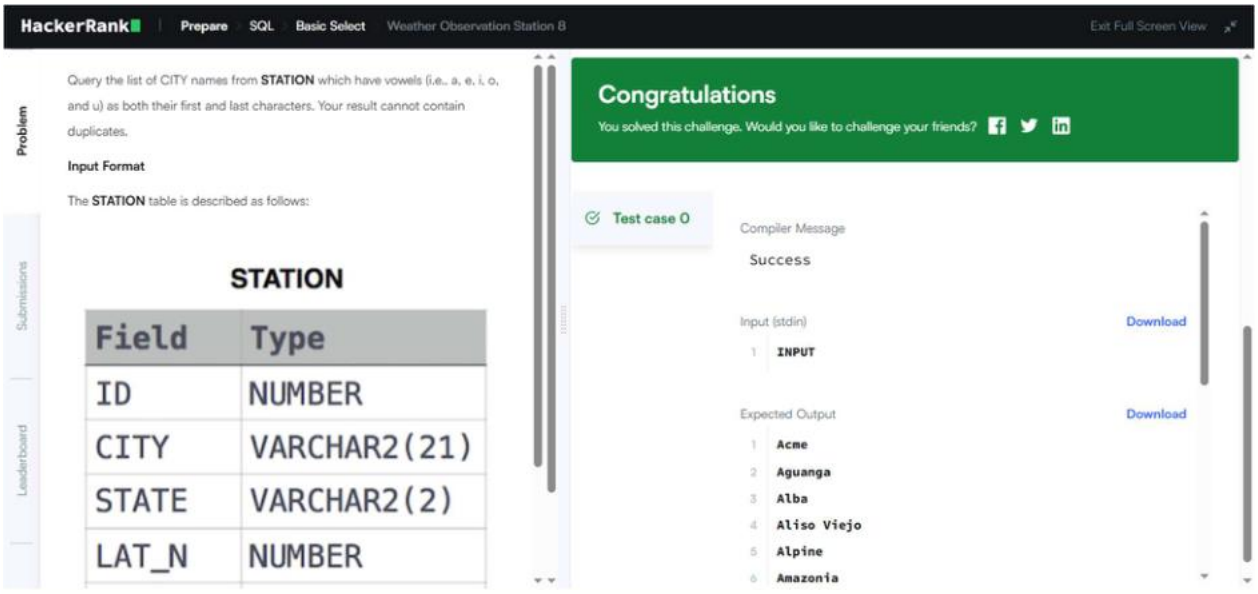
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select distinct city from station where city regexp '^[aeiou].*[aeiou]$';
```



WEATHER OBSERVATION STATION 9

Query the list of *CITY* names from **STATION** that *do not start* with vowels. Your result cannot contain duplicates.

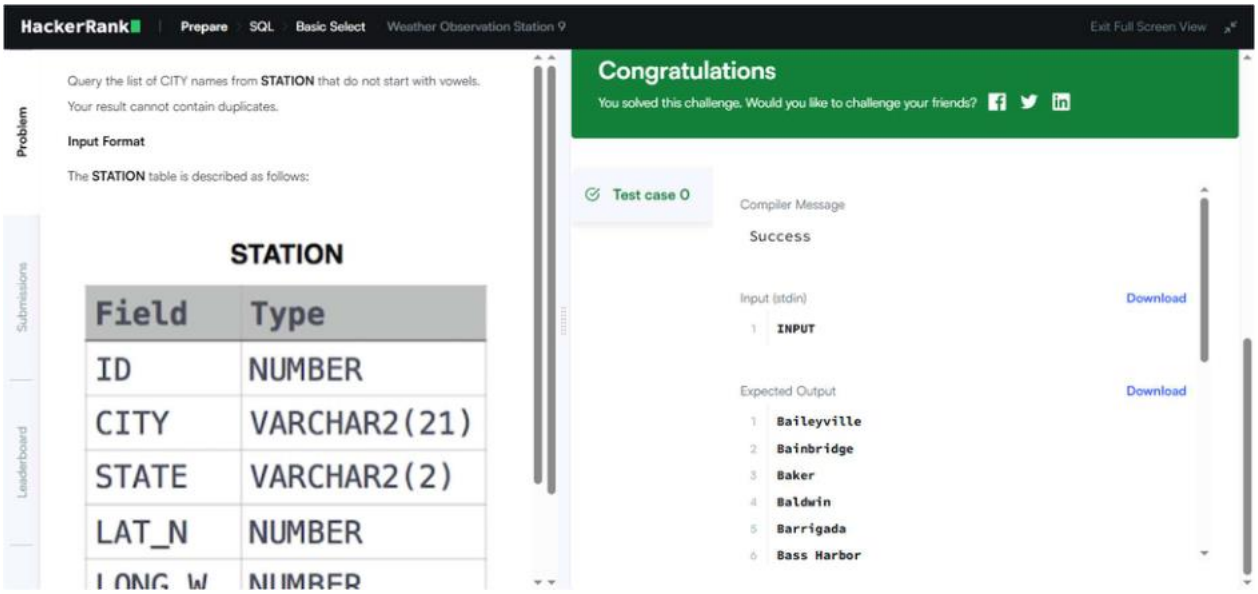
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select distinct city from station where city regexp '^[aeiou].*[aeiou]$';
```



WEATHER OBSERVATION STATION 10

Query the list of *CITY* names from **STATION** that *do not end* with vowels. Your result cannot contain duplicates.

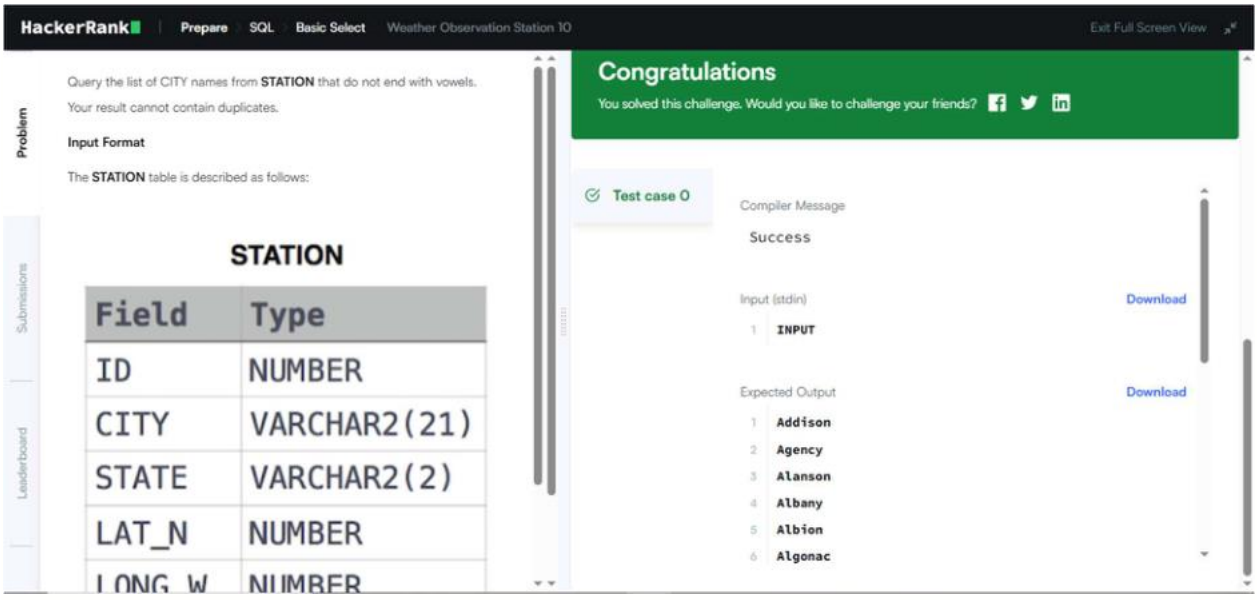
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select distinct city from station where city regexp '[^aeiouaeiou]$';
```



WEATHER OBSERVATION STATION 11

Query the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels.

Your result cannot contain duplicates.

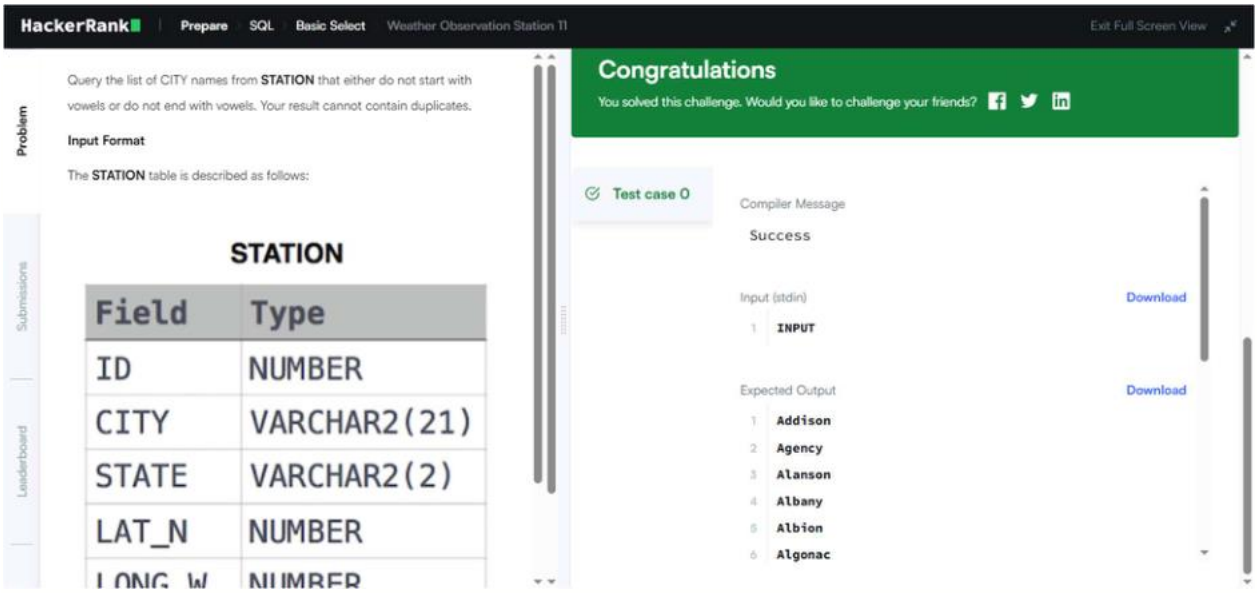
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select distinct city from station where left(city, 1) not in ('a','e','i','o','u','a','e','i','o','u') or right(city, 1) not in ('a','e','i','o','u','a','e','i','o','u');
```



WEATHER OBSERVATION STATION 12

Query the list of *CITY* names from **STATION** that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates.

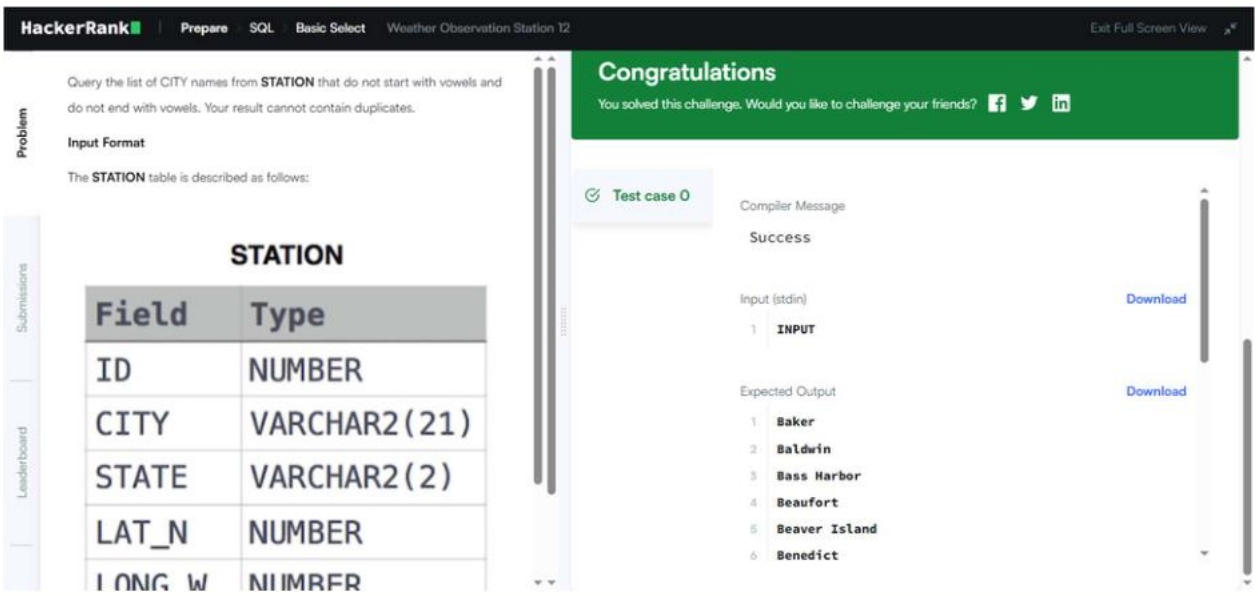
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select distinct city from station where city not regexp '^[aeiouaeiou]' and city not regexp '[aeiouaeiou]$';
```



HIGHER THAN 75 MARKS

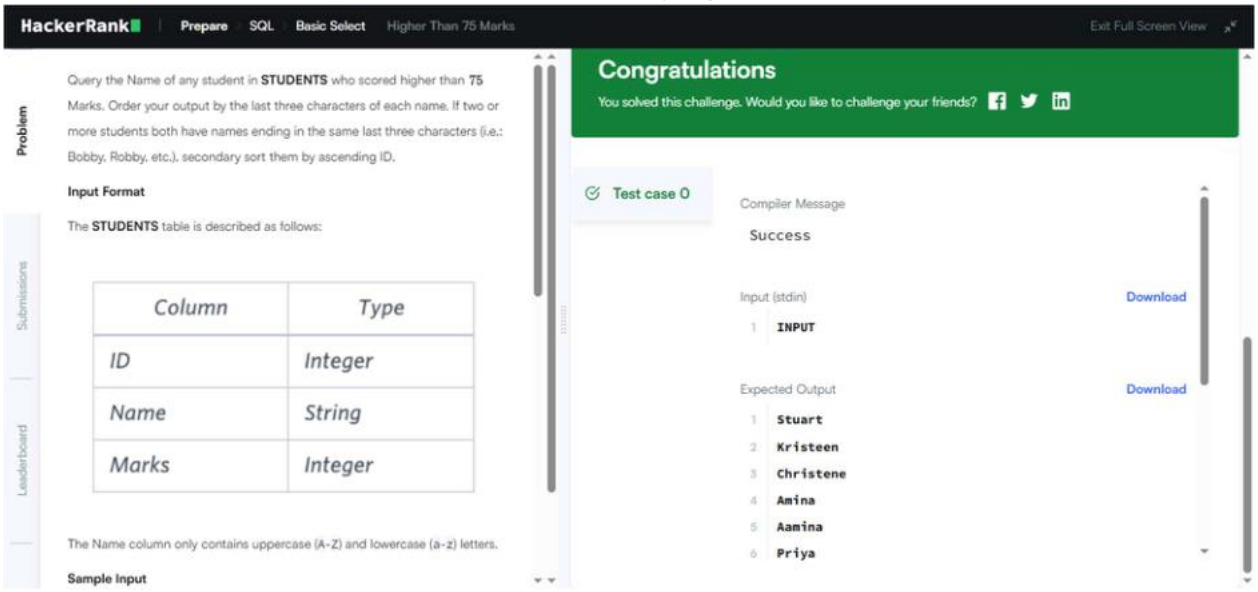
Query the *Name* of any student in **STUDENTS** who scored higher than *Marks*. Order your output by the *last*

three characters of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending *ID*.

Input Format

Column	Type
ID	Integer
Name	String
Marks	Integer

select name from students where marks > 75 order by right(name, 3), id asc;



EMPLOYEE NAMES

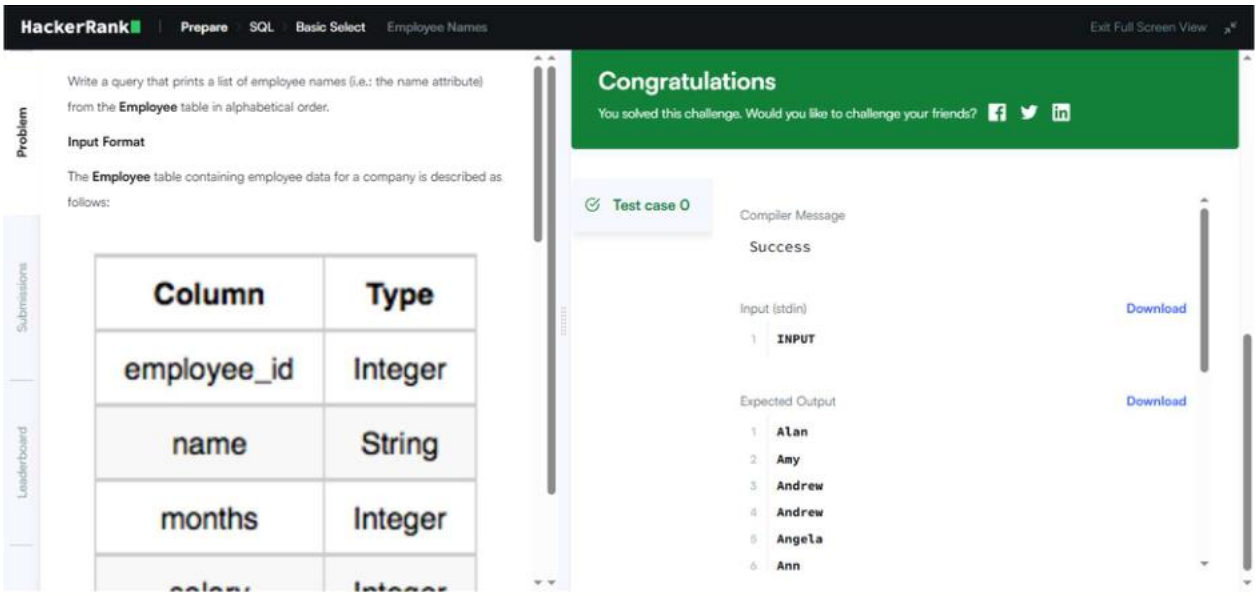
Write a query that prints a list of employee names (i.e.: the *name* attribute) from the **Employee** table in alphabetical order.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

select name from employee order by name;



EMPLOYEE SALARIES

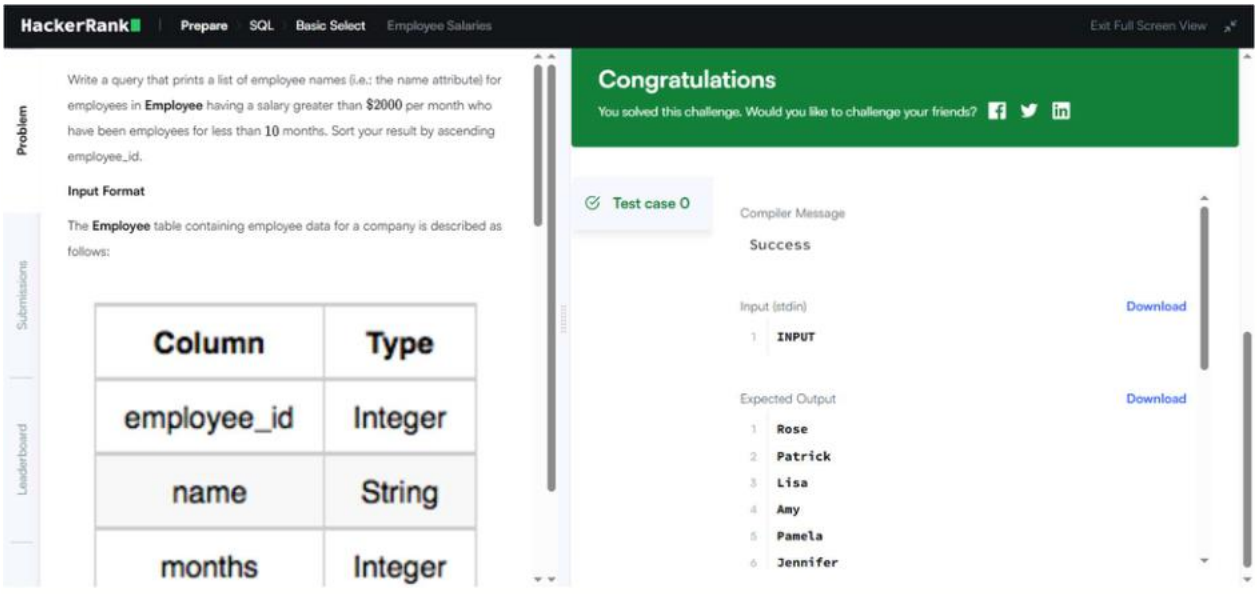
Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in **Employee** having a salary greater than per month who have been employees for less than months. Sort your result by ascending *employee_id*.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

select name from employee where salary > 2000 and months < 10 order by employee_id asc;



TYPE OF TRIANGLE

Write a query identifying the *type* of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

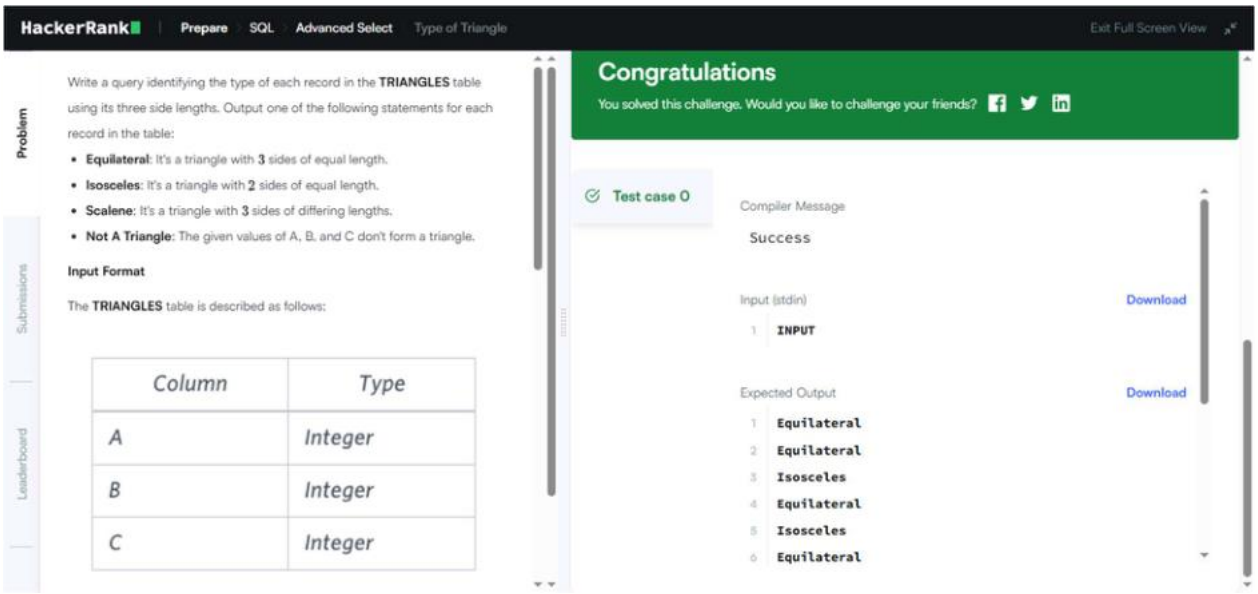
- **Equilateral**: It's a triangle with sides of equal length.
- **Isosceles**: It's a triangle with sides of equal length.
- **Scalene**: It's a triangle with sides of differing lengths.
- **Not A Triangle**: The given values of *A*, *B*, and *C* don't form a triangle.

Input Format

The **TRIANGLES** table is described as follows:

Column	Type
A	Integer
B	Integer
C	Integer

```
select
   case
      when a + b <= c or a + c <= b or b + c <= a then 'not a
triangle'
      when a = b and b = c then 'equilateral'
      when a = b or b = c or a = c then 'isosceles'
      else 'scalene'
   end
from triangles;
```


HackerRank
Prepare
SQL
Advanced Select
Type of Triangle
Exit Full Screen View
Problem
Submissions
Leaderboard
Write a query identifying the type of each record in the TRIANGLES table using its three side lengths. Output one of the following statements for each record in the table:
Equilateral: It's a triangle with 3 sides of equal length.
Isosceles: It's a triangle with 2 sides of equal length.
Scalene: It's a triangle with 3 sides of differing lengths.
Not A Triangle: The given values of A, B, and C don't form a triangle.
Input Format
The TRIANGLES table is described as follows:
Column
Type
A
Integer
B
Integer
C
Integer
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 Equilateral
2 Equilateral
3 Isosceles
4 Equilateral
5 Isosceles
6 Equilateral

REVISING AGGREGATIONS - THE COUNT FUNCTION

Query a *count* of the number of cities in **CITY** having a *Population* larger than .

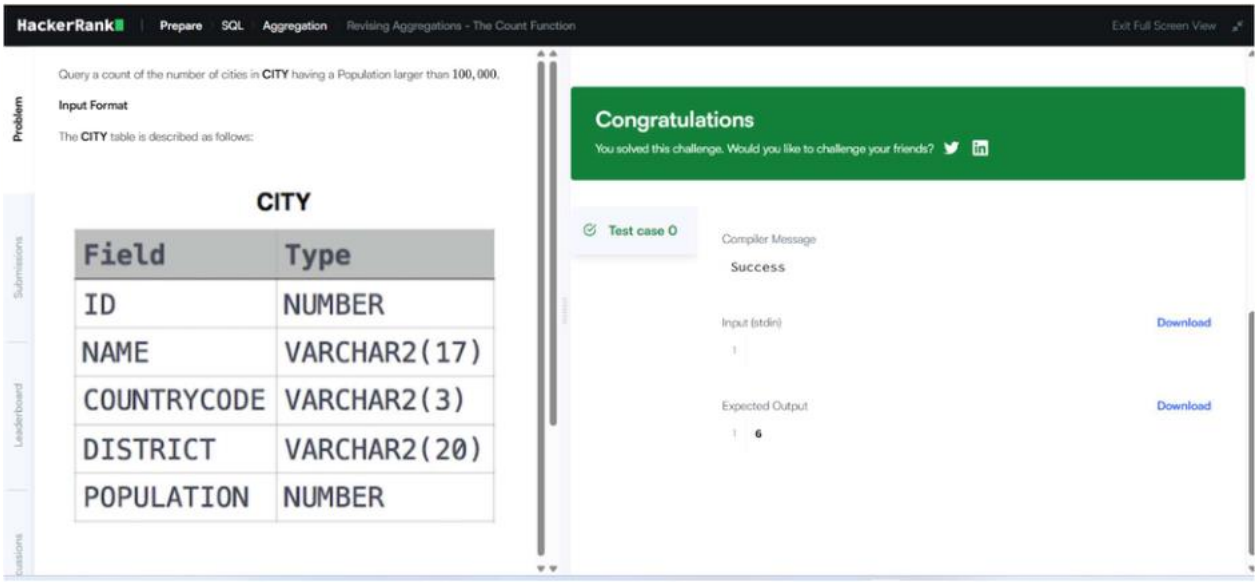
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT COUNT(*) FROM CITY WHERE POPULATION > 100000;
```



REVISING AGGREGATIONS - THE SUM FUNCTION

Query the total population of all cities in **CITY** where *District* is **California**.

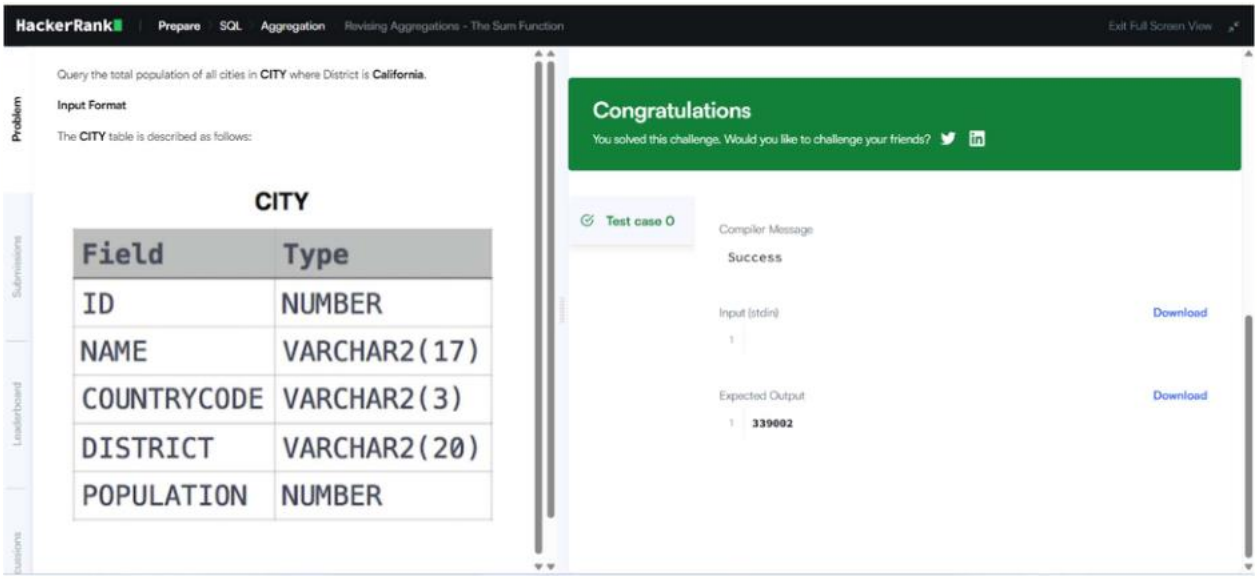
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT SUM(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```



REVISING AGGREGATIONS – AVERAGES

Query the average population of all cities in **CITY** where *District* is **California**.

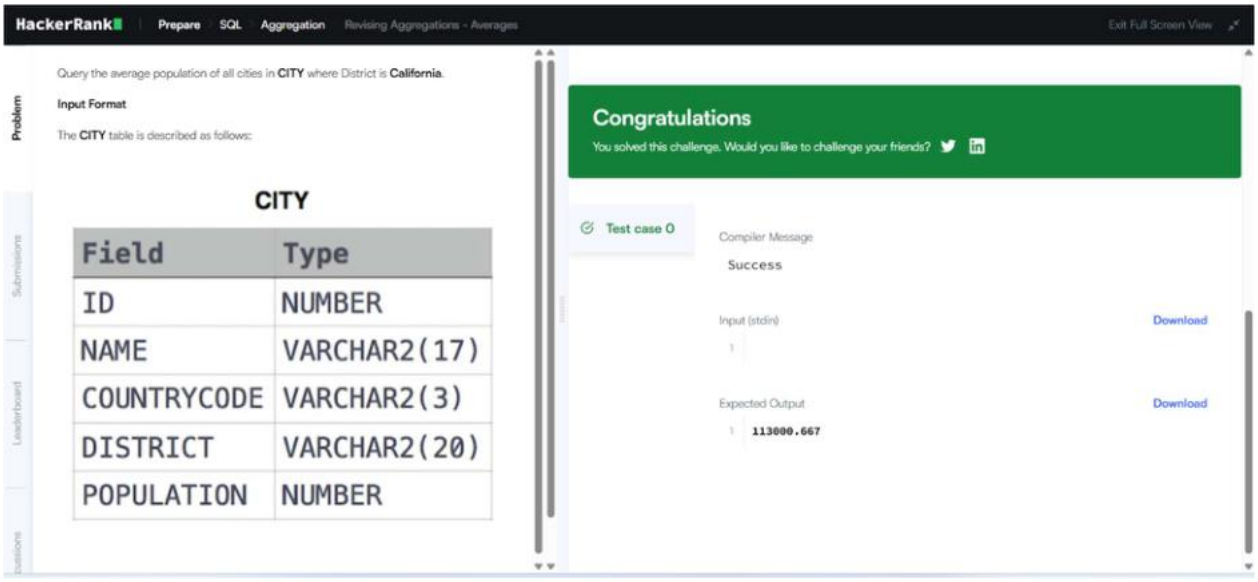
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

SELECT AVG(POPULATION) FROM CITY WHERE DISTRICT = 'California';



AVERAGE POPULATION

Query the average population for all cities in **CITY**, rounded *down* to the nearest integer.

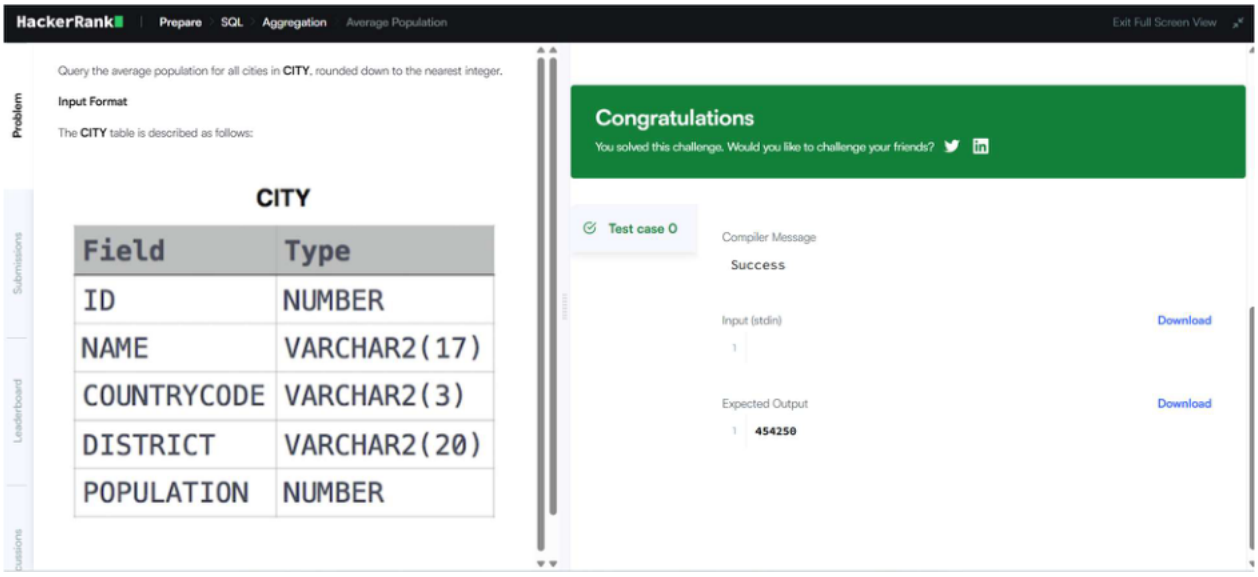
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT FLOOR (AVG(POPULATION)) FROM CITY;
```



JAPAN POPULATION

Query the sum of the populations for all Japanese cities in **CITY**. The *COUNTRYCODE* for Japan is **JPN**.

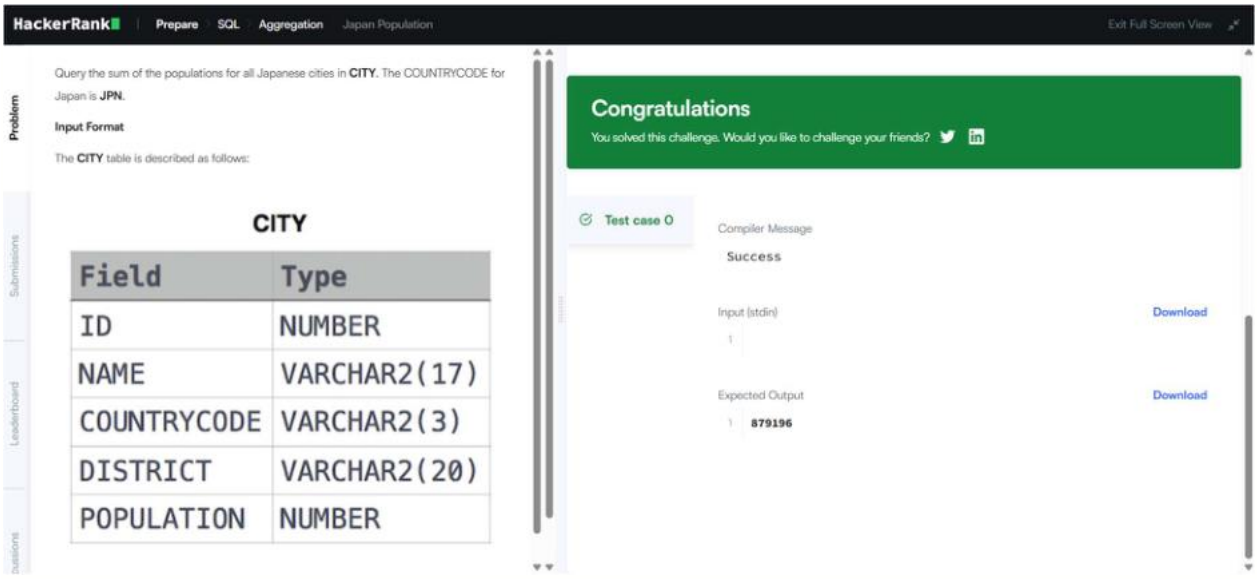
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT SUM(POPULATION) FROM CITY WHERE COUNTRYCODE = 'JPN';
```



POPULATION DENSITY DIFFERENCE

Query the difference between the maximum and minimum populations in **CITY**.

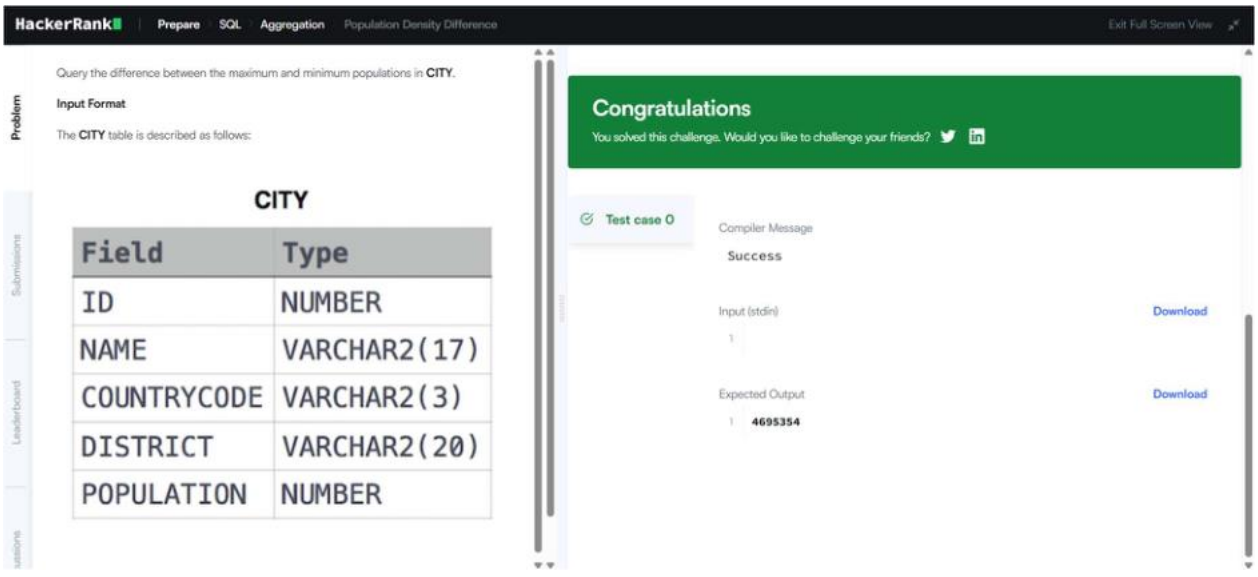
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT MAX(POPULATION) - MIN(POPULATION) FROM CITY;
```



THE BLUNDER

Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

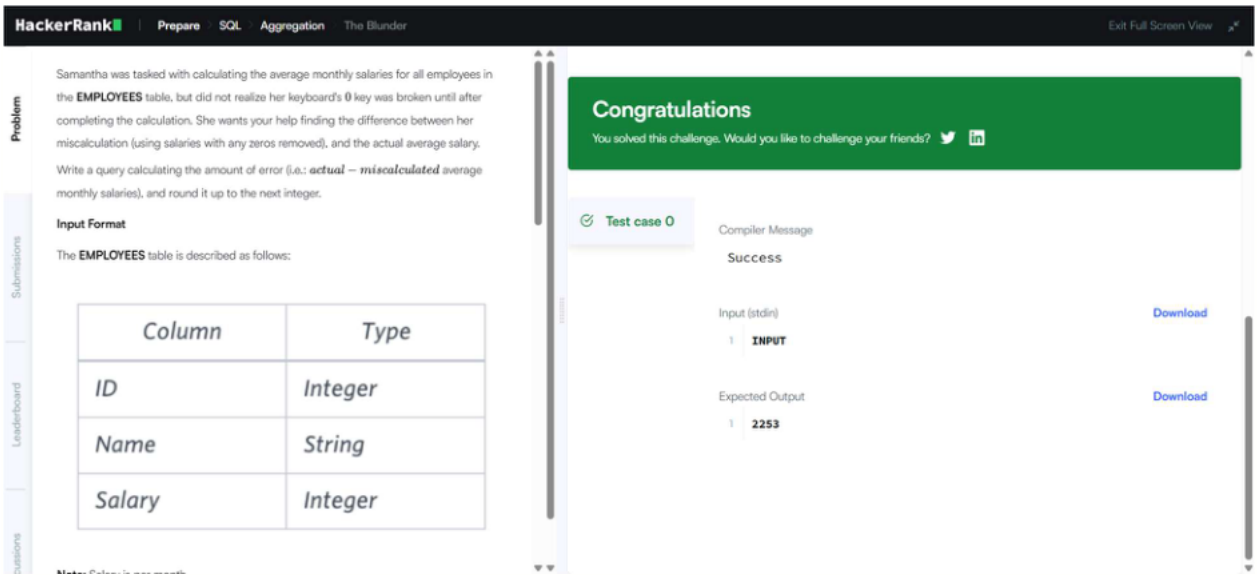
Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

The **EMPLOYEES** table is described as follows:

Column	Type
ID	Integer
Name	String
Salary	Integer

```
SELECT CEIL( AVG(Salary) - AVG(CAST(REPLACE(Salary, '0', '') AS
UNSIGNED)))FROM
EMPLOYEES;
```



TOP EARNERS

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the **Employee** table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

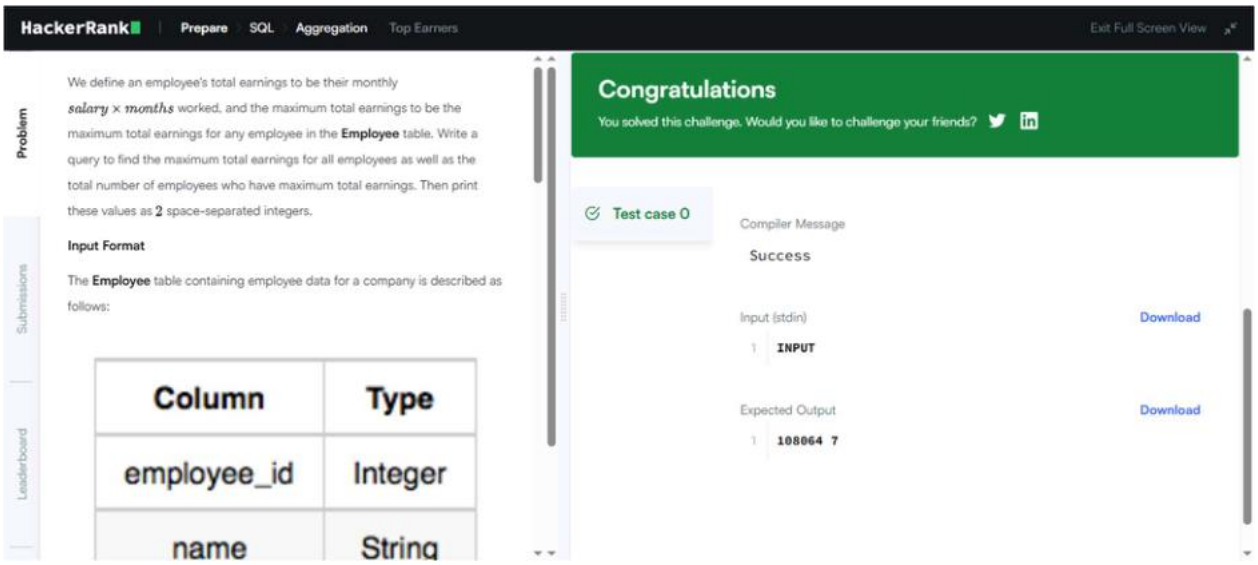
Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

```
SELECT
   (salary * months) AS earnings,
   COUNT(*) AS total_count
FROM EMPLOYEE
GROUP BY earnings
ORDER BY earnings DESC
LIMIT 1;
```

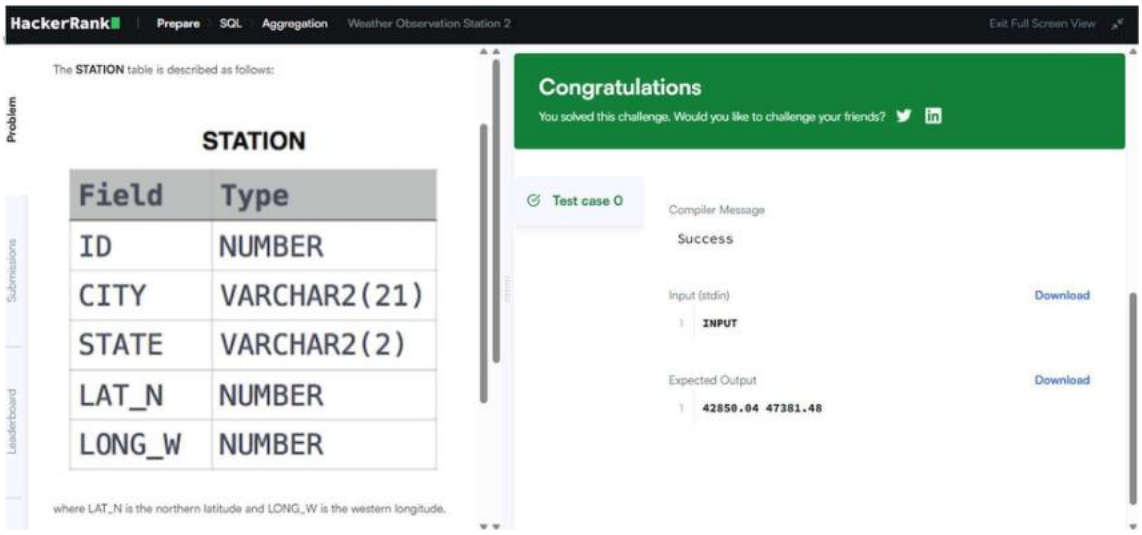

HackerRank
Prepare
SQL
Aggregation
Top Earners
Exit Full Screen View
Problem
Submissions
Leaderboard
We define an employee's total earnings to be their monthly salary × months worked, and the maximum total earnings to be the maximum total earnings for any employee in the Employee table. Write a query to find the maximum total earnings for all employees as well as the total number of employees who have maximum total earnings. Then print these values as 2 space-separated integers.
Input Format
The Employee table containing employee data for a company is described as follows:
Column
Type
employee_id
Integer
name
String
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 108064 7

WEATHER OBSERVATION STATION 2

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER



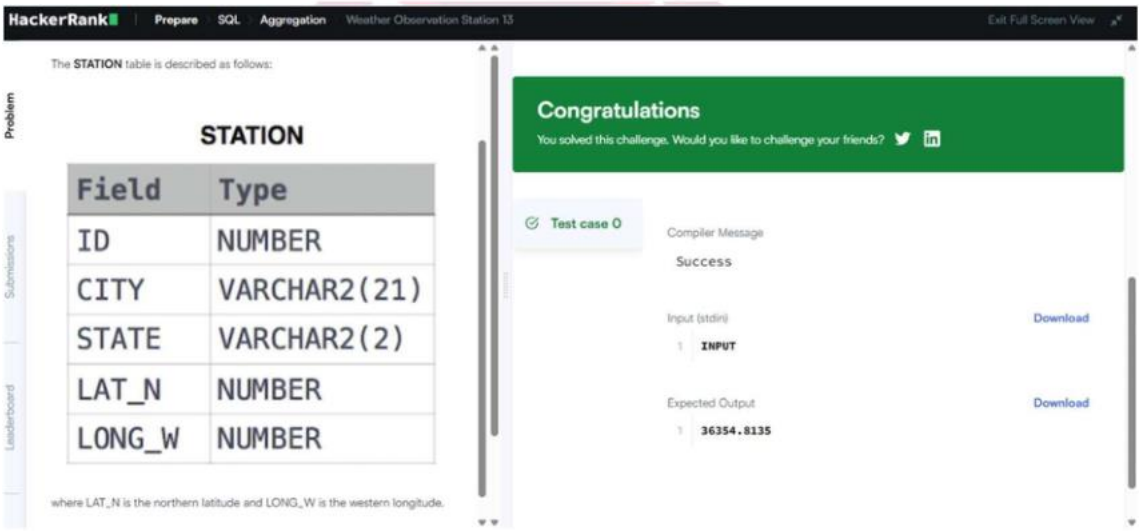
WEATHER OBSERVATION STATION 13

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



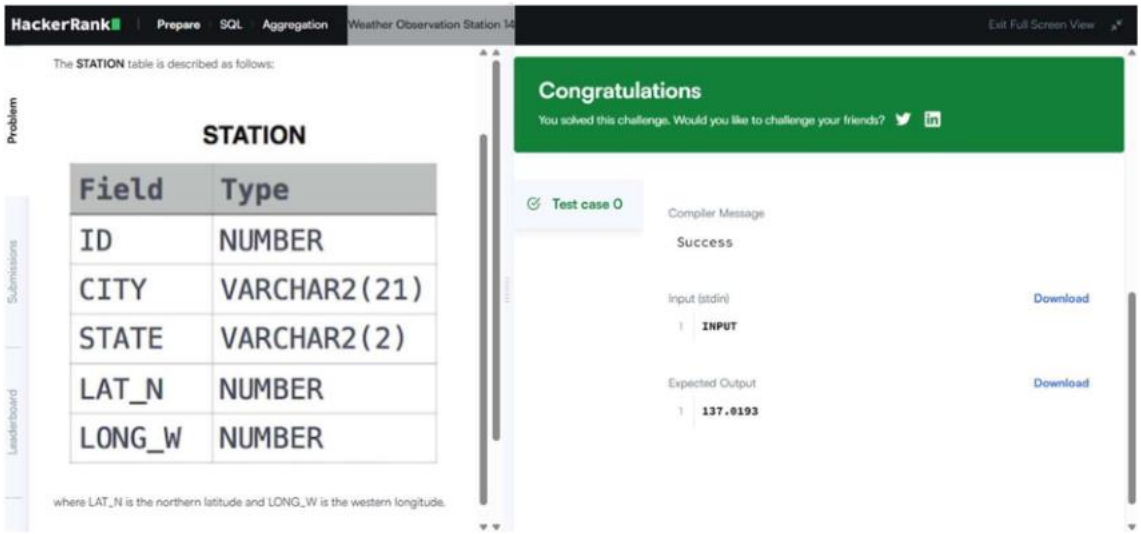
WEATHER OBSERVATION STATION 14

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



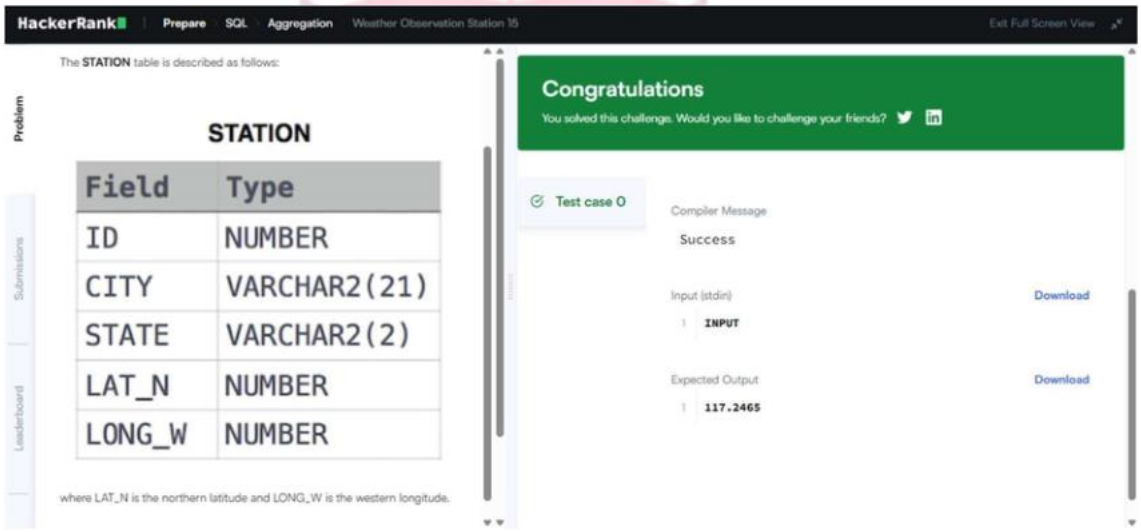
WEATHER OBSERVATION STATION 15

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



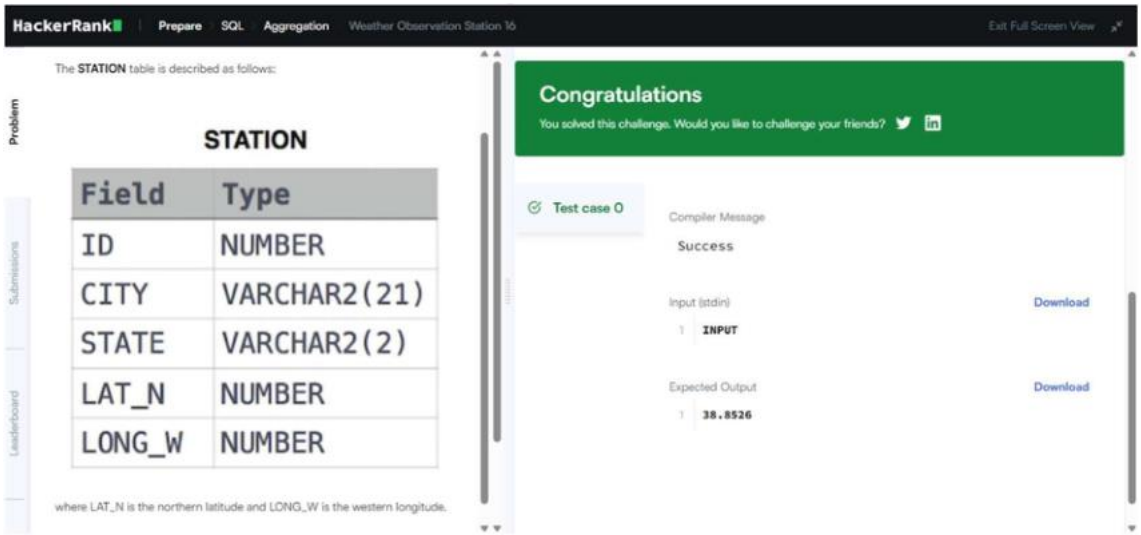
WEATHER OBSERVATION STATION 16

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



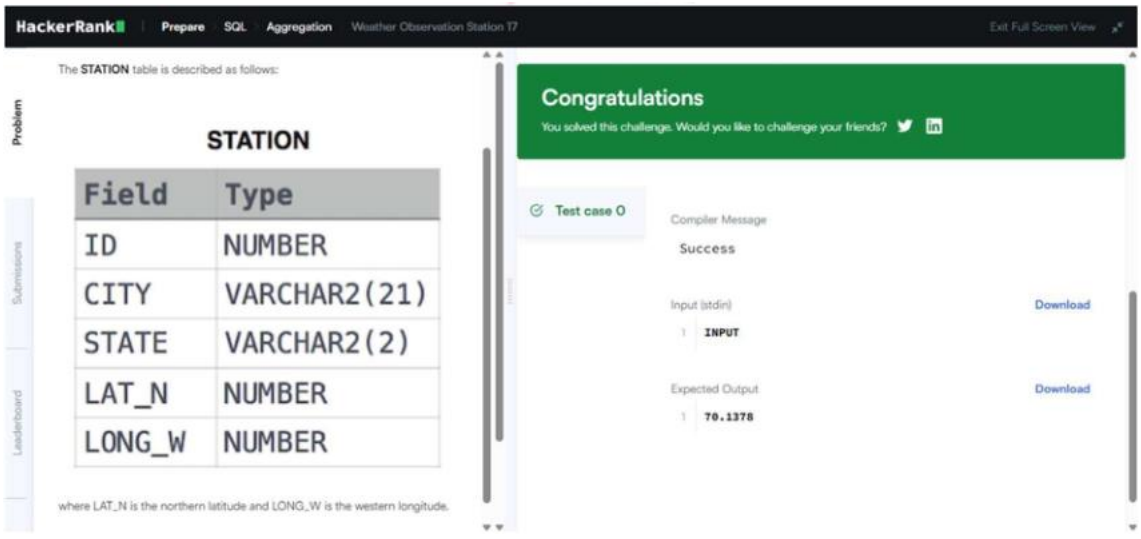
WEATHER OBSERVATION STATION 17

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



POPULATION CENSUS

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the CONTINENT is 'Asia'.

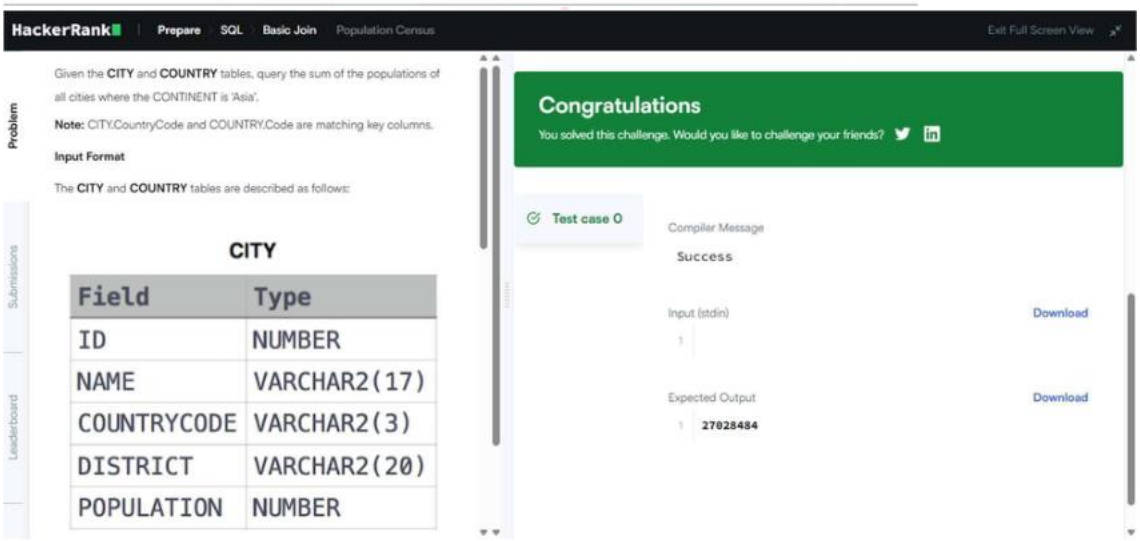
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



AFRICAN CITIES

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the CONTINENT is 'Africa'.

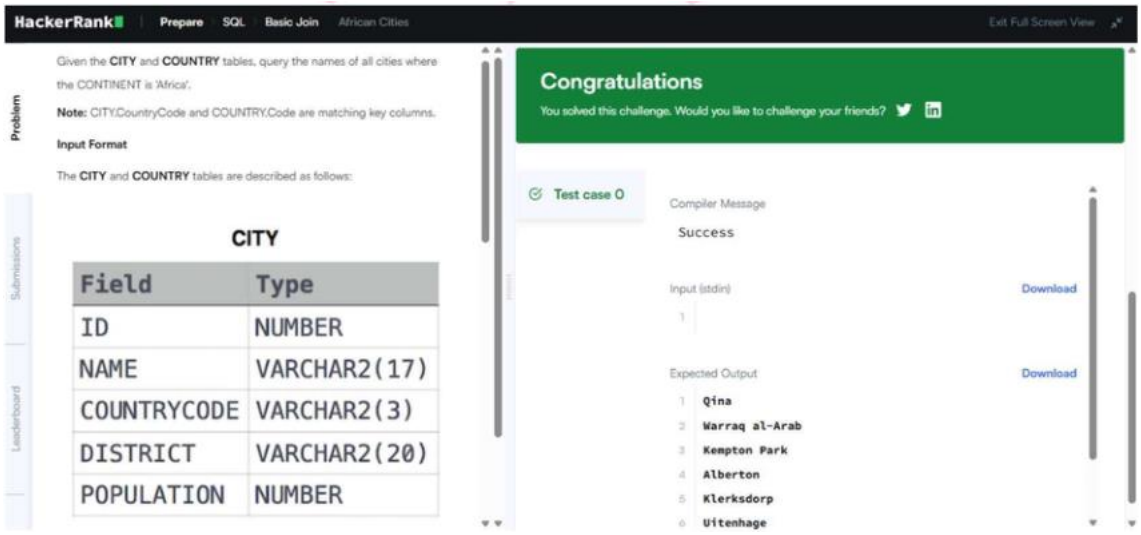
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



AVERAGE POPULATION OF EACH CONTINENT

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (COUNTRY.Continent) and their respective average city populations (CITY.Population) rounded down to the nearest integer.

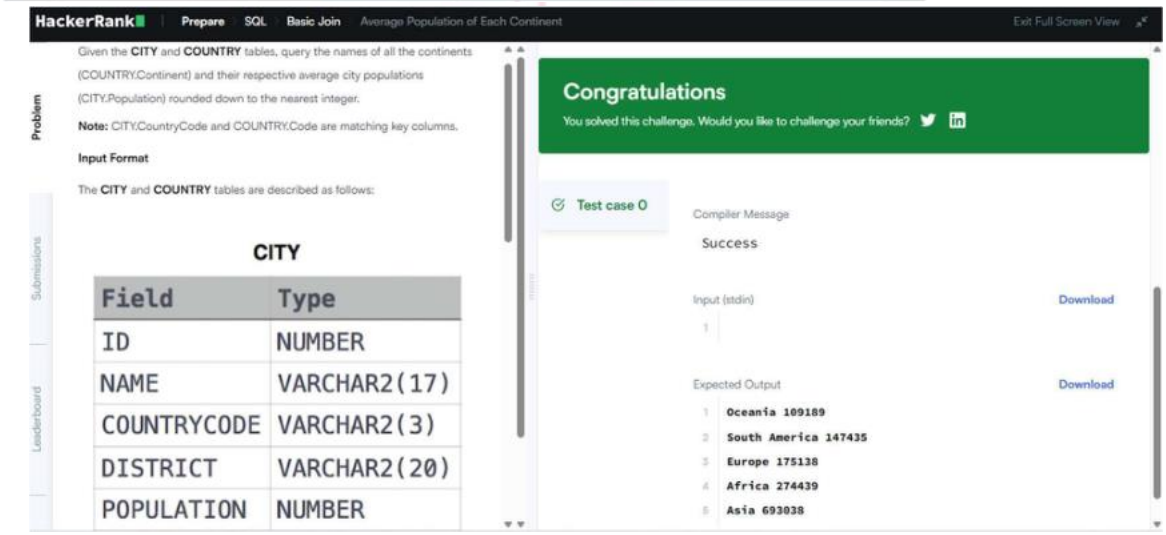
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

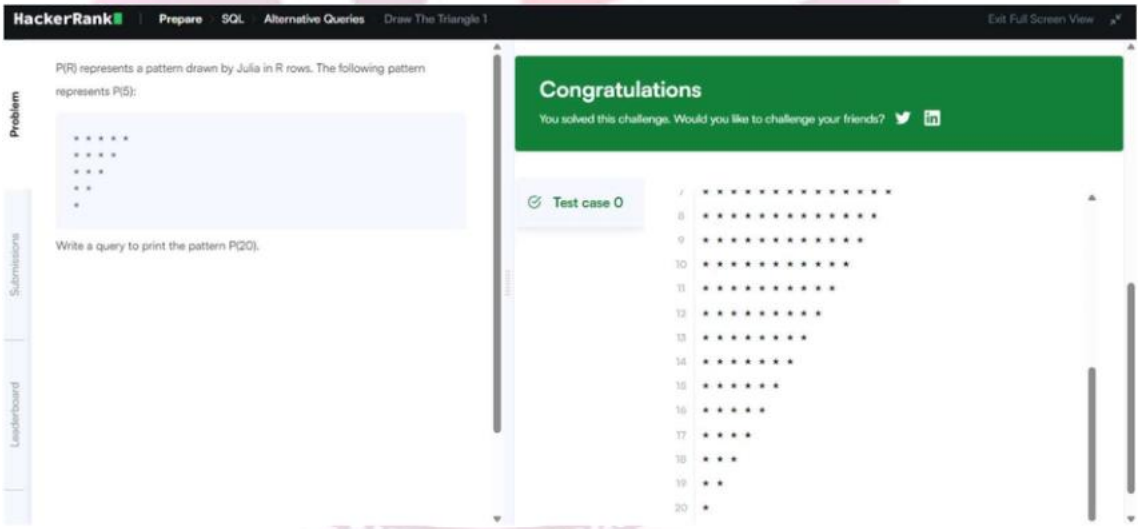


DRAW THE TRIANGLE 1

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
* * * * *
* * * *
* * *
* *
*
```

Write a query to print the pattern P(20).

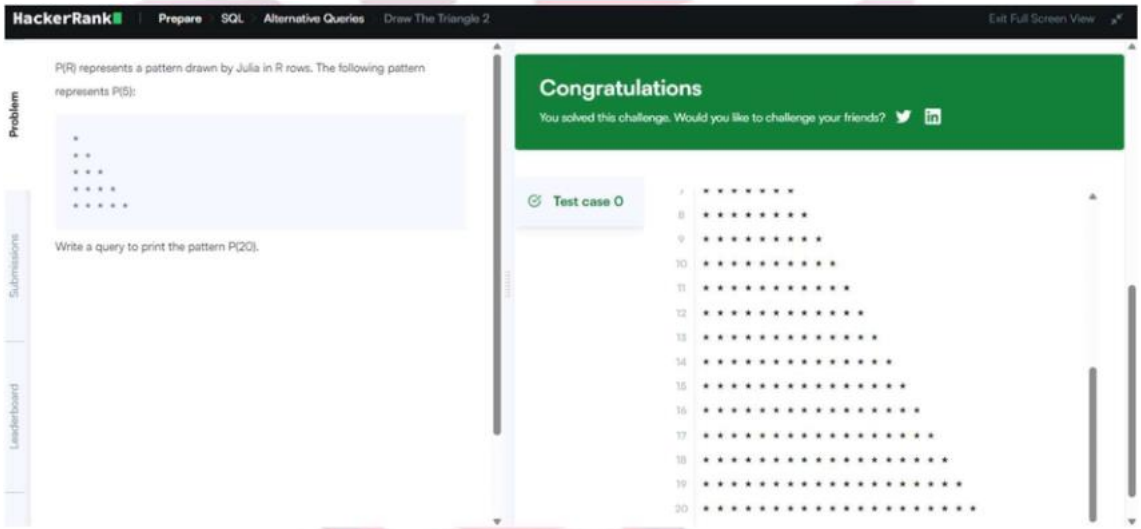


DRAW THE TRIANGLE 2

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
*
* *
* * *
* * * *
* * * * *
```

Write a query to print the pattern P(20).



AUTONOMOUS

THE PADS

Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in OCCUPATIONS, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).

1.Query the number of ocurrences of each occupation in OCCUPATIONS. Sort the occurrences in *ascending order*, and output them in the following format:where [occupation_count] is the number of occurrences of an occupation in OCCUPATIONS and [occupation] is the *lowercase* occupation name. If more than one *Occupation* has the same [occupation_count], they should be ordered alphabetically.

Input Format

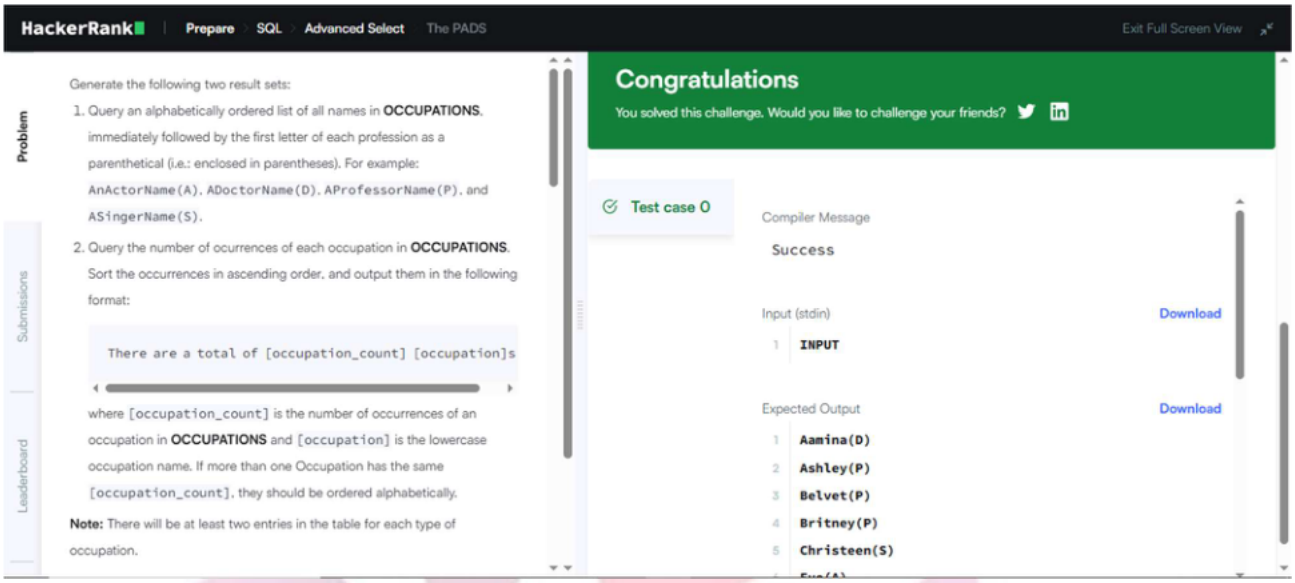
The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY Name;
SELECT CONCAT('There are a total of ', COUNT(Occupation), ' ', LOWER(Occupation), 's.')
FROM OCCUPATIONS
```

GROUP BY Occupation

ORDER BY COUNT(Occupation), Occupation;



OCCUPATIONS

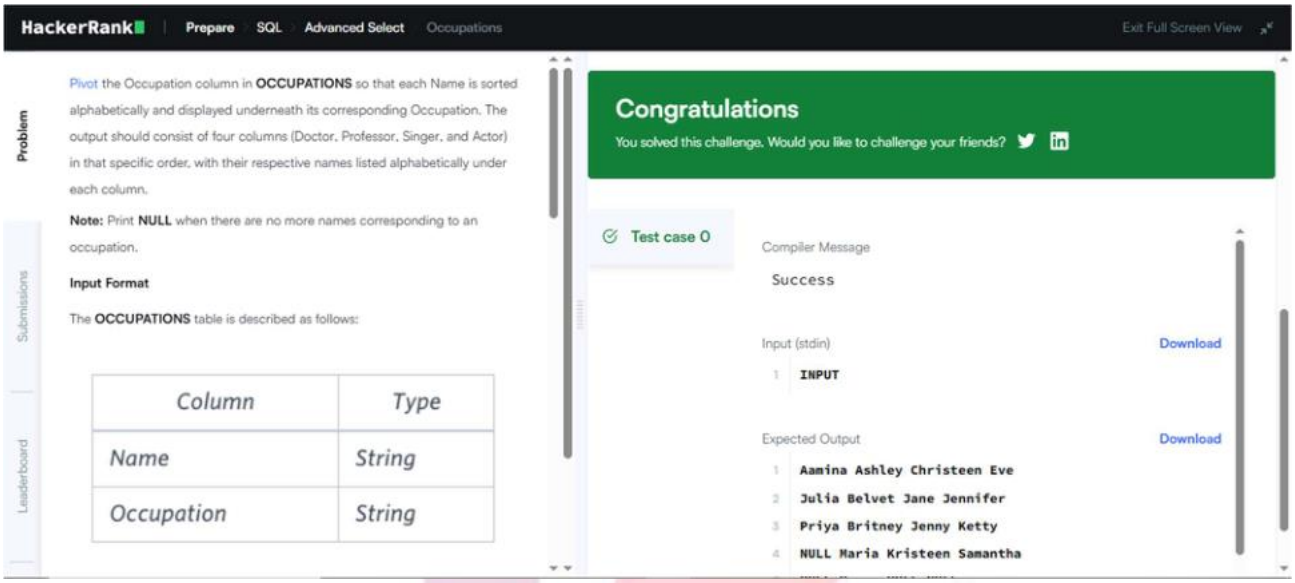
The *Occupation* column in OCCUPATIONS so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Input Format

The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

```
SET @r1=0, @r2=0, @r3=0, @r4=0;
SELECT MIN(Doctor), MIN(Professor), MIN(Singer), MIN(Actor)
FROM (
   SELECT
      CASE
         WHEN Occupation = 'Doctor' THEN (@r1 := @r1 + 1)
         WHEN Occupation = 'Professor' THEN (@r2 := @r2 + 1)
         WHEN Occupation = 'Singer' THEN (@r3 := @r3 + 1)
         WHEN Occupation = 'Actor' THEN (@r4 := @r4 + 1)
      END AS RowNum,
      CASE WHEN Occupation = 'Doctor' THEN Name END AS Doctor,
      CASE WHEN Occupation = 'Professor' THEN Name END AS Professor,
      CASE WHEN Occupation = 'Singer' THEN Name END AS Singer,
      CASE WHEN Occupation = 'Actor' THEN Name END AS Actor
   FROM OCCUPATIONS
   ORDER BY Name
) AS Temp
GROUP BY RowNum;
```



BINARY TREE NODES

You are given a table, *BST*, containing two columns: *N* and *P*, where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*

Column	Type
N	Integer
P	Integer

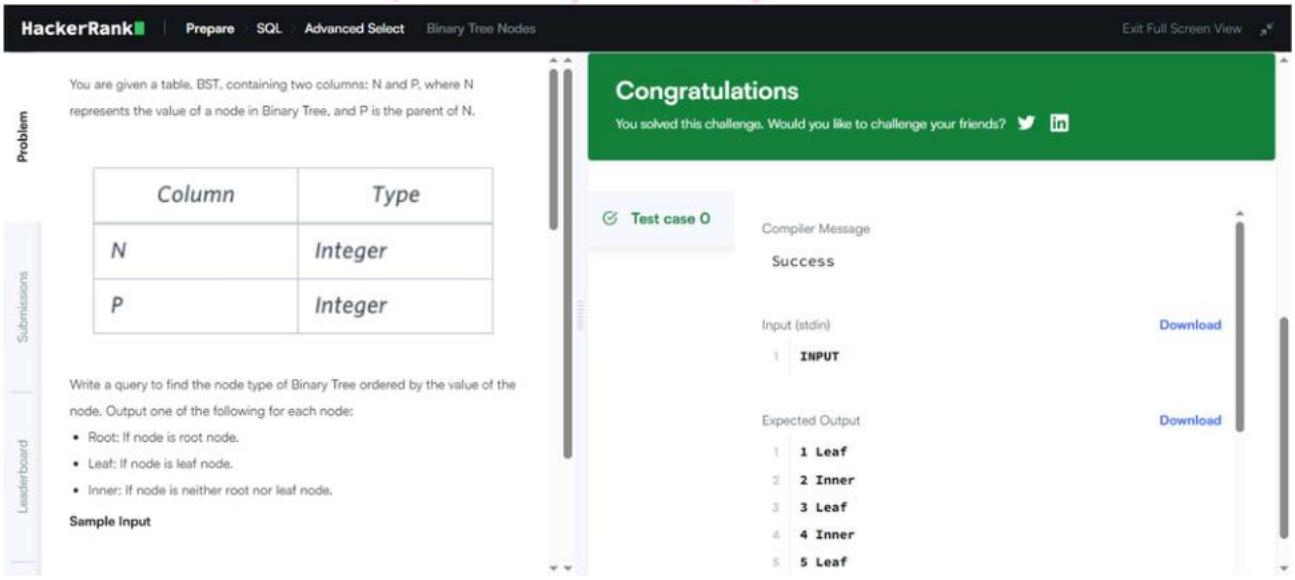
.Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

Sample Input

N	P
1	2
3	2
6	8
9	8
2	5
8	5
5	null

SELECT N, CASE WHEN P IS NULL THEN 'Root' WHEN N IN (SELECT P FROM BST) THEN 'Inner' ELSE 'Leaf' END FROM BST ORDER BY N;



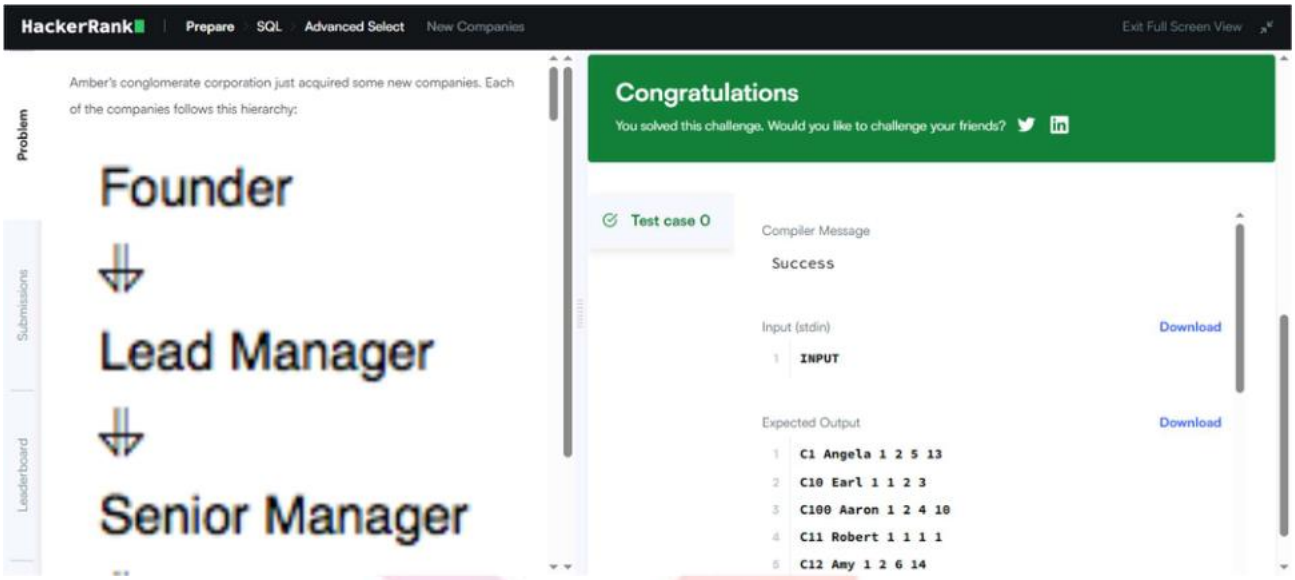
NEW COMPANIES

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:



Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

SELECT c.company_code, c.founder, COUNT(DISTINCT e.lead_manager_code), COUNT(DISTINCT e.senior_manager_code), COUNT(DISTINCT e.manager_code), COUNT(DISTINCT e.employee_code) FROM Company c LEFT JOIN Employee e ON c.company_code = e.company_code GROUP BY c.company_code, c.founder ORDER BY c.company_code;



WEATHER ONSERVATION STATION 18

Consider p1(a,b) and p2(c,d)to be two points on a *2D* plane.

- A happens to equal the minimum value in *Northern Latitude* (*LAT_N* in **STATION**).
- B happens to equal the minimum value in *Western Longitude* (*LONG_W* in **STATION**).
- C happens to equal the maximum value in *Northern Latitude* (*LAT_N* in **STATION**).
- D happens to equal the maximum value in *Western Longitude* (*LONG_W* in **STATION**).

Query the Manhattan Distance between points and and round it to a scale of decimal places.

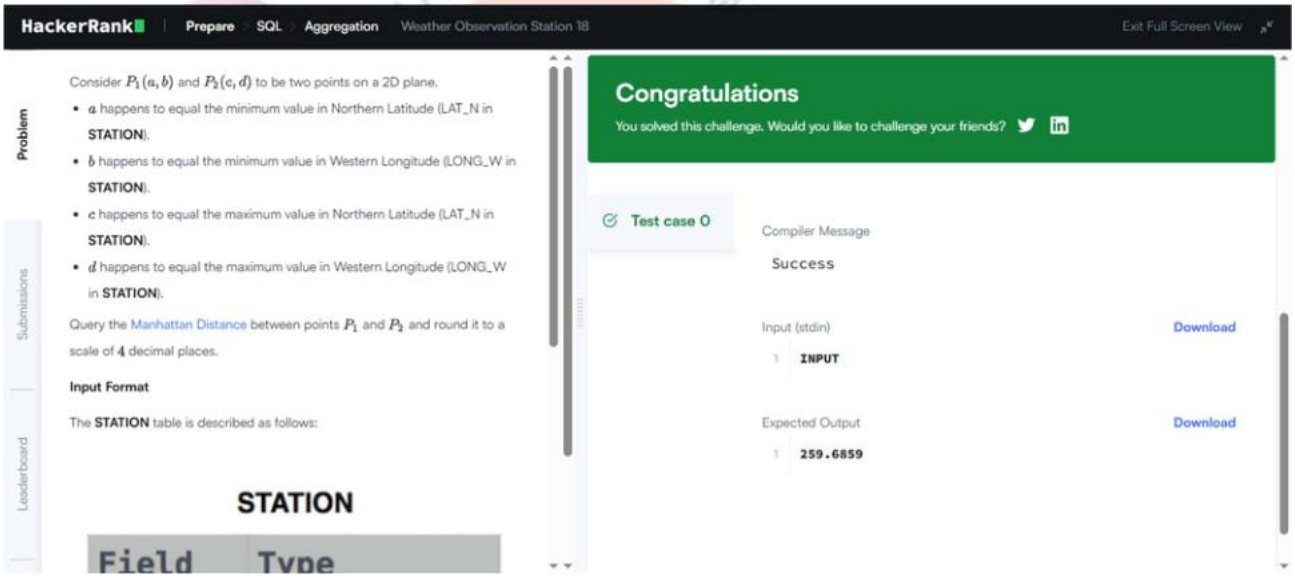
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
SELECT ROUND(ABS(MIN(LAT_N) - MAX(LAT_N)) + ABS(MIN(LONG_W) - MAX(LONG_W)), 4) FROM STATION;
```



WEATHER ONSERVATION STATION 19

Consider ranto be two points on a 2D plane where Are the respective minimum and maximum values of *Northern Latitude* (*LAT_N*) and are the respective minimum and maximum values of *Western Longitude* (*LONG_W*) in STATION.

Query thebetween point and *ormat your answer* to display decimal digits.

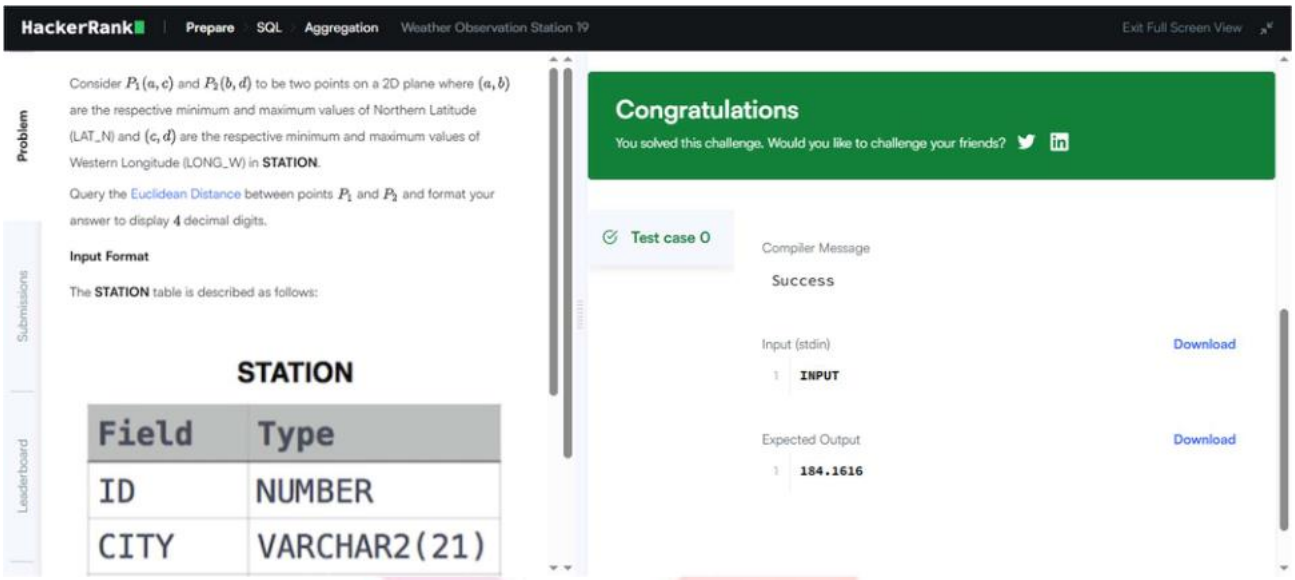
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
SELECT ROUND(SQRT(POW(MIN(LAT_N) - MAX(LAT_N), 2) + POW(MIN(LONG_W) - MAX(LONG_W), 2)), 4) FROM STATION;
```



WEATHER ONSERVATION STATION 20

A median is defined as a number separating the higher half of a data set from the lower half.

Query the *median* of the *Northern Latitudes* from STATION and round your answer to decimal places.

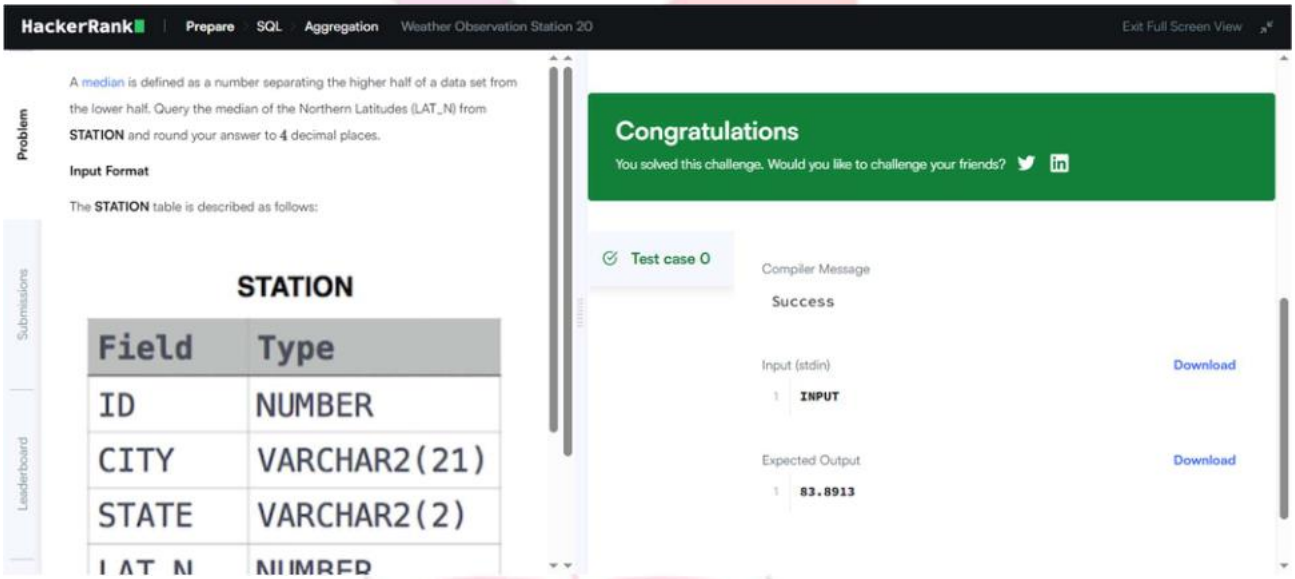
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
SELECT ROUND(AVG(LAT_N), 4)
FROM (
   SELECT LAT_N,
       ROW_NUMBER() OVER (ORDER BY LAT_N) AS RowNum,
       COUNT(*) OVER () AS TotalRows
   FROM STATION
) AS RankedStation
WHERE RowNum IN (FLOOR((TotalRows + 1) / 2), CEIL((TotalRows + 1) / 2));
```



THE REPORT

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

Column	Type
ID	Integer
Name	String
Marks	Integer

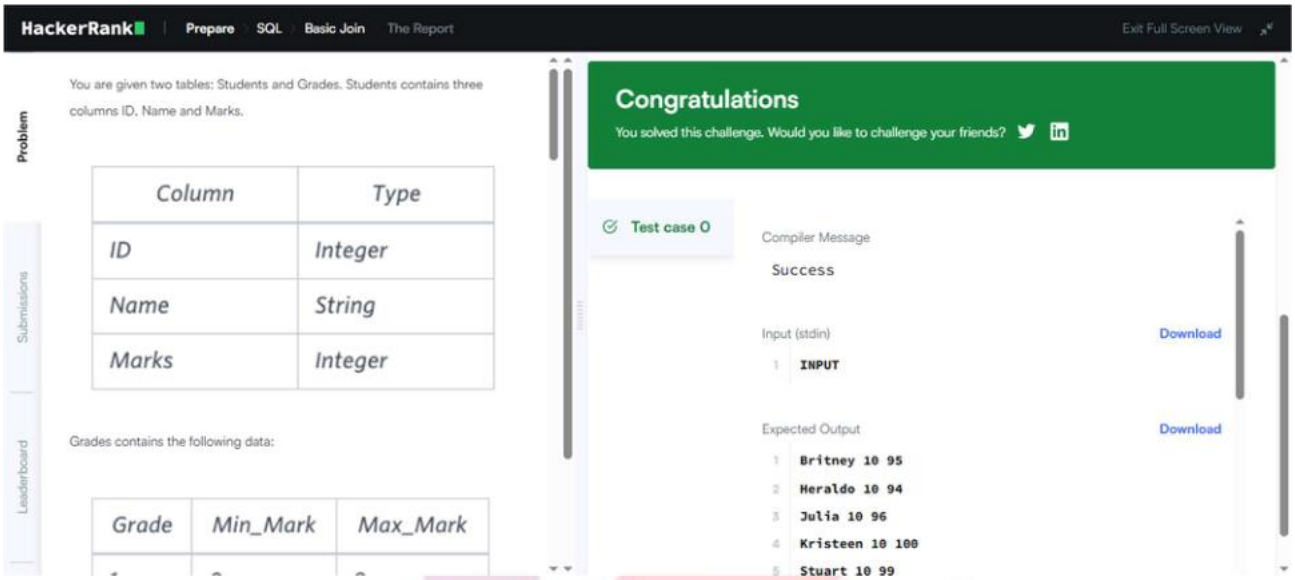
Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives *Eve* a task to generate a report containing three columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a grade lower than *8*. The report must be in descenading order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their

name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

```
SELECT
    CASE WHEN g.Grade >= 8 THEN s.Name ELSE NULL END AS Name,
    g.Grade,
    s.Marks
FROM Students s
JOIN Grades g
  ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
    g.Grade DESC,
    CASE WHEN g.Grade >= 8 THEN s.Name END ASC,
    CASE WHEN g.Grade < 8 THEN s.Marks END ASC;
```



TOP COMPETITORS

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*.

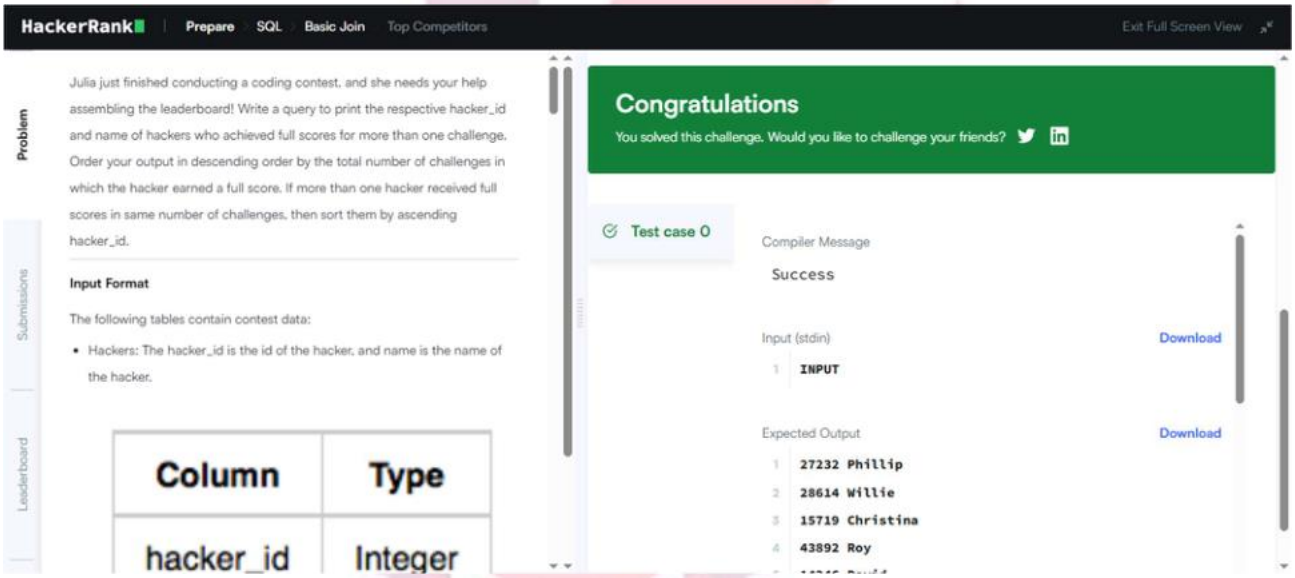
Input Format

The following tables contain contest data:

Hackers: The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
SELECT h.hacker_id, h.name
FROM Submissions s
JOIN Challenges c ON s.challenge_id = c.challenge_id
JOIN Difficulty d ON c.difficulty_level = d.difficulty_level
JOIN Hackers h ON s.hacker_id = h.hacker_id
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```



OLLIVANDER'S INVENTORY

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of

the wands that Ron's interested in, sorted in order of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

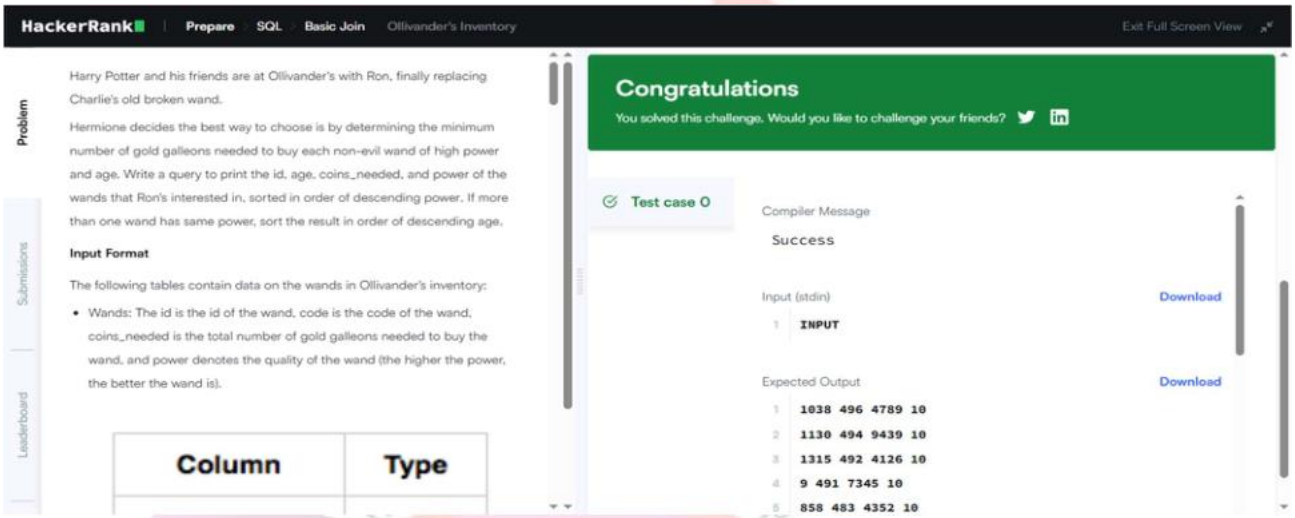
The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

```
SELECT w.id, p.age, w.coins_needed, w.power
FROM Wands AS w
JOIN Wands_Property AS p
  ON w.code = p.code
WHERE p.is_evil = 0
  AND w.coins_needed = (
      SELECT MIN(x.coins_needed)
      FROM Wands AS x
      JOIN Wands_Property AS y
```

```
    ON x.code = y.code
  WHERE x.power = w.power
    AND y.age = p.age
)
ORDER BY w.power DESC, p.age DESC;
```



CHALLENGES

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the

total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

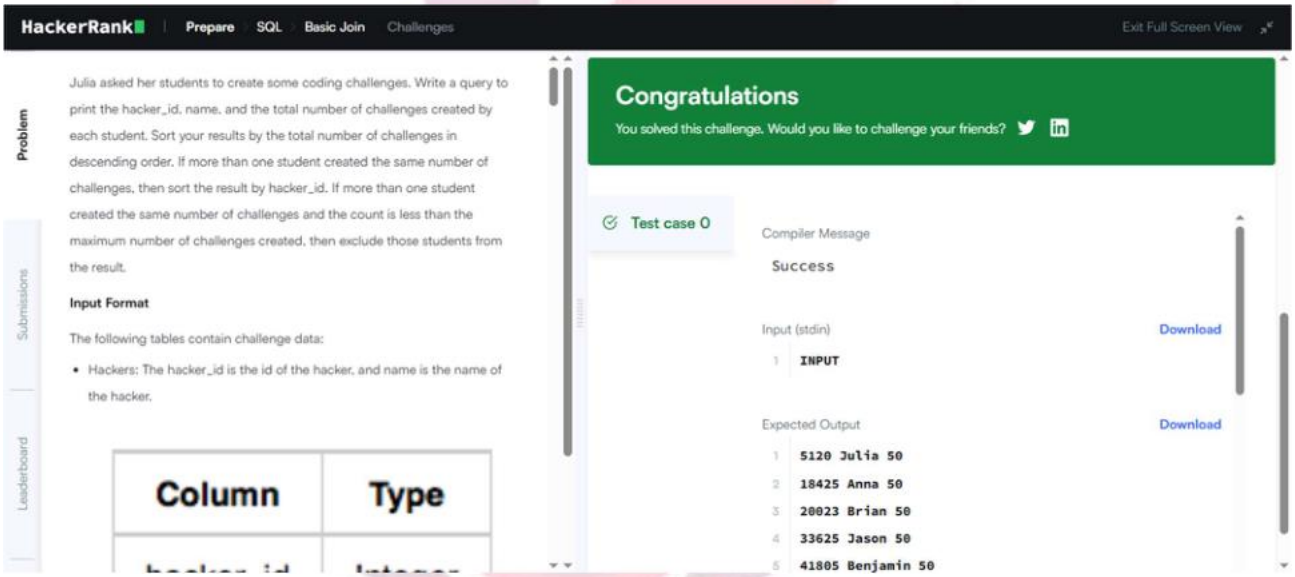
- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
WITH ChallengeCounts AS (
    SELECT
        h.hacker_id,
        h.name,
        COUNT(c.challenge_id) AS total_challenges
    FROM Hackers h
    JOIN Challenges c ON h.hacker_id = c.hacker_id
    GROUP BY h.hacker_id, h.name
)

SELECT hacker_id, name, total_challenges
FROM ChallengeCounts
WHERE total_challenges = (
    /* Condition 1: Keep it if it matches the maximum challenges created by any hacker */
    SELECT MAX(total_challenges) FROM ChallengeCounts
)
```

```
OR total_challenges IN (
    /* Condition 2: Keep it if the challenge count is completely unique */
    SELECT total_challenges
    FROM ChallengeCounts
    GROUP BY total_challenges
    HAVING COUNT(total_challenges) = 1
)
ORDER BY total_challenges DESC, hacker_id ASC;
```



CONTEST LEADERBOARD

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id, name*, and total score of the hackers ordered by the descending score. If more than one hacker

achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of from your result.

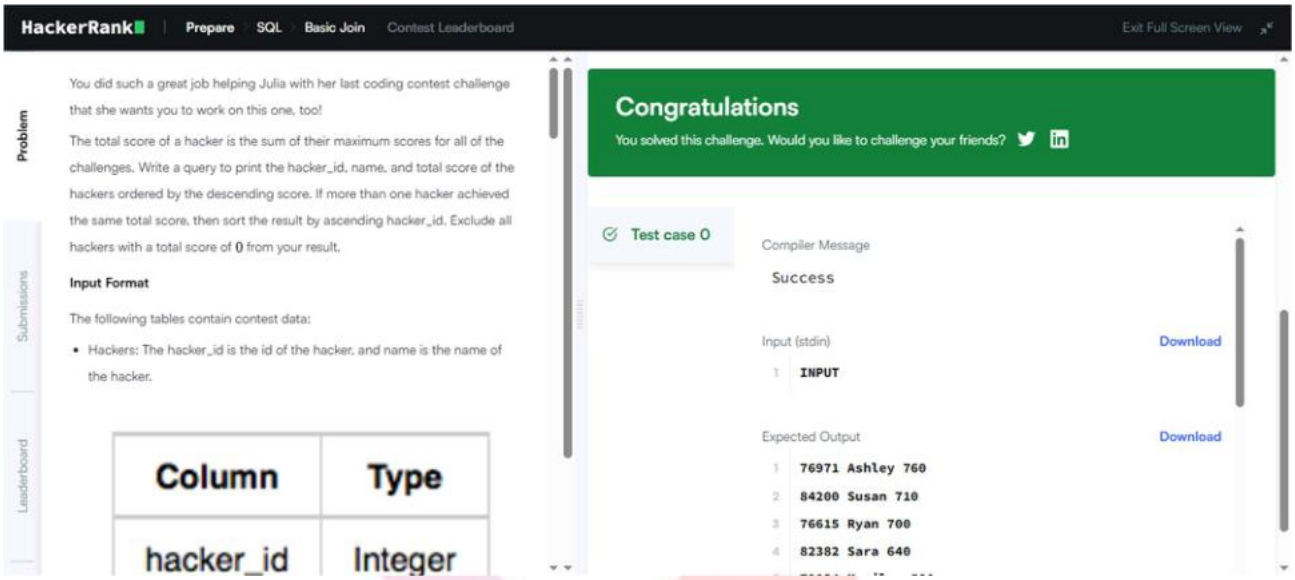
Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
SELECT h.hacker_id, h.name, SUM(t.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT hacker_id, challenge_id, MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) t ON h.hacker_id = t.hacker_id
GROUP BY h.hacker_id, h.name
HAVING total_score > 0
ORDER BY total_score DESC, h.hacker_id ASC;
```



SQL PROJECT PLANNING

You are given a table, *Projects*, containing three columns: *Task_ID, Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	*Type*
Task_ID	*Integer*
Start_Date	*Date*
End_Date	*Date*

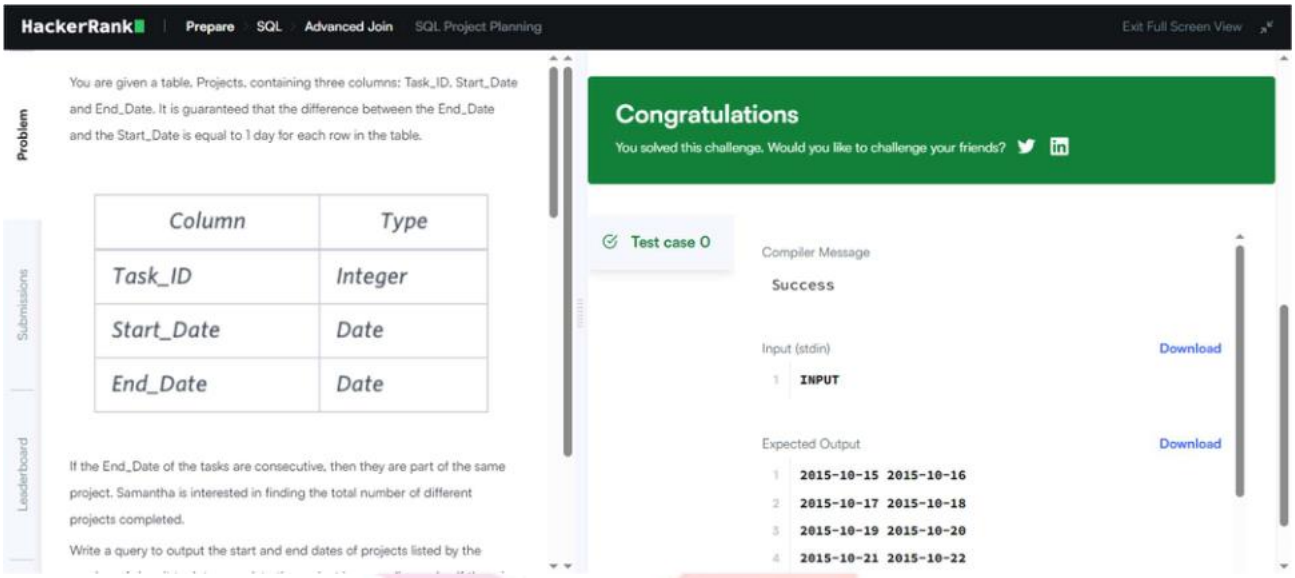
If the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

```
SELECT A.Start_Date, MIN(B.End_Date)
FROM
   (SELECT Start_Date
   FROM Projects
   WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)) AS A
   INNER JOIN
   (SELECT End_Date
   FROM Projects
   WHERE End_Date NOT IN (SELECT Start_Date FROM Projects)) AS B
WHERE A.Start_Date < B.End_Date
GROUP BY A.Start_Date
ORDER BY MIN(B.End_Date) - A.Start_Date ASC, A.Start_Date;
```



PLACEMENTS

You are given three tables: *Students*, *Friends* and *Packages*.*Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month)

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

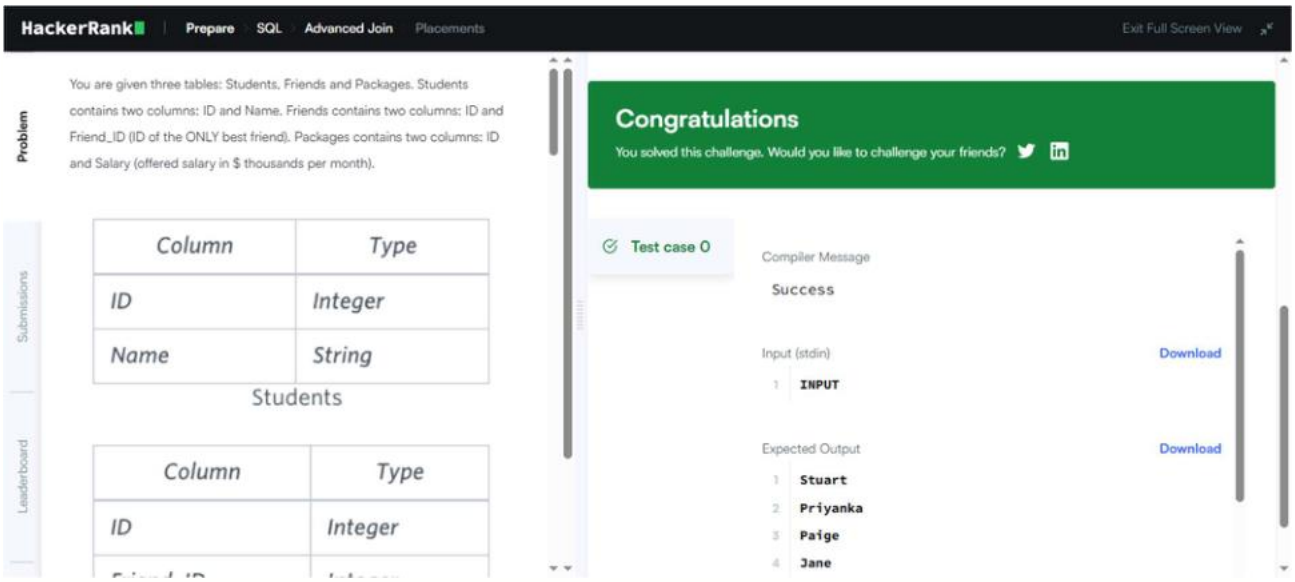
Friends

Column	Type
ID	Integer
Salary	Float

Packages

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students got same salary offer.

```
SELECT s.Name
FROM Students s
JOIN Friends f ON s.ID = f.ID
JOIN Packages p1 ON s.ID = p1.ID
JOIN Packages p2 ON f.Friend_ID = p2.ID
WHERE p2.Salary > p1.Salary
ORDER BY p2.Salary;
```



SYMMETRIC PAIRS

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	Type
X	Integer
Y	Integer

Two pairs *(X1, Y1)* and *(X2, Y2)* are said to be *symmetricpairs* if $X1 = Y2$ and $X2 = Y1$.

Write a query to output all such *symmetricpairs* in ascending order by the value of *X*. List the rows such that $X1 \leq Y1$.

```
SELECT f1.X, f1.Y

FROM Functions f1

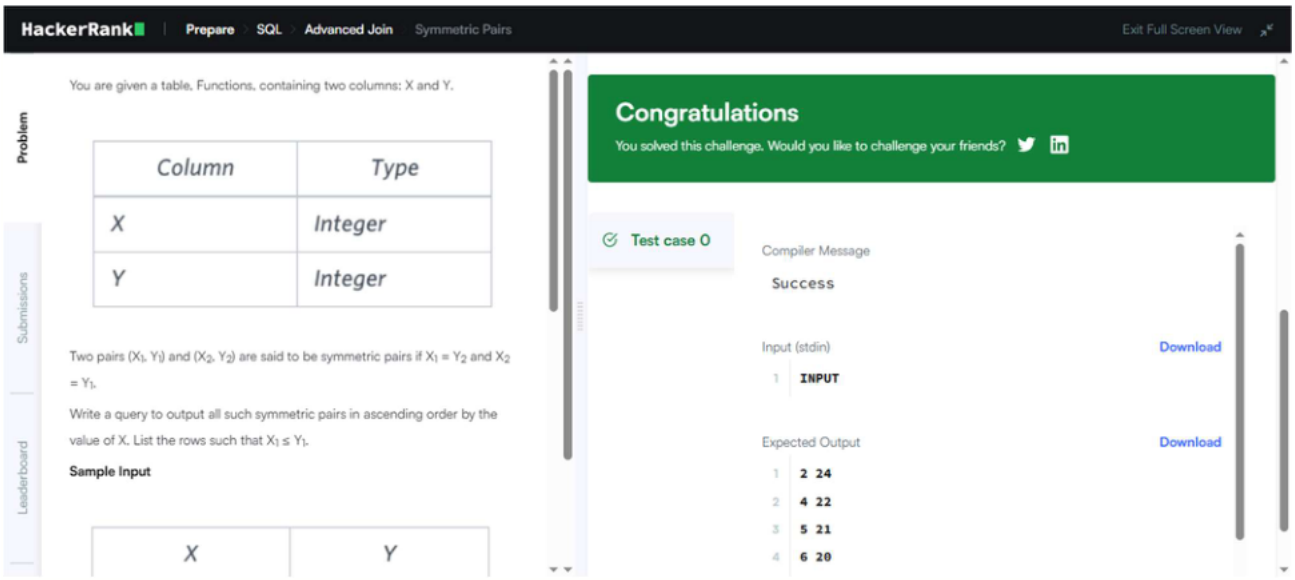
JOIN Functions f2

   ON f1.X = f2.Y AND f1.Y = f2.X

GROUP BY f1.X, f1.Y

HAVING f1.X < f1.Y OR COUNT(f1.X) > 1

ORDER BY f1.X;
```



PRINT PRIME NUMBERS

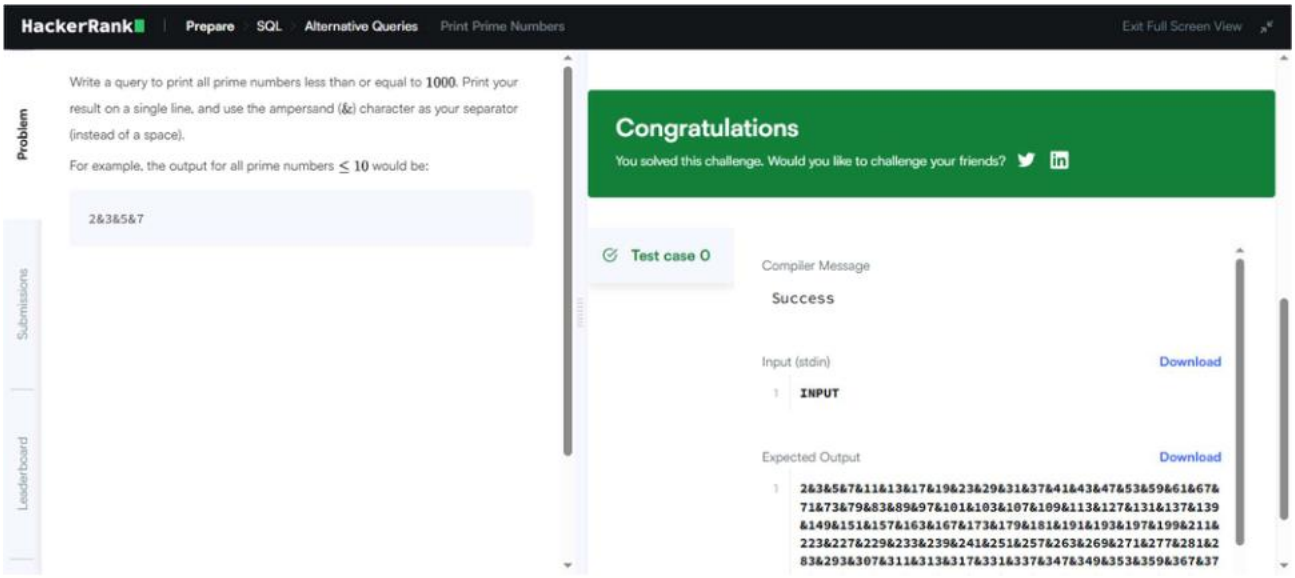
Write a query to print all *prime numbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

For example, the output for all prime numbers would be:

2&3&5&7

```
WITH RECURSIVE Numbers AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n + 1 FROM Numbers WHERE n < 1000
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
```

```
FROM Numbers n1
WHERE NOT EXISTS (
   SELECT 1
   FROM Numbers n2
   WHERE n2.n < n1.n AND n1.n % n2.n = 0
);
```



AUTONOMOUS

INTERVIEWS

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created the contest, and *name* is the name of the hacker.

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

- *Colleges:* The *college_id* is the id of the college, and *contest_id* is the id of the contest that Samantha used to screen the candidates.

Column	Type
college_id	Integer
contest_id	Integer

- *Challenges:* The *challenge_id* is the id of the challenge that belongs to one of the contests whose contest_id

  Samantha forgot, and *college_id* is the id of the college where the challenge was given to candidates.

Column	Type
challenge_id	Integer
college_id	Integer

- *View_Stats:* The *challenge_id* is the id of the challenge, *total_views* is the number of times the challenge

was viewed by candidates, and *total_unique_views* is the number of times the challenge was viewed by

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

- *Submission_Stats:* The *challenge_id* is the id of the challenge, *total_submissions* is the number of submissions for the challenge, and *total_accepted_submission* is the number of submissions that achieved

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Contests Table:

contest_id	hacker_id	name
66406	17973	Rose
66556	79153	Angela
94828	80275	Frank

Colleges Table:

college_id	contest_id
11219	66406
32473	66556
56685	94828

Challenges Table:

challenge_id	college_id
18765	11219
47127	11219
60292	32473
72974	56685

View_Stats Table:

challenge_id	total_views	total_unique_views
47127	26	19
47127	15	14
18765	43	10
18765	72	13
75516	35	17
60292	11	10
72974	41	15
75516	75	11

Submission_Stats Table:

challenge_id	total_submissions	total_accepted_submissions
75516	34	12
47127	27	10
47127	56	18
75516	74	12
75516	83	8
72974	68	24
72974	82	14
47127	28	11

SELECT

c.contest_id,

c.hacker_id,

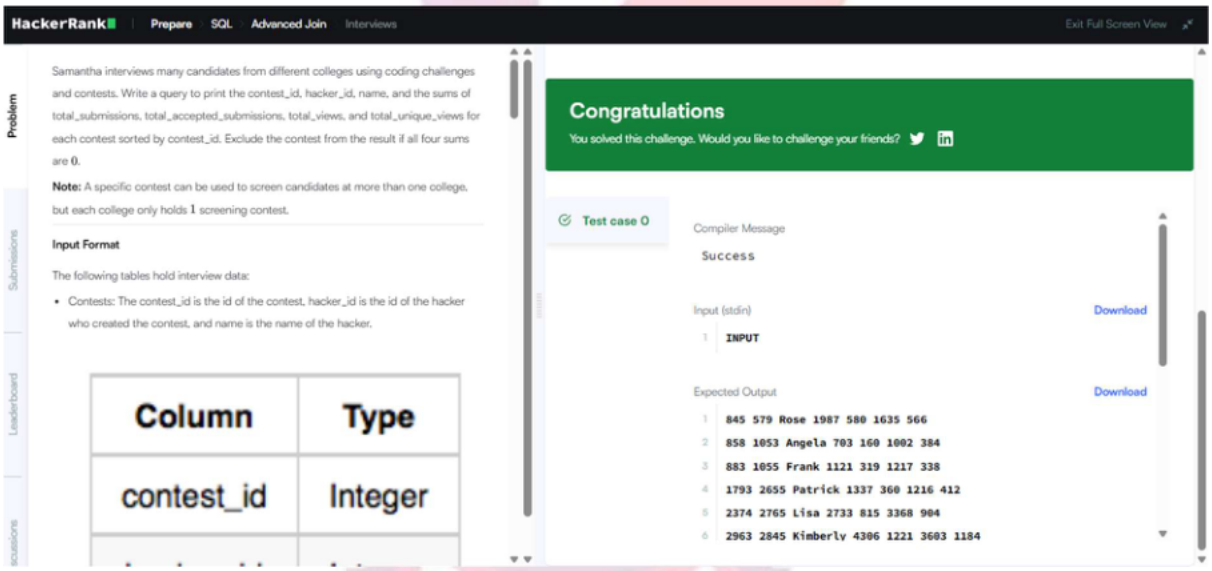
```
    c.name,
    SUM(COALESCE(s.total_submissions, 0)) AS total_submissions,
    SUM(COALESCE(s.total_accepted_submissions, 0)) AS
   total_accepted_submissions,

    SUM(COALESCE(v.total_views, 0)) AS total_views,
    SUM(COALESCE(v.total_unique_views, 0)) AS
total_unique_views
FROM Contests c
JOIN Colleges col
    ON c.contest_id = col.contest_id
JOIN Challenges ch
    ON col.college_id = ch.college_id
LEFT JOIN (
    SELECT

        challenge_id,
        SUM(total_submissions) AS total_submissions,
        SUM(total_accepted_submissions) AS total_accepted_submissions
    FROM Submission_Stats
    GROUP BY challenge_id
) s

ON ch.challenge_id = s.challenge_id
LEFT JOIN (
    SELECT
        challenge_id,
        SUM(total_views) AS total_views,
        SUM(total_unique_views) AS total_unique_views
    FROM View_Stats
    GROUP BY challenge_id
) v
ON ch.challenge_id = v.challenge_id
GROUP BY
    c.contest_id,
```

```
    c.hacker_id,
    c.name
HAVING
    SUM(COALESCE(s.total_submissions, 0)) +
    SUM(COALESCE(s.total_accepted_submissions,
0)) +
    SUM(COALESCE(v.total_views, 0)) +
    SUM(COALESCE(v.total_unique_views, 0)) >0
ORDER BY
    c.contest_id;
```



15 DAYS OF LEARNING SQL

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end
date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was *March 06, 2016*.

Hackers Table:

hacker_id	name
15758	Rose
20703	Angela
36396	Frank
38289	Patrick
44065	Lisa
53473	Kimberly
62529	Bonnie
79722	Michael

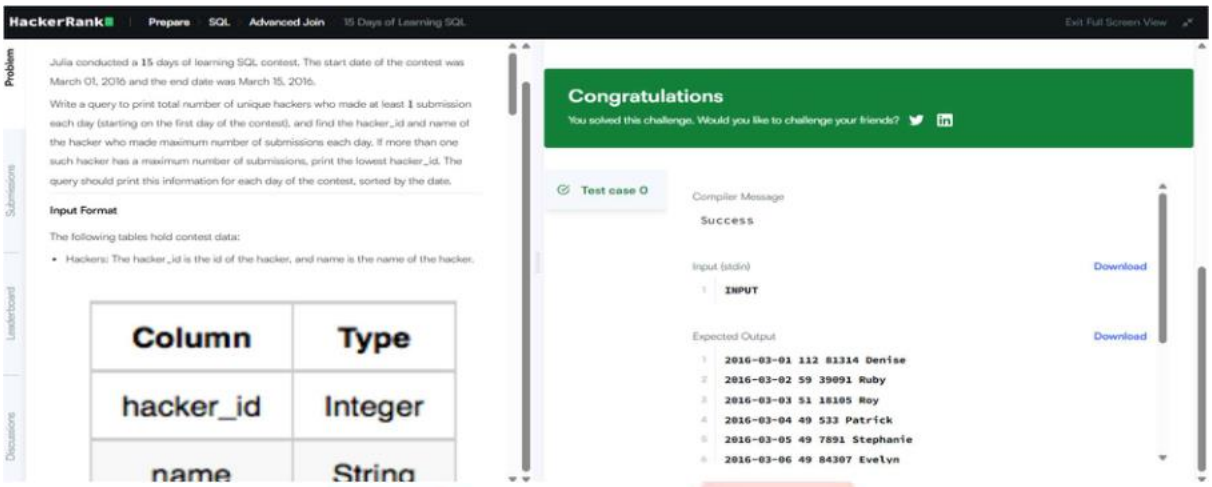
Submissions **Table:**



submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

```
        SELECT
  s.submission_date,
  (
    SELECT COUNT(DISTINCT
 hacker_id)
    FROM Submissions s2
    WHERE s2.submission_date <= s.submission_date
    AND (
      SELECT COUNT(DISTINCT
   s3.submission_date)
      FROM Submissions s3
      WHERE s3.hacker_id = s2.hacker_id
      AND s3.submission_date <= s.submission_date
    ) = DATEDIFF(s.submission_date, '2016-03-01')
                          + 1
  ) AS total_hackers,
  s.hacker_id,
  h.name
FROM Submissions s
JOIN Hackers h
ON s.hacker_id = h.hacker_id
WHERE s.hacker_id = (
  SELECT s1.hacker_id
  FROM Submissions s1
  WHERE s1.submission_date = s.submission_date
  GROUP BY s1.hacker_id
  ORDER BY COUNT(*) DESC, s1.hacker_id
  LIMIT 1
)
GROUP BY s.submission_date, s.hacker_id, h.name
```

ORDER BY s.submission_date;



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

CODECHEF

BASIC COMMANDS

1. ALL PRODUCTS

Task

Write a query to output Products table.

Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT

select*from products;

OUTPUT

Your Output

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture

✓ Excellent work!

2. HIGH PRICE OF PRODUCTS

Task

Write a query to find all product_name and category that have a price greater than 100.00 from the **Products** table.

Table name: **Products**

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT

```
SELECT product_name, category FROM Products WHERE price > 100.00;
```

OUTPUT

Your Output

```
┌──────────────┬─────────────┐
│ product_name │  category   │
├──────────────┼─────────────┤
│ Laptop       │ Electronics │
│ Desk Chair   │ Furniture   │
│ Smartphone   │ Electronics │
│ Standing Desk │ Furniture  │
│ Tablet       │ Electronics │
└──────────────┴─────────────┘
```

✔ Excellent work!

3. AVERAGE SALARY

Task

- Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

INPUT

```
SELECT AVG(salary) AS avg_salary FROM Works;
```

OUTPUT

Your Output

avg_salary
62600.155

Hooray, you did it!

4. LOCATE PEOPLE

Task

- Write a query to retrieve the department_name and location of departments located in a city that starts with 'S'.

Table name: **departments**

department_id	department_name	location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Boston

INPUT

```
SELECT department_name, location FROM departments WHERE location LIKE 'S%';
```

OUTPUT

Your Output

department_name	location
Research and Development	San Francisco
IT Support	Seattle
Product Management	San Francisco

✔ Awesome, you nailed it!

5. DISTINCT COMPANIES

Task

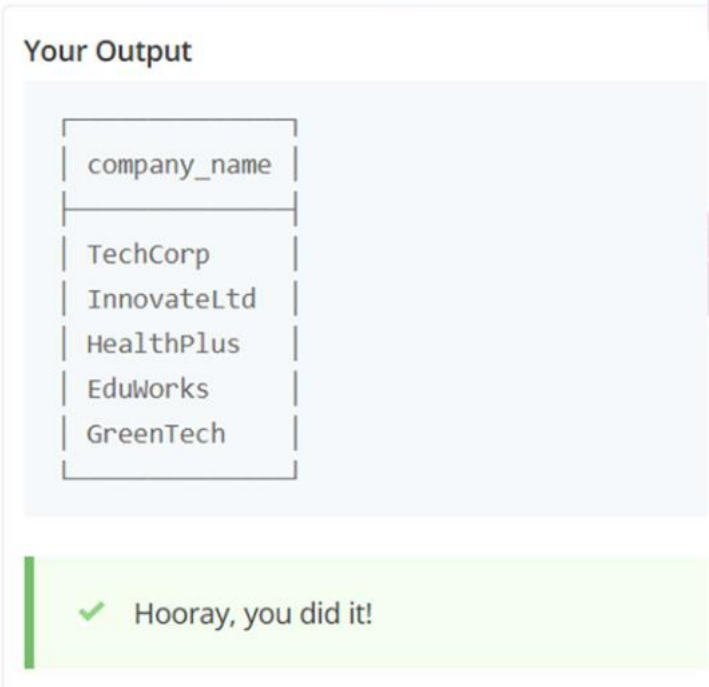
Write a query to select all the distinct companies (company_name) in the Works table.
Table name: **Works**

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

INPUT

```
SELECT DISTINCT company_name FROM Works;
```

OUTPUT



6. FICTION COLLECTION SIZE

Task

Write a query to find the total count of books whose genre is **Fiction**.
Note: Output column name should be **fiction_count**.

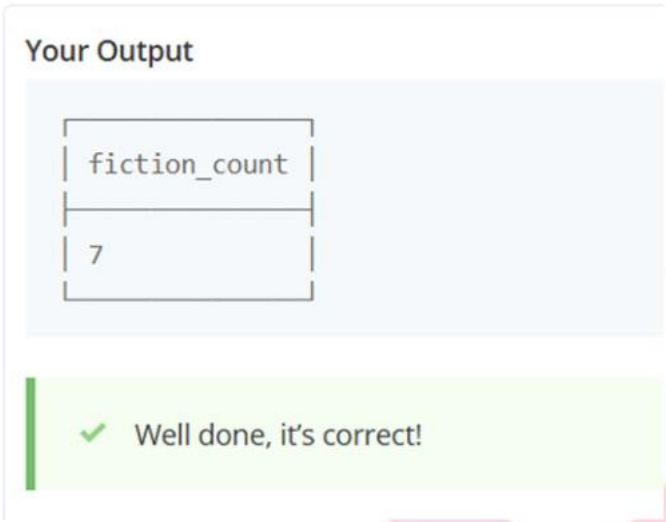
Table name: **Books**

id	title	author	genre	price	published_year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985
14	A Tale of Two	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

INPUT

```
SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction';
```

OUTPUT



7. LIST OF MOVIES WITH RATINGS

Task

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table: **Cinema**

Movie_id	Movie_name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

INPUT

```
SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.0;
```

OUTPUT



8. HANDLING NULL VALUES

Task

Write a query to retrieve book_id, title, author and published_year of the books which have **NULL** rating for their books.

Table name: Library

book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

INPUT

select book_id, title, author, published_year from library where rating is null;

OUTPUT

Your Output

book_id	title	author	published_year
2	To Kill a Mockingbird	Harper Lee	1960
4	The Catcher in the Rye	J.D. Salinger	1951

✔ Excellent work!

9. SALARY OF EMPLOYEES

Task

Create a query to retrieve the employee_name, company, and salary for employees in the full-time category, ordered by salary in **descending** order

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

INPUT

```
SELECT employee_name, company, salary FROM Employees WHERE category = 'Full-Time' ORDER BY salary DESC;
```

OUTPUT

Your Output

employee_name	company	salary
Bob Brown	FinServ	90000
David Green	TechCorp	85000
John Smith	TechCorp	80000
Grace Grey	TechCorp	70000
Frank Black	HealthPlus	60000

✔ You got it right!

10. DEPARTMENT OF EACH EMPLOYEE

Task

Write a query to group the employees by their department and display the total number of employees (as total_employees) in each department.

Table name: Employees

employee_id	employee_nam	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

INPUT

```
select department, count(*) as total_employees from employees group by department;
```

OUTPUT

Your Output

department	total_employees
Engineering	2
Finance	2
HR	2
IT	1
Marketing	2
Sales	1

✔ Perfect answer!

11. ARTICLE VIEWS

Task

Write a query to retrieve the author_id, author_name, and publication_name for authors whose articles got zero views. The result should be sorted by author_id in ascending order.

Return the result table sorted by id in **ascending order**.

Table name: Views

article_id	author_id	author_name	viewer_id	view_count	publication_nam
101	1	John Doe	1	3	The Daily
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business
106	1	John Doe	2	0	The Daily
107	2	Jane Smith	2	2	Global News

INPUT

select distinct author_id, author_name, publication_name from views where view_count = 0 order by author_id asc;

OUTPUT

Your Output

author_id	author_name	publication_name
1	John Doe	The Daily Times
3	Emily Clark	Tech Monthly
5	Sara White	The Business Journal

✔ You got it right!

12. PLAYER PERFORMANCE INSIGHTS

Task

Write a query to find the names of the **top 3 distinct players** by highest score who have **won** matches, including their **scores.**

Table 1: **Players**

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: **Matches**

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

INPUT

select distinct player_name, score from players join matches where winner=player_name order by score desc limit 3

OUTPUT

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

✔ Awesome, you nailed it!

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

13 . PLAYER DETAILS

Task

Write a query to retrieve the details of the **last five matches** played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named **Players** and **Matches**.

Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	jack	1400	10

Matches

match_id	player_1	player_2	winner	match_date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

INPUT

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score
FROM Matches m
JOIN Players p ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

OUTPUT

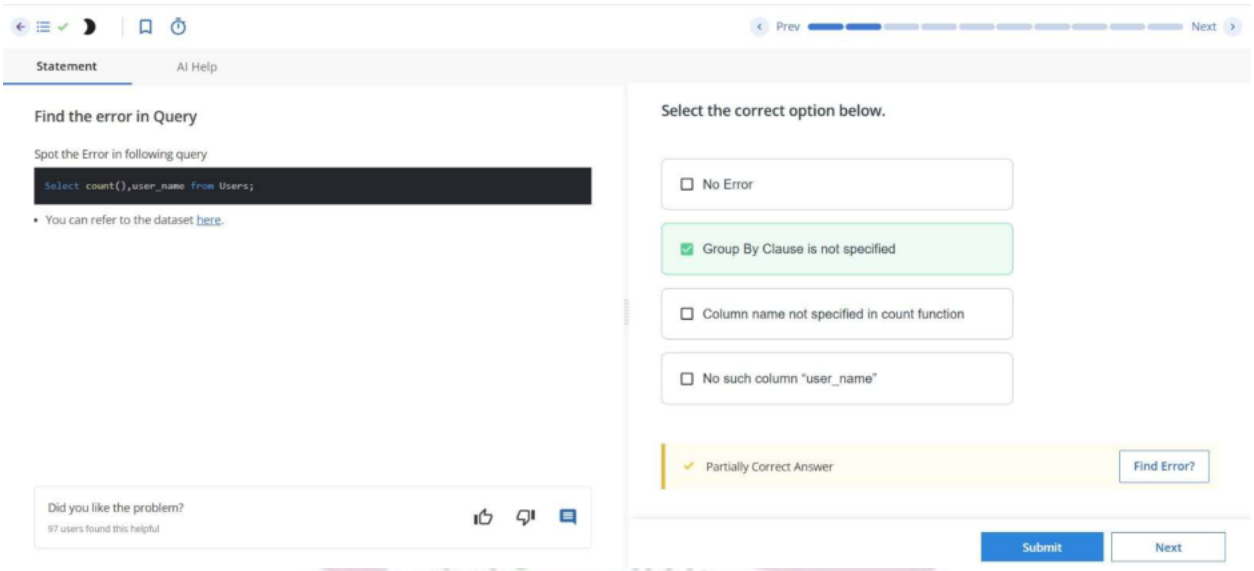
Your Output

match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

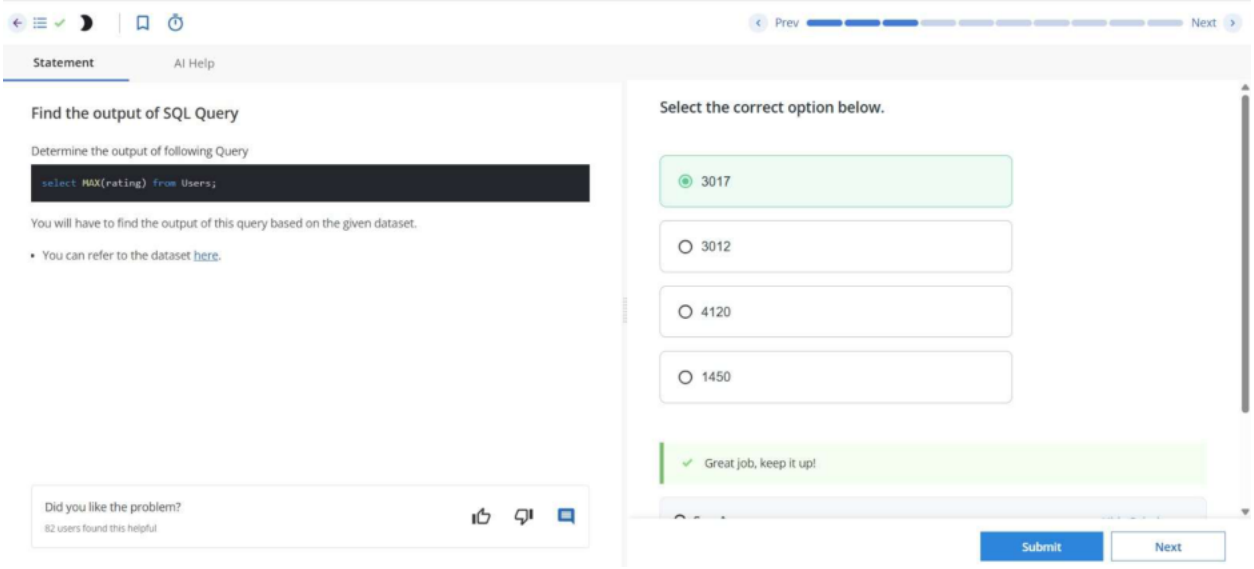
✔ Awesome, you nailed it!

CASE STUDY – CODECHEF DATABASE

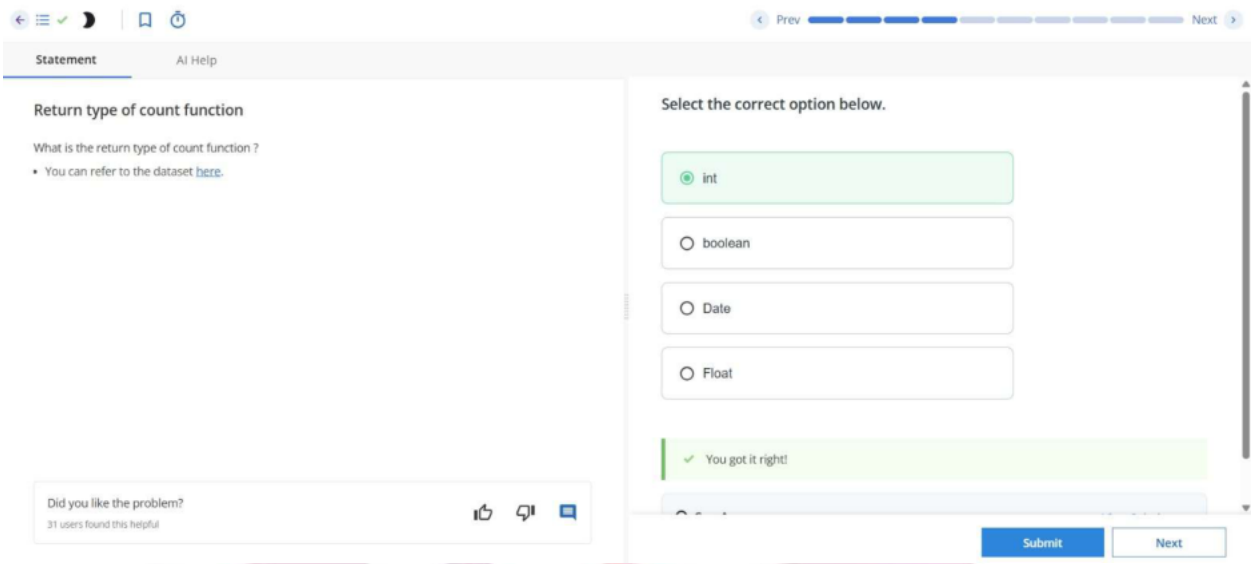
1.FIND THE ERROR IN QUERY



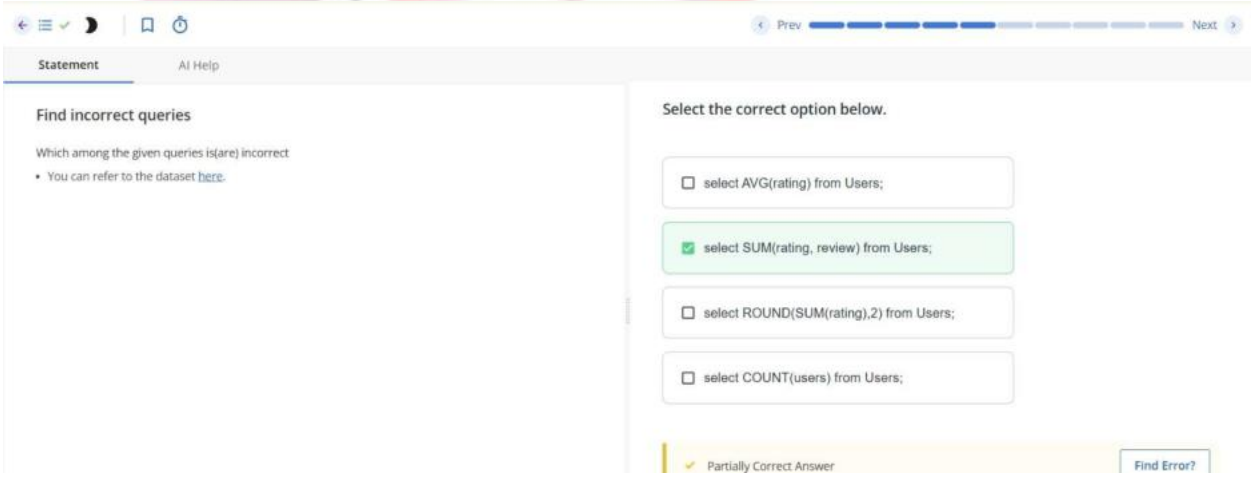
2. FIND THE OUTPUT OF SQL QUERY



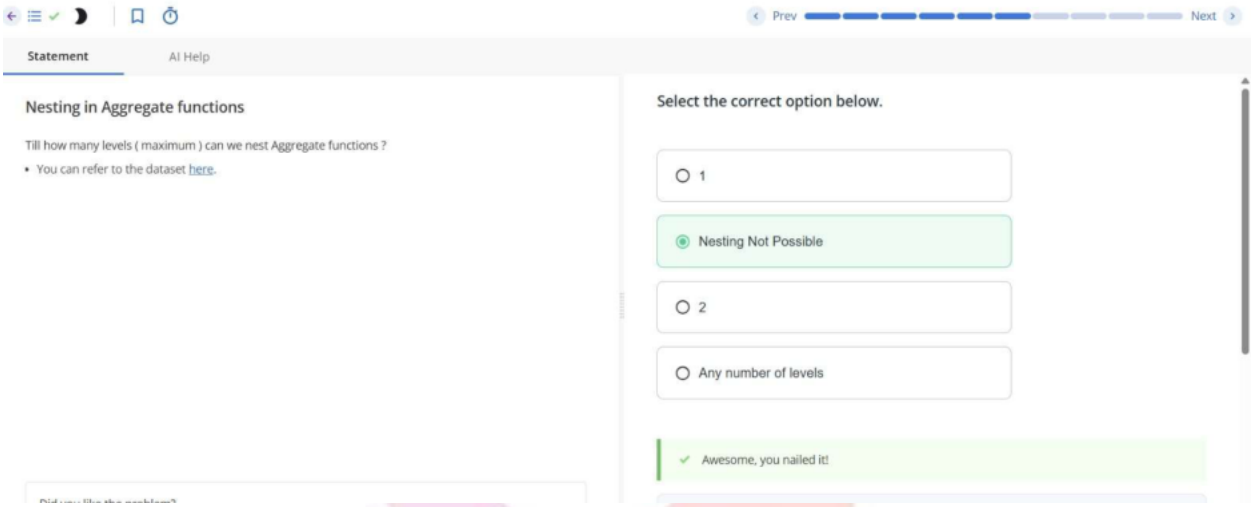
3. RETURN TYPE OF COUNT FUNCTION



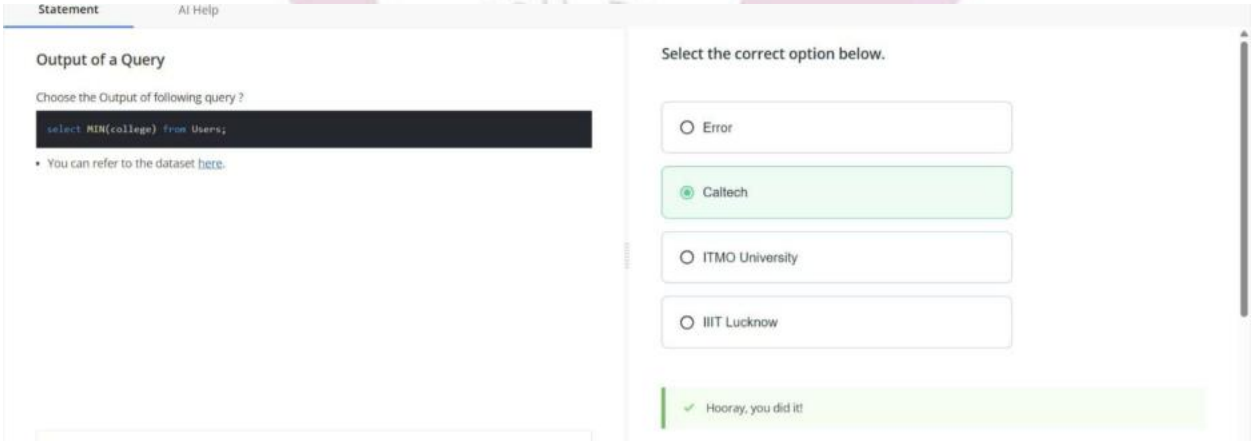
4. FIND INCORRECT QUERIES



5.NESTED IN AGGREGATE FUNCTIONS

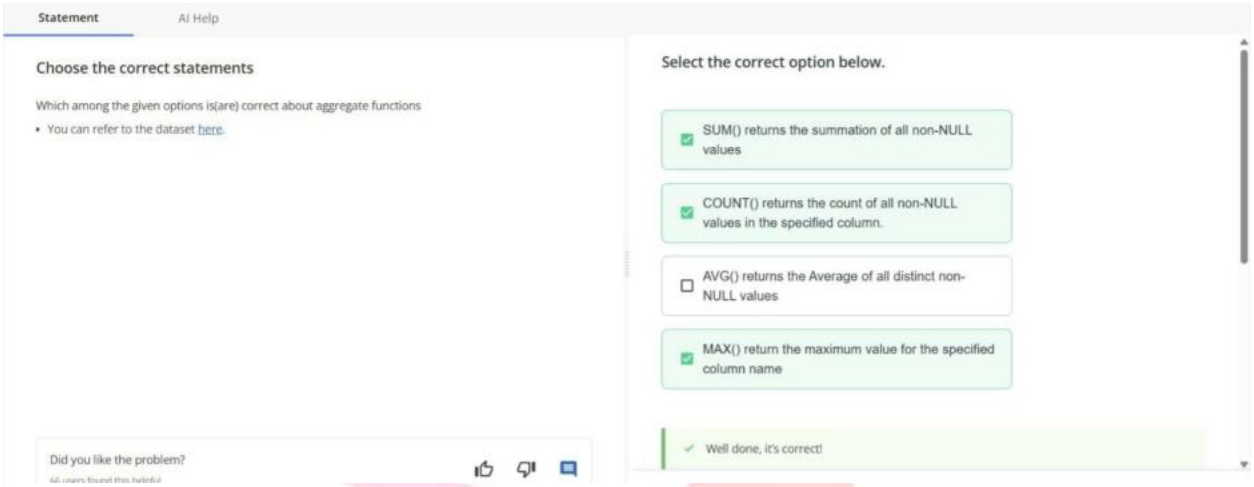


6. OUTPUT OF A QUERY

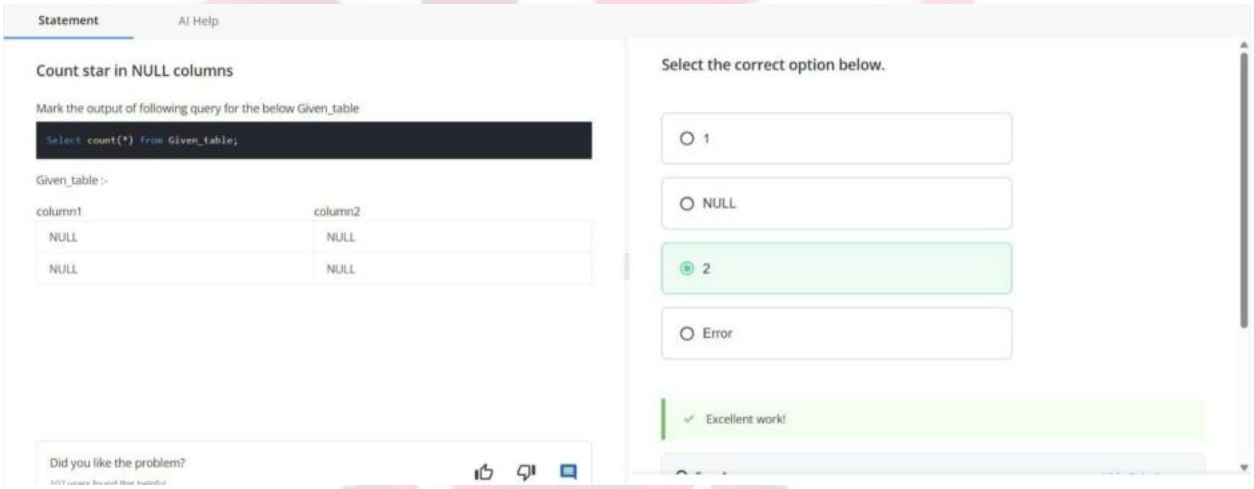


AUTONOMOUS

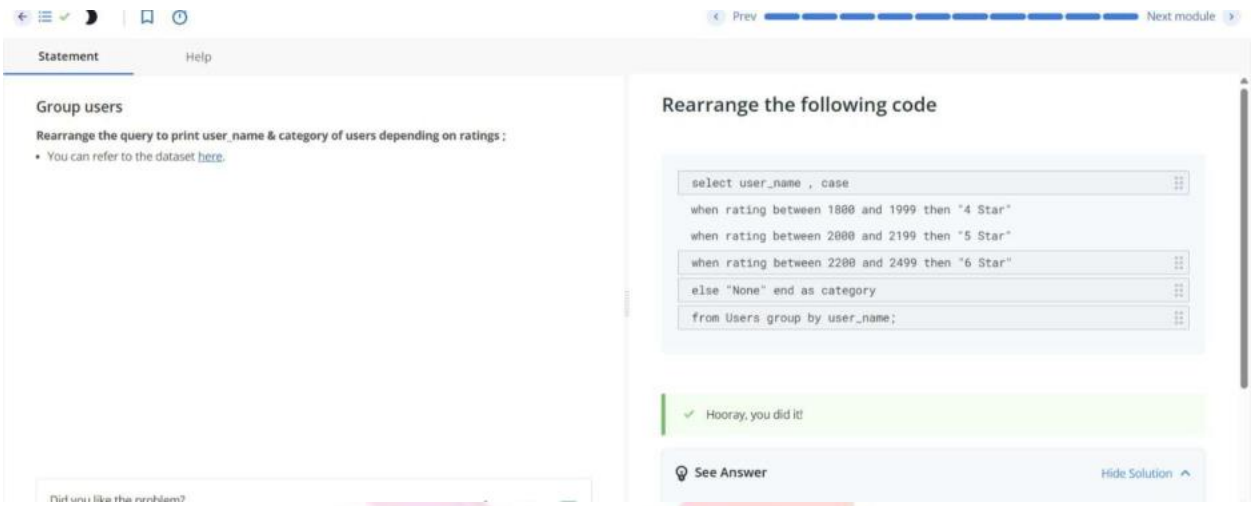
7. CHOOSE THE CORRECT STATEMENTS



8. COUNT STAR IN NULL COLUMNS



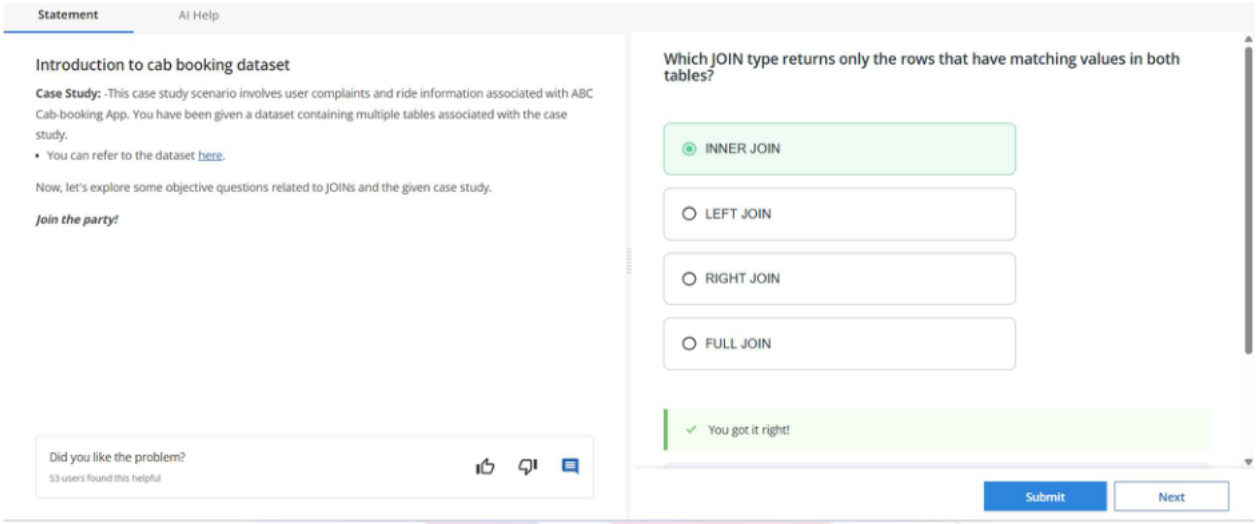
9.GROUP USERS



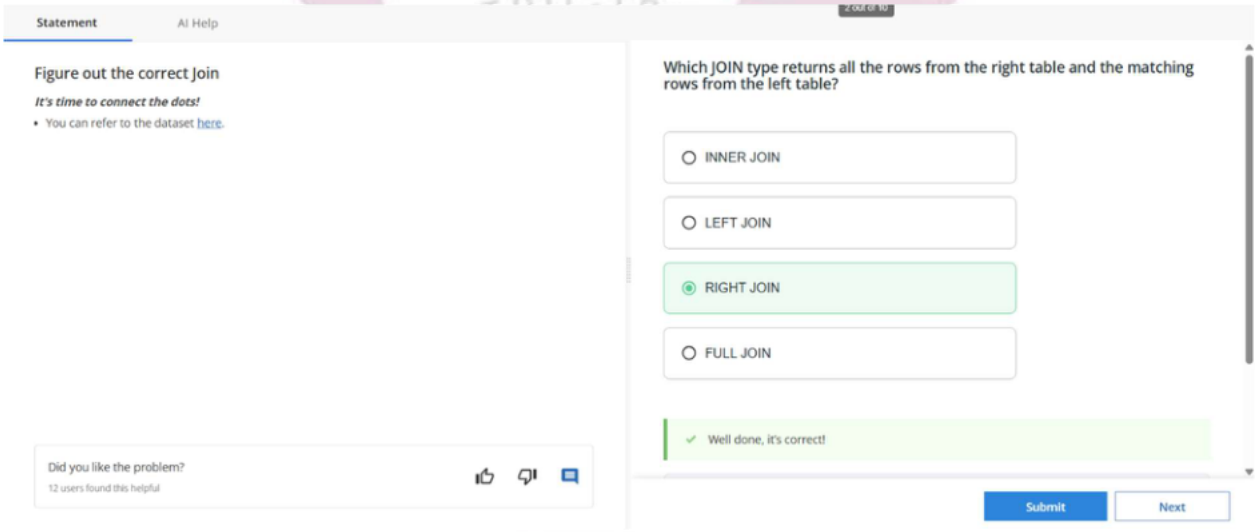
TRUTH-TOIL-TRIUMPH
AUTONOMOUS

CASE STUDY – ABC CAB BOOKIN APP

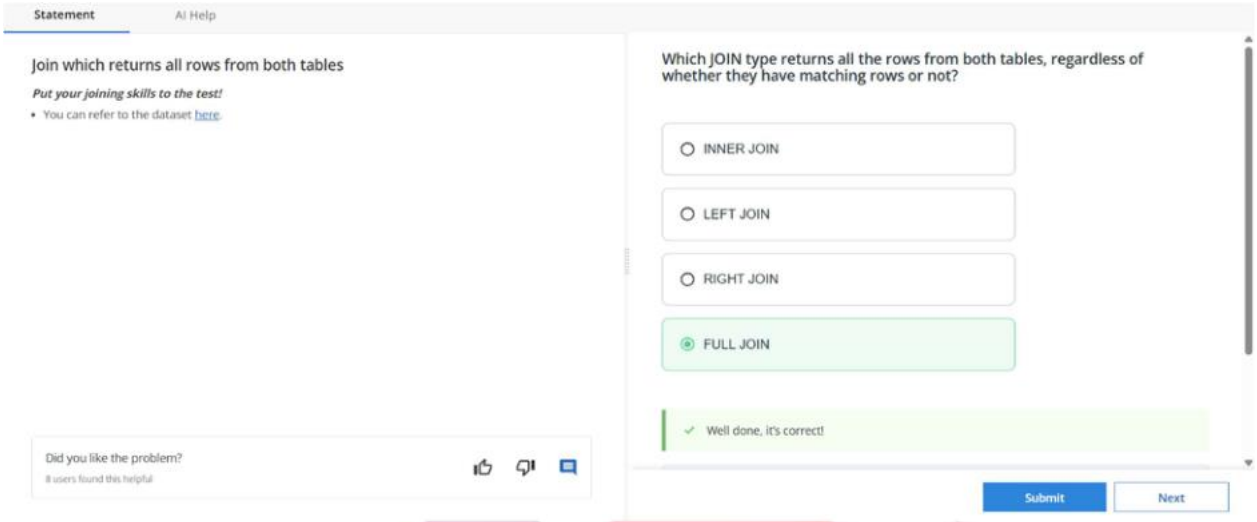
1.INTRODUCTION TO CAB BOOKING DATASET



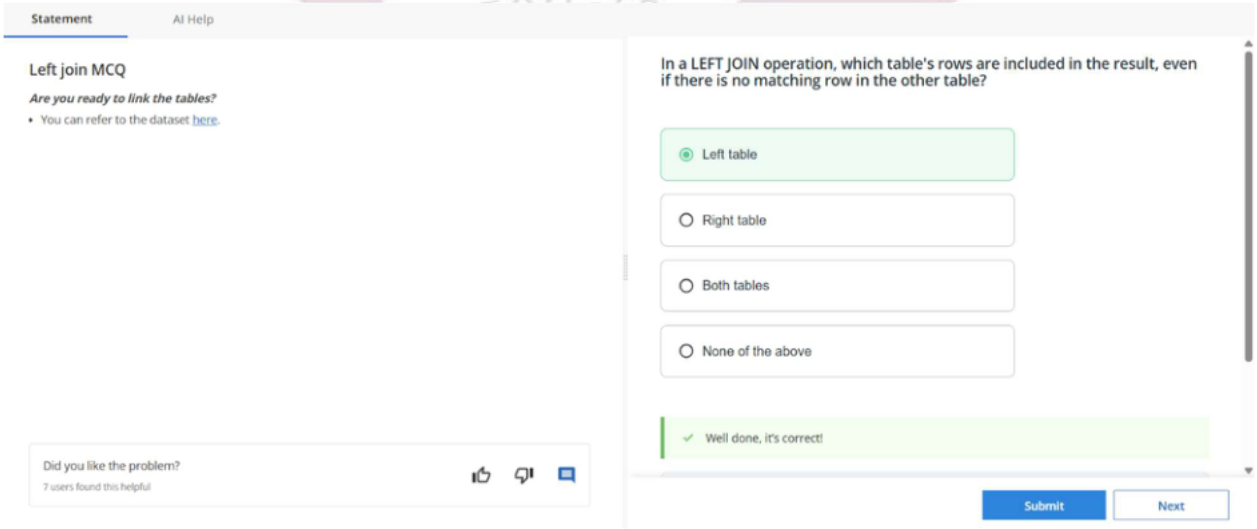
2. FIGURE OUT THE CORRECT JOIN



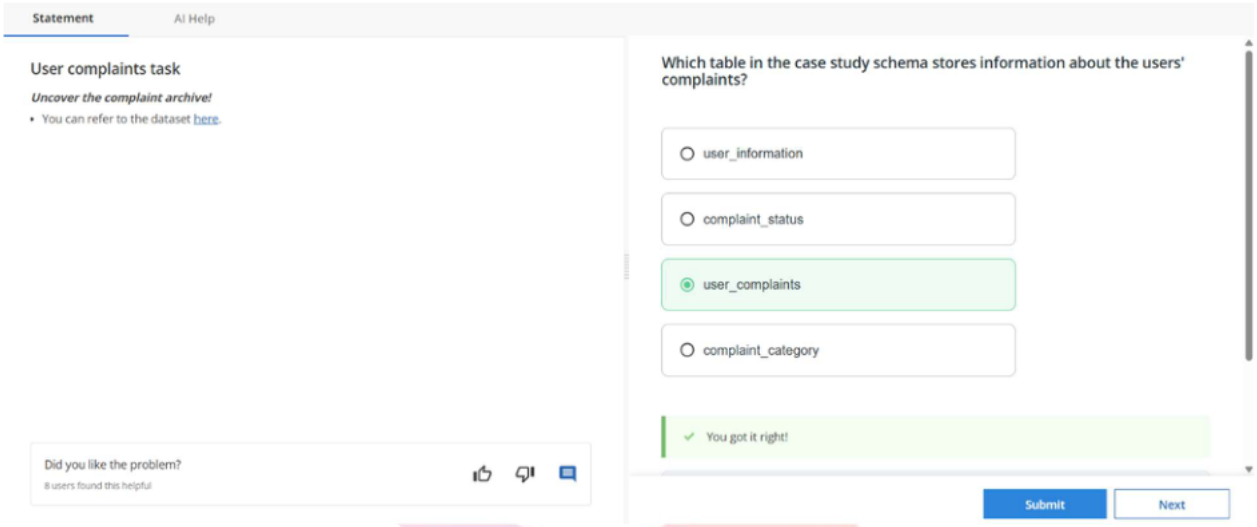
3. JOIN WHICH RETURN ALL ROWS FROM BOTH TABLES



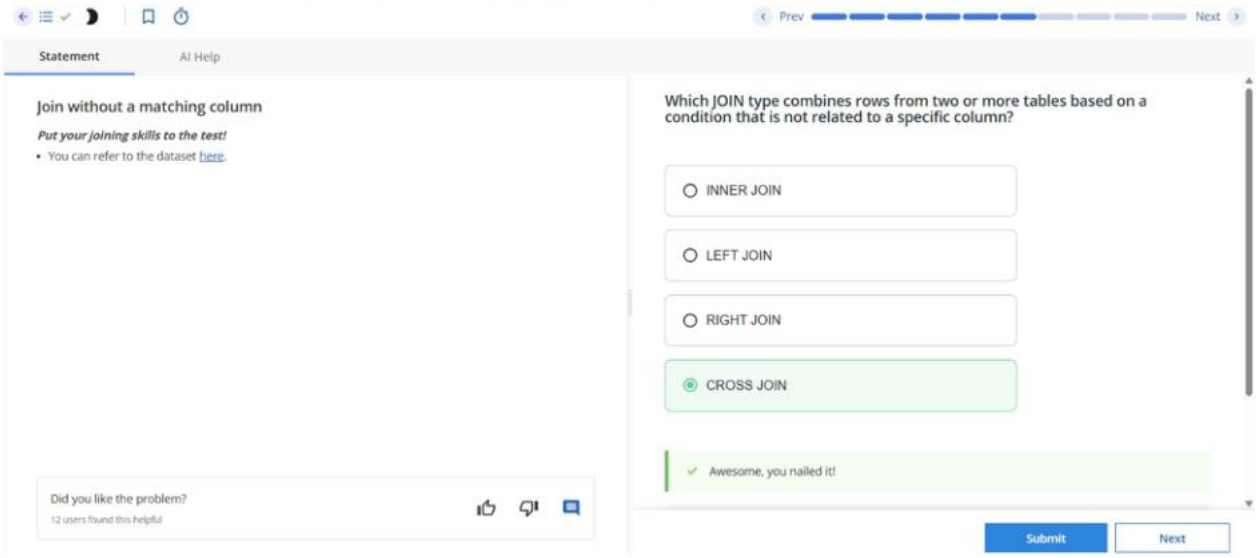
4. LEFT JOIN MCQ



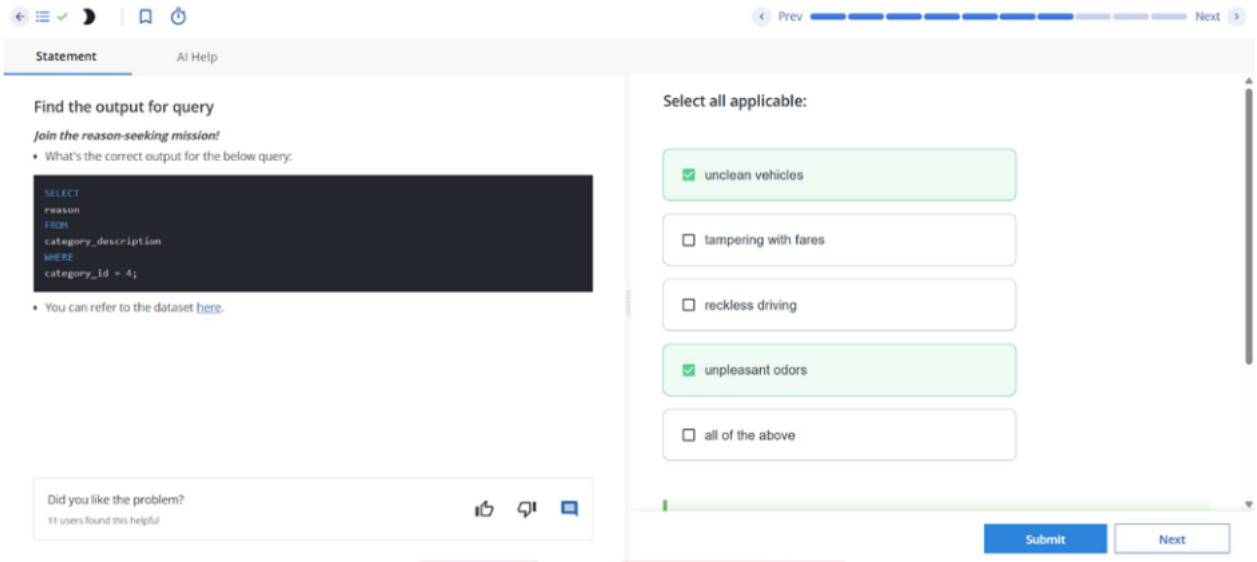
5. USER COMPLAINTS TASK



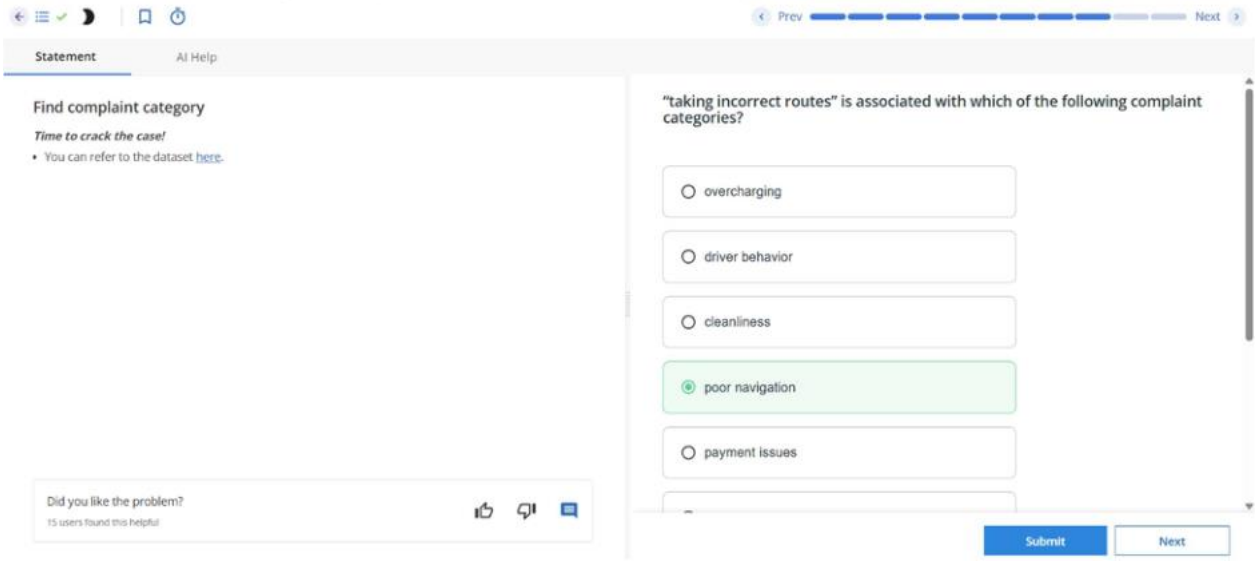
6. JOIN WITHOUT A MATCHING COLUMN



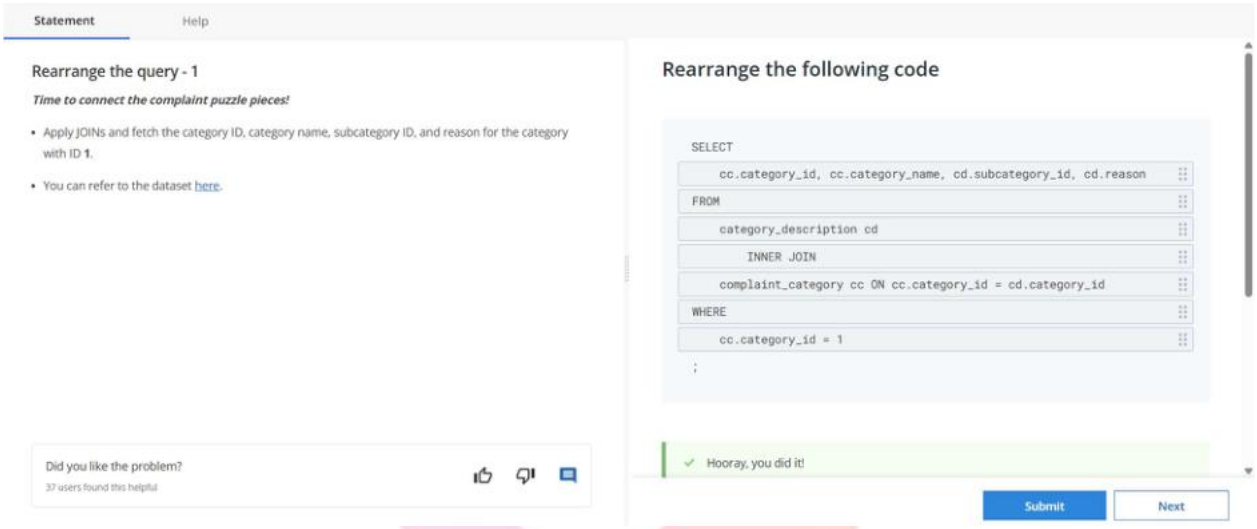
7. FIND THE OUTPUT FOR QUERY



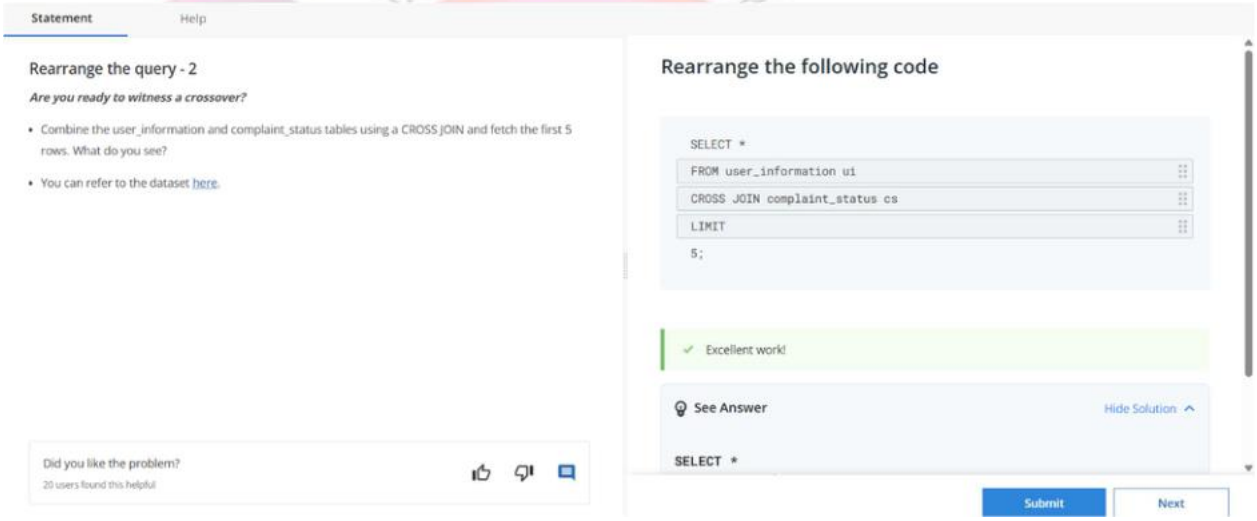
8. FIND COMPLAINT CATEGORY



9. REARRANGE THE QUERY - 1

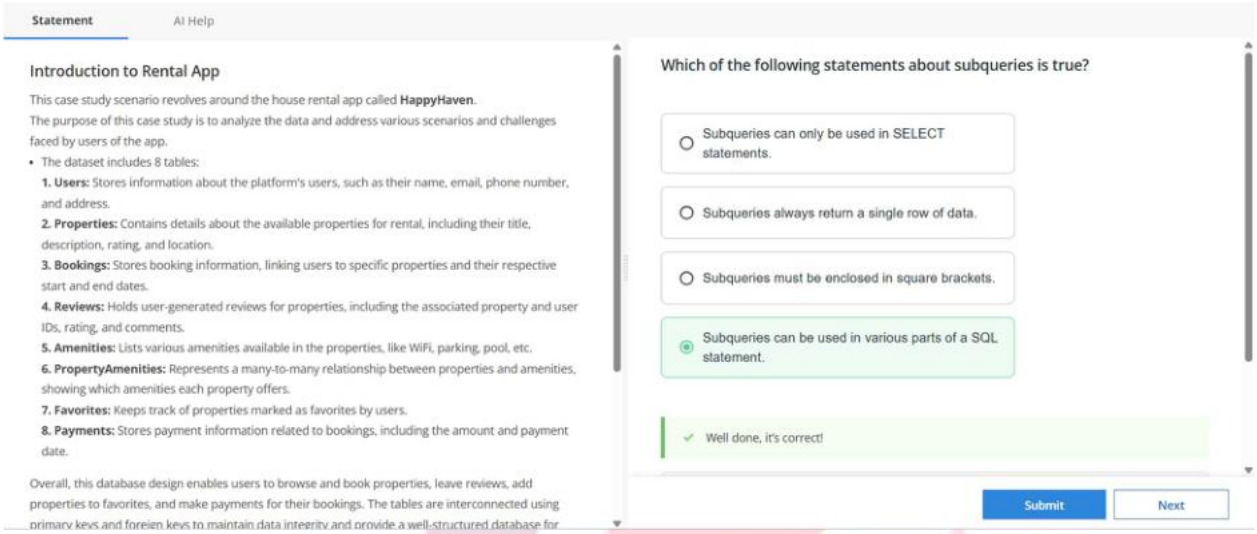


10 . REARRANGE THE QUERY - 2

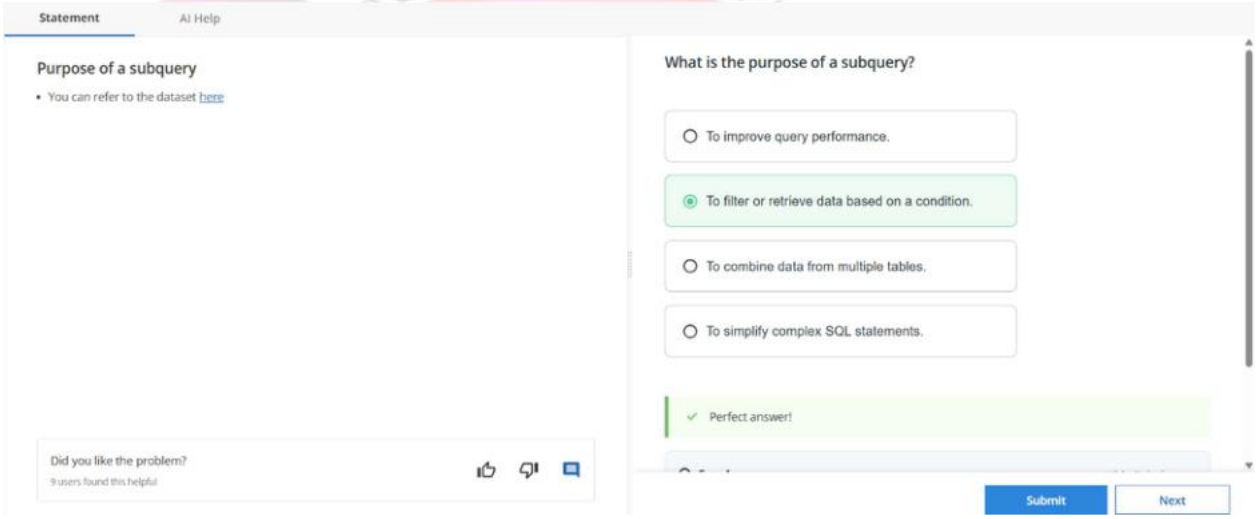


CASE STUDY – HAPPY HAVEN HOUSE RENTAL APP

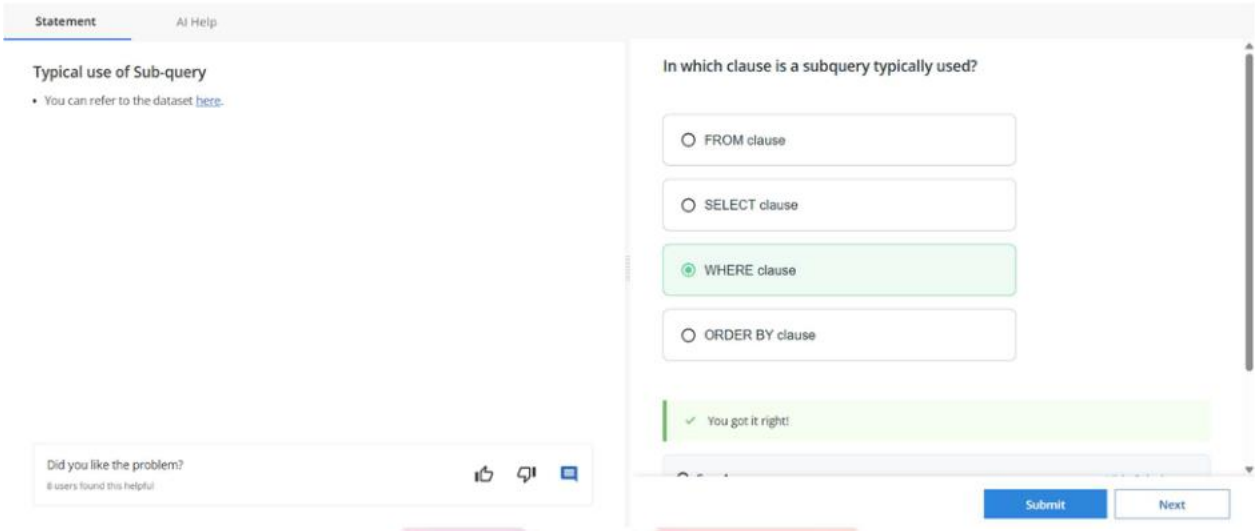
1.INTRODUCTION TO RENTAL APP



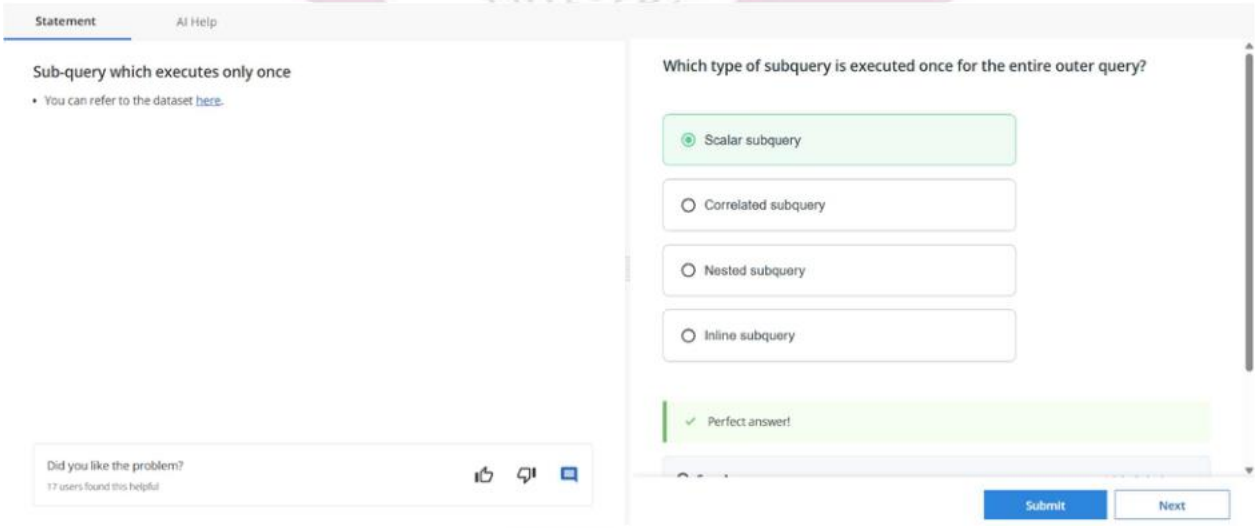
2. PURPOSE OF A SUBQUERY



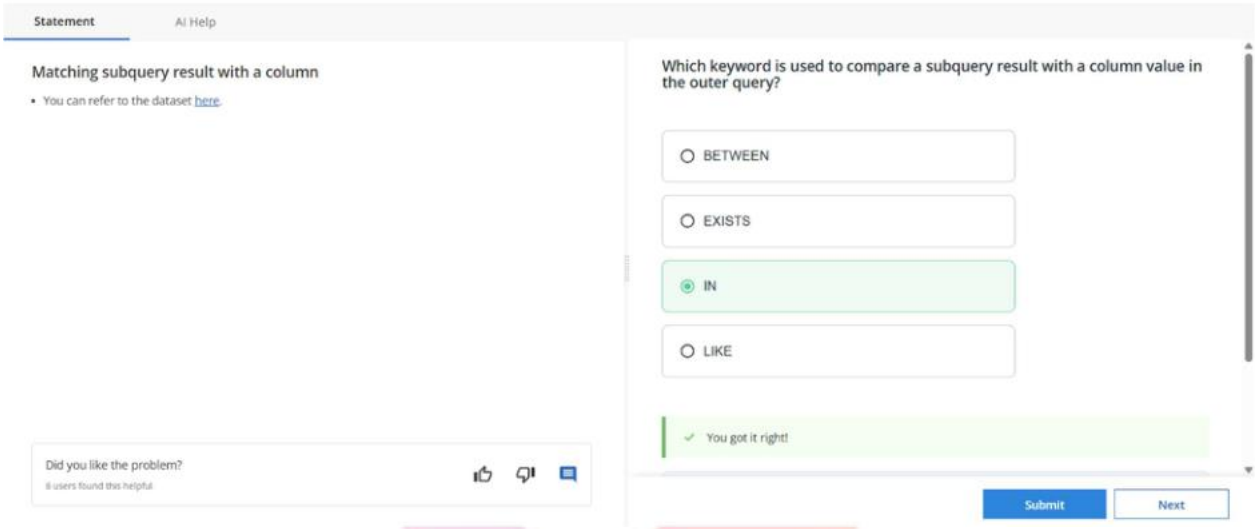
3. TYPICAL USE OF SUB-QUERY



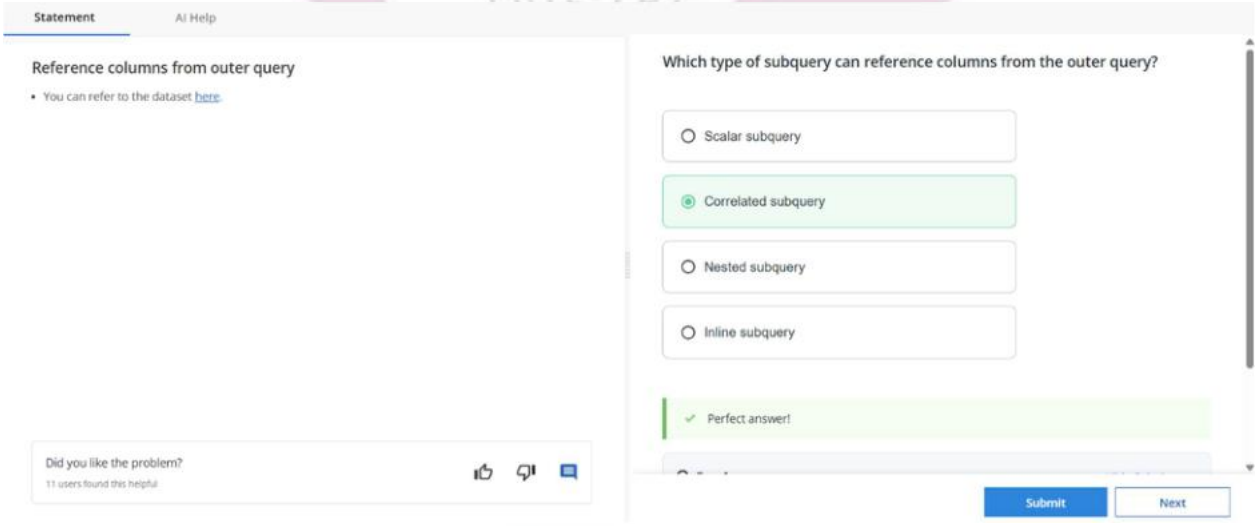
4. SUB-QUERY WHICH EXECUTES ONLY ONCE



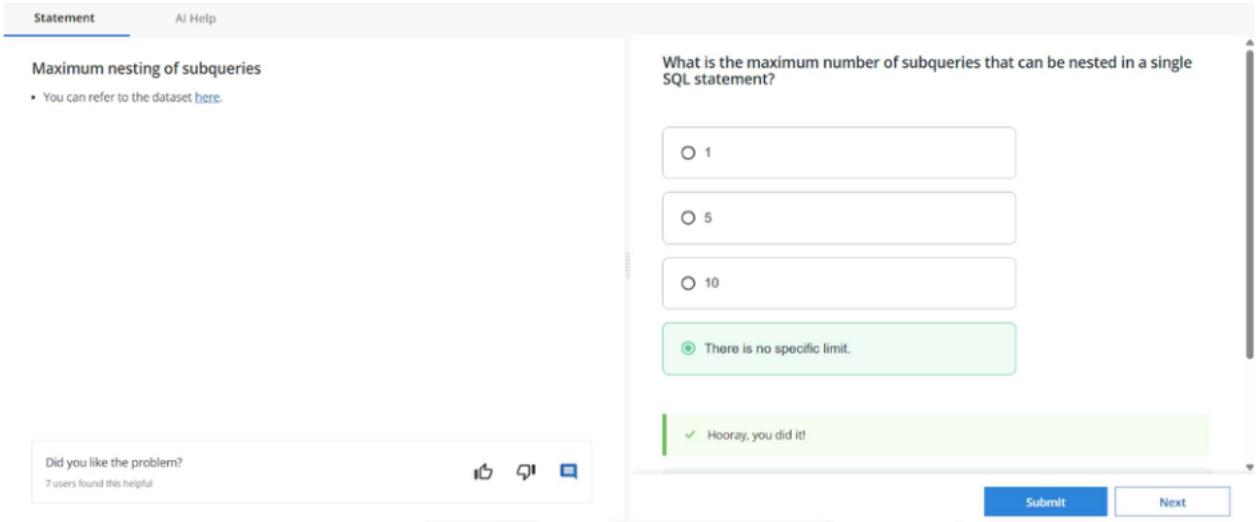
5. MATCHING SUBQUERY RESULT WITH A COLUMN



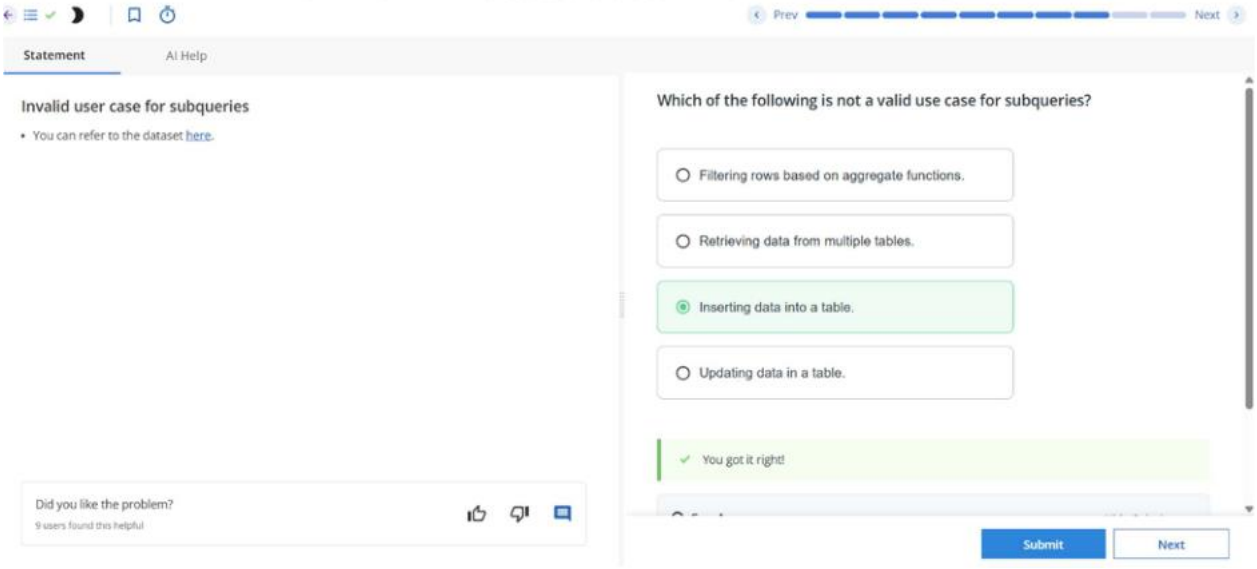
6. REFERENCE COLUMNS FROM OUTER QUERY



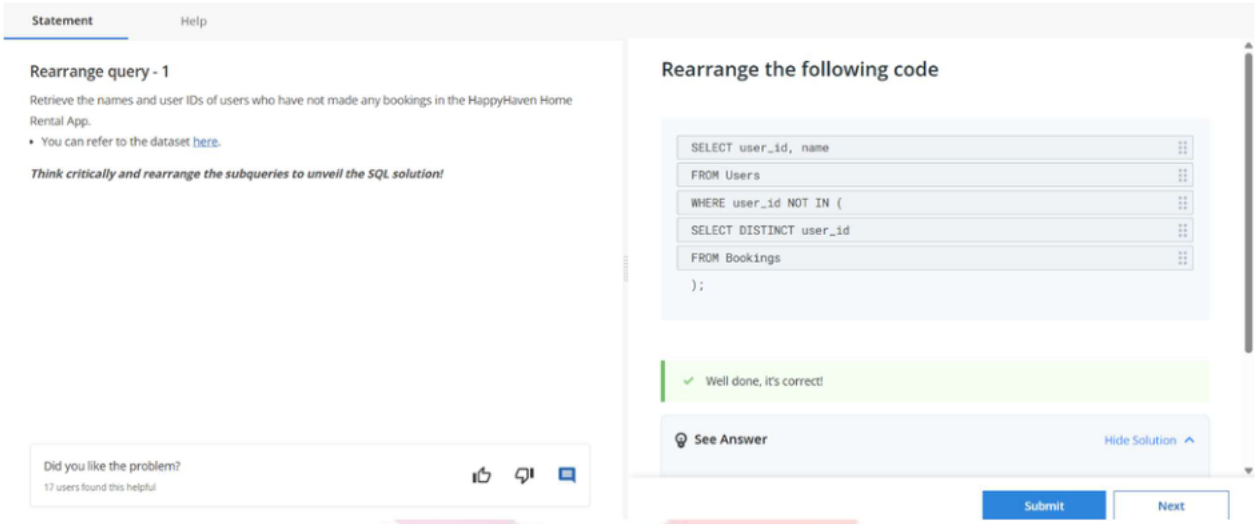
7. MAXIMUM NESTING OF SUBQUERIES



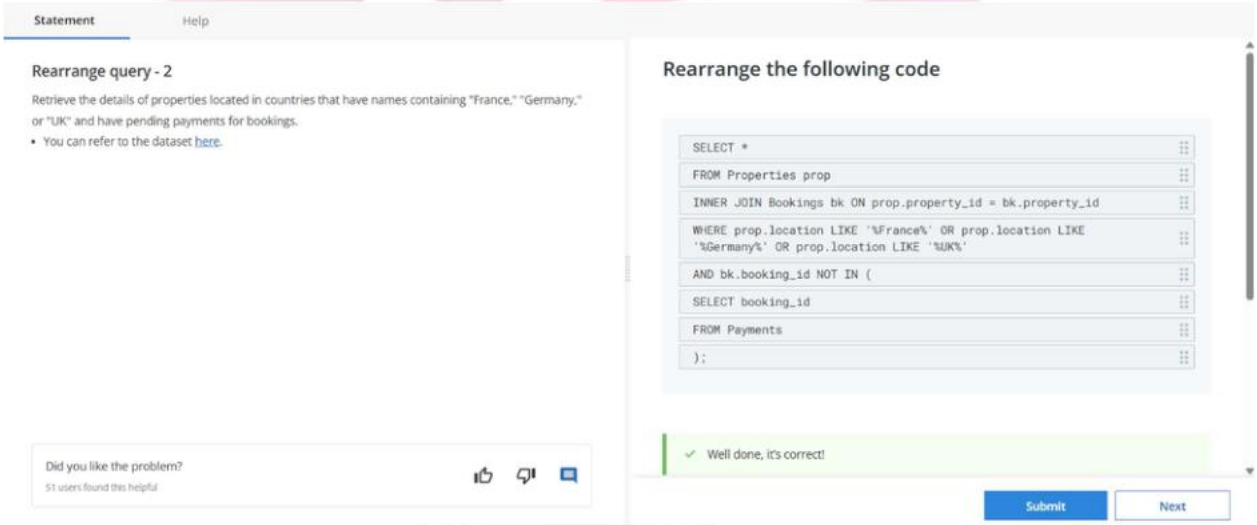
8. INVALID USER CASE FOR SUBQUERIES



9. REARRANGE QUERY-1

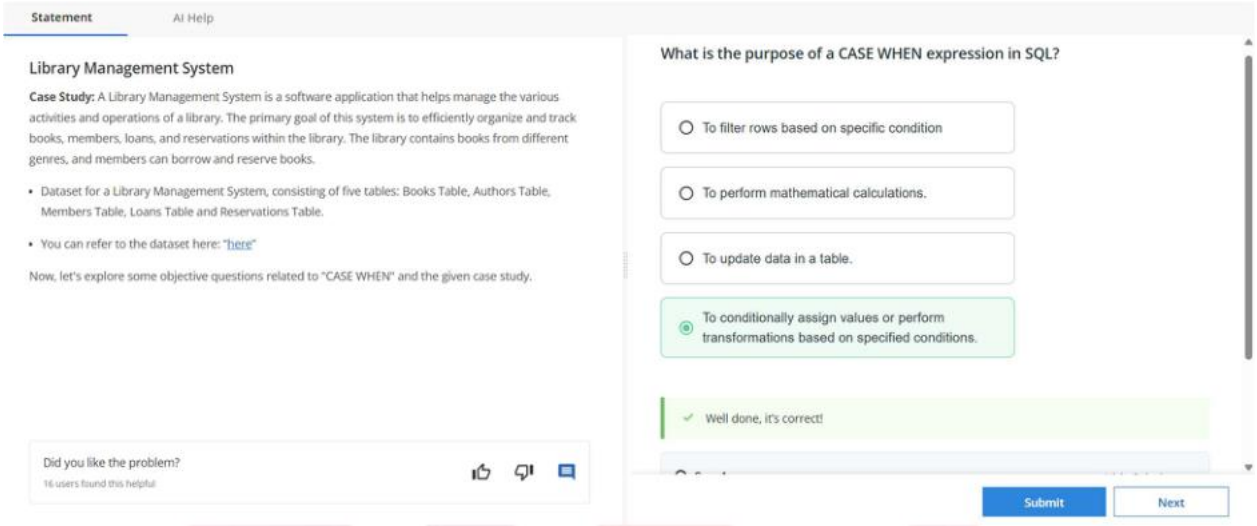


10. REARRANGE QUERY - 2

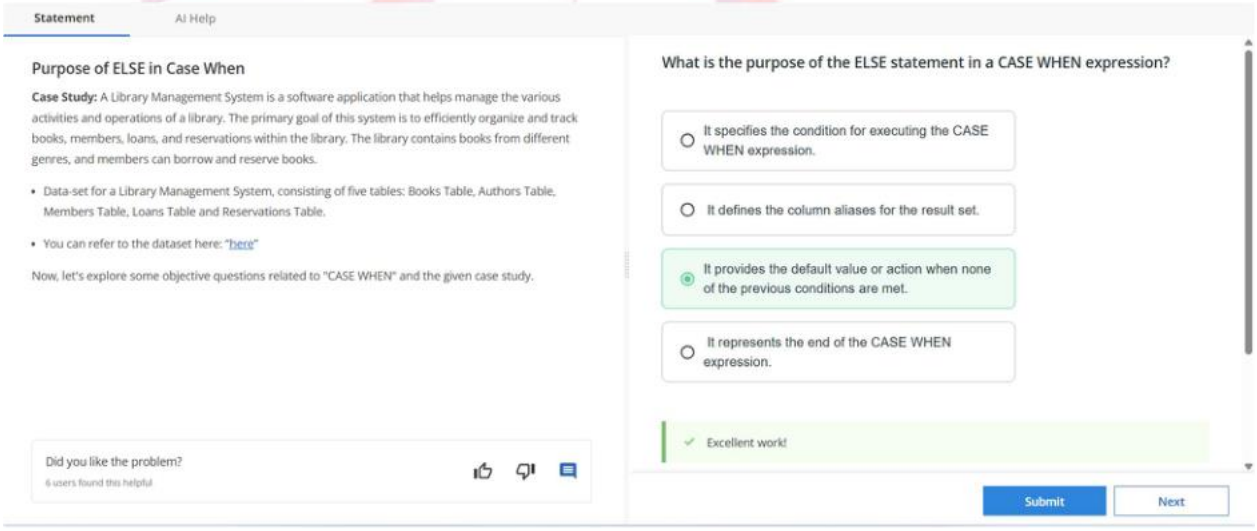


CASE STUDY – LIBRARY MANAGEMENT SYSTEM

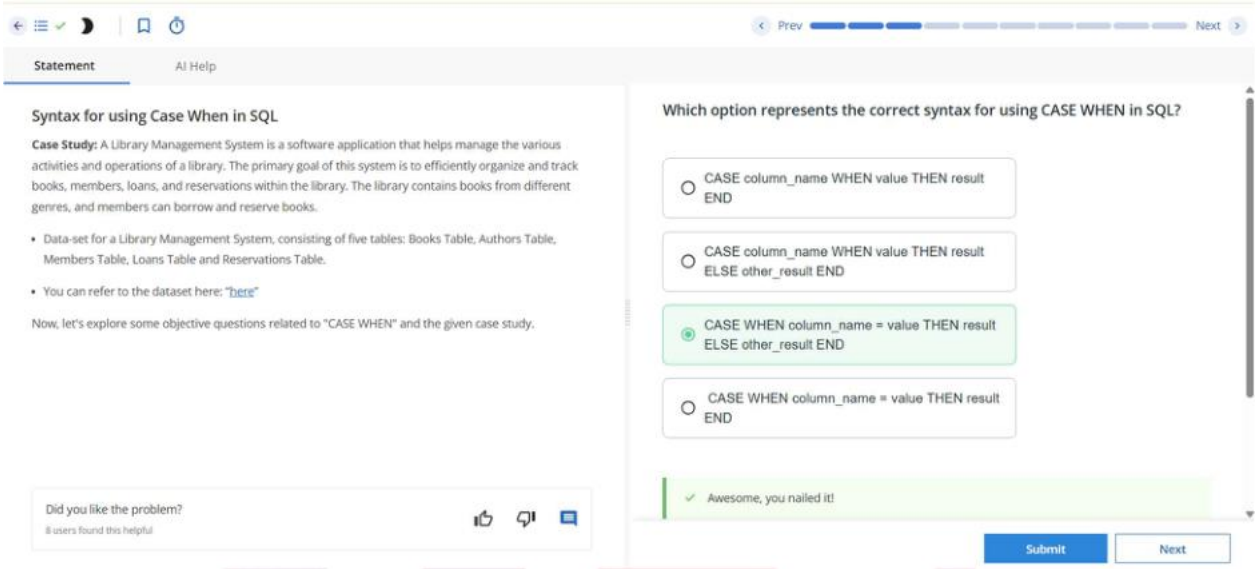
1.LIBRARY MANAGEMENT SYSTEM



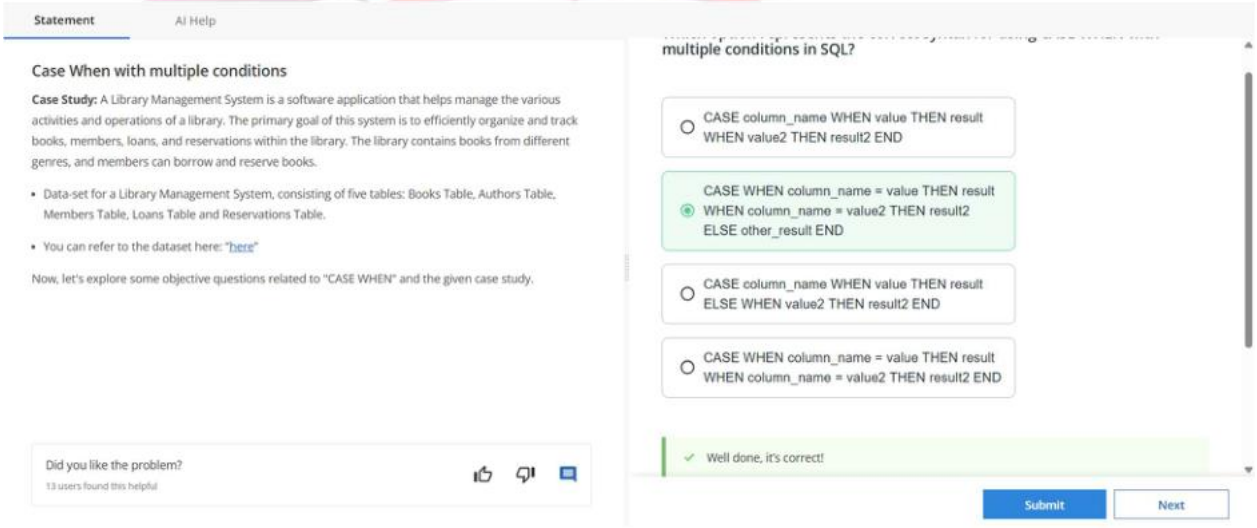
2.PURPOSE OF ELSE IN CASE WHEN



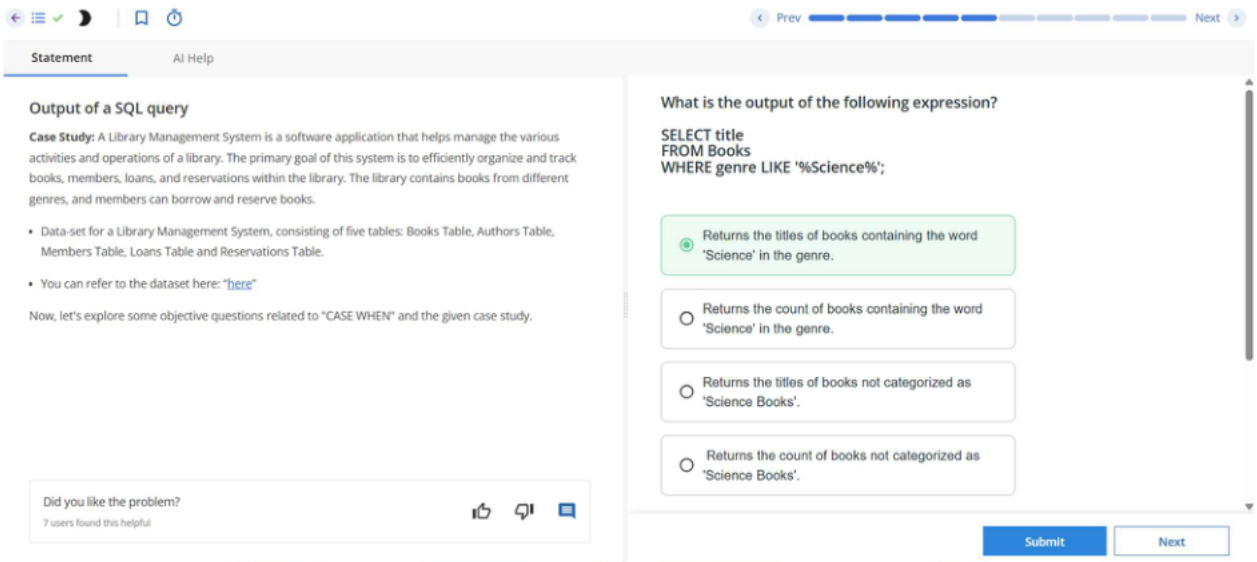
3.SYNTAX FOR USING CASE WHEN IN SQL



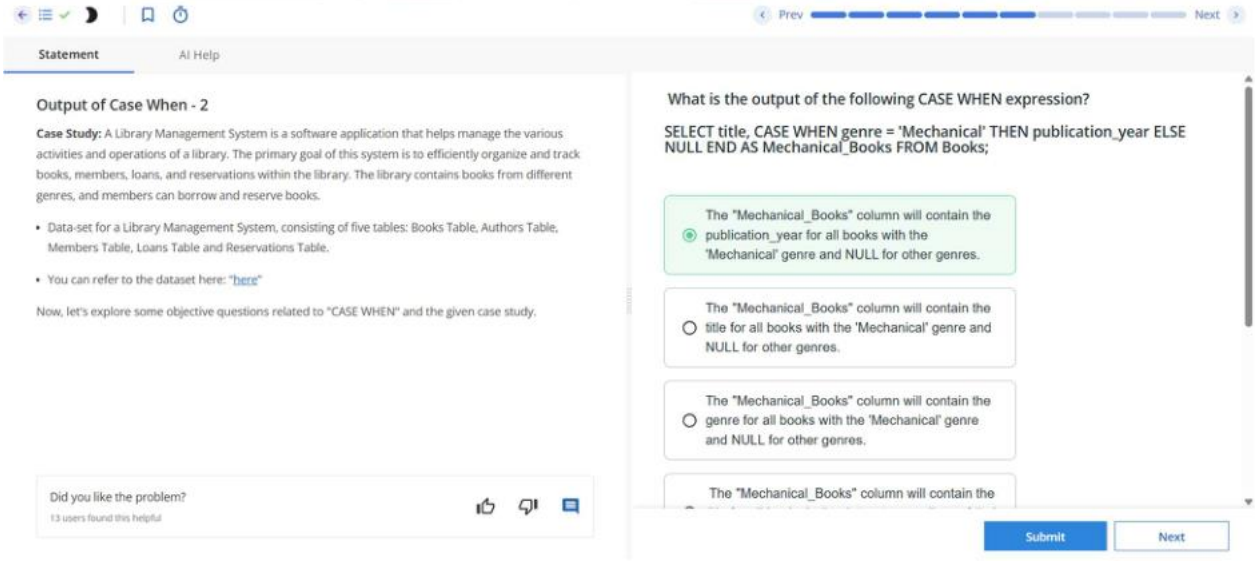
4. CASE WHEN WITH MULTIPLE CONDITIONS



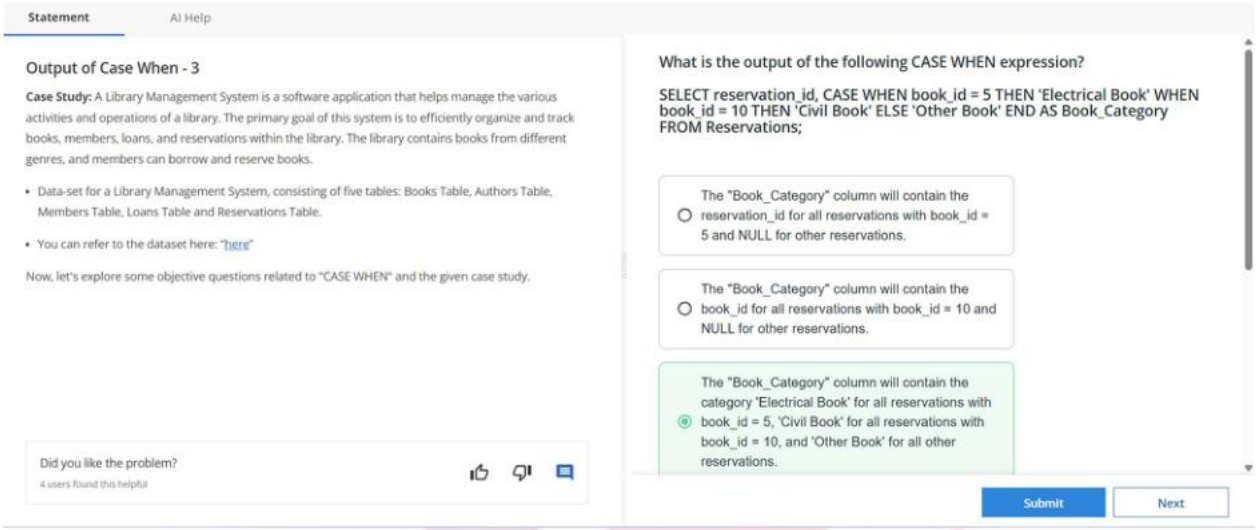
5. OUTPUT OF A SQL QUERY



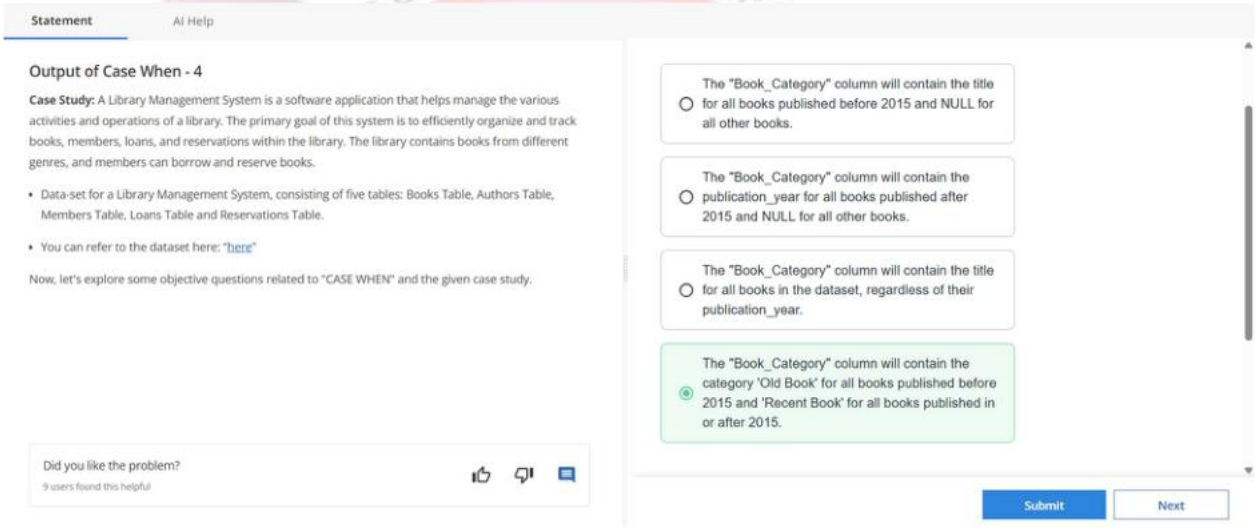
6. OUTPUT OF CASE WHEN - 2



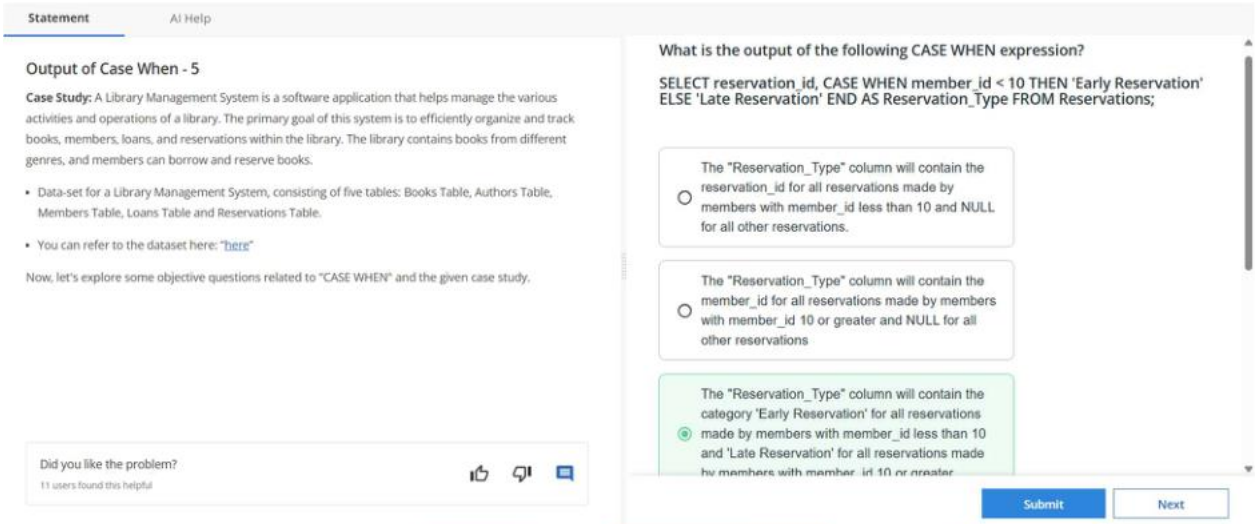
7. OUTPUT OF CASE WHEN - 3



8. OUTPUT OF CASE WHEN - 4



9. OUTPUT OF CASE WHEN -5



10. OUTPUT OF CASE WHEN - 6

